Vidyalankar Institute of Technology (VIT)

Wadala (W), Mumbai – 400 037 | Accredited A+ by NAAC

Department of Information Technology

Subject	Data Warehousing and Mining (DWM – PCIT10T / PCIT10P)		
Experiment No.	01	**Semester / Division**	T.E. Semester V | TEINFT - A (Odd Sem 2026-2027)
Laboratory Batch	Batch - 3	**Date of Performance**	—
Student 1	Manthan Shah (Roll No: 24101A0078)	**Student 2**	Ishan Gurav (Roll No: 24101A0077)
Professor In-charge	Dr. Varsha Bhosale		

Experiment 01: Build Data Warehouse / Data Mart & Design Dimensional Data Models

1. Aim / Objective

To design and implement a **Data Warehouse** for the Stock Market dataset using three dimensional modelling techniques: *Star Schema*, *Snowflake Schema*, and *Fact Constellation (Galaxy) Schema*. The warehouse stores daily trade facts (Open, High, Low, Close prices, Volume, Daily Return) together with conformed time, stock, and exchange dimensions.

2. Theory

2.1 Data Warehouse

A *Data Warehouse (DW)* is a subject-oriented, integrated, time-variant, and non-volatile collection of data used to support management decision-making (Inmon, 1992). It separates analytical workloads from operational databases (OLTP) by storing pre-aggregated, historical data in a schema optimised for read-heavy OLAP queries.

2.2 Star Schema

The **Star Schema** is the simplest dimensional model. A central *Fact table* (`Fact_DailyTrades`) holds quantitative trade metrics and foreign keys to denormalized *Dimension tables* (`Dim_Time`, `Dim_Stock`, `Dim_Exchange`). Queries involve at most one join per dimension, delivering the fastest analytical performance.

2.3 Snowflake Schema

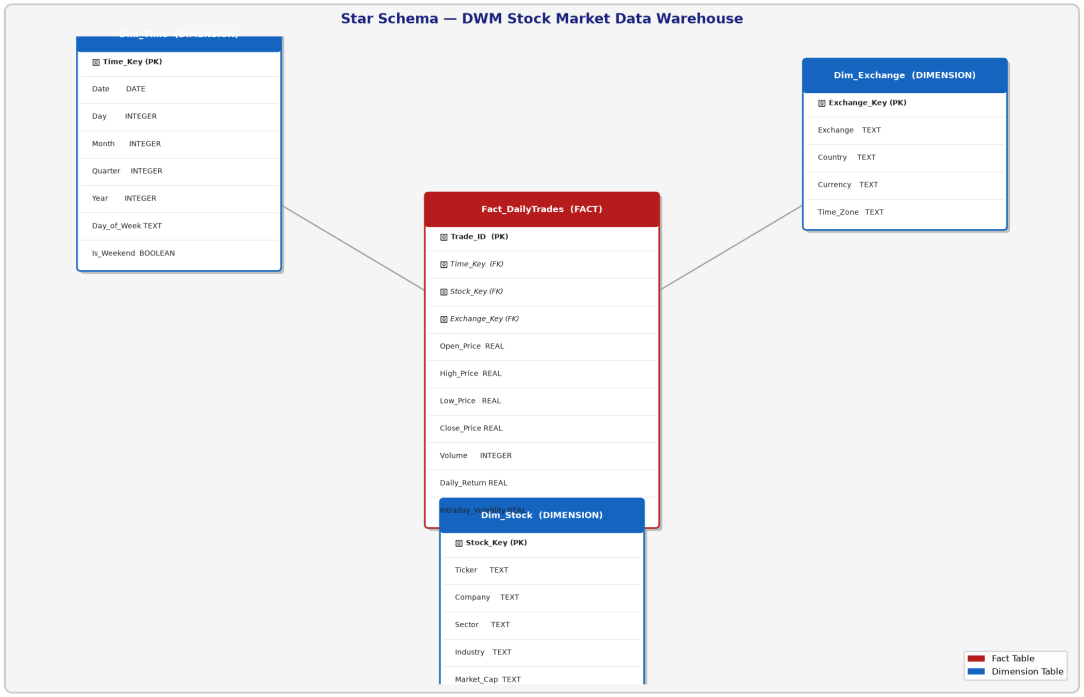
The **Snowflake Schema** extends the star by further normalizing dimension tables. `Dim_Stock_Snowflake` references a separate `Dim_Sector` sub-dimension, eliminating redundant sector/industry storage at the cost of additional joins.

2.4 Fact Constellation (Galaxy) Schema

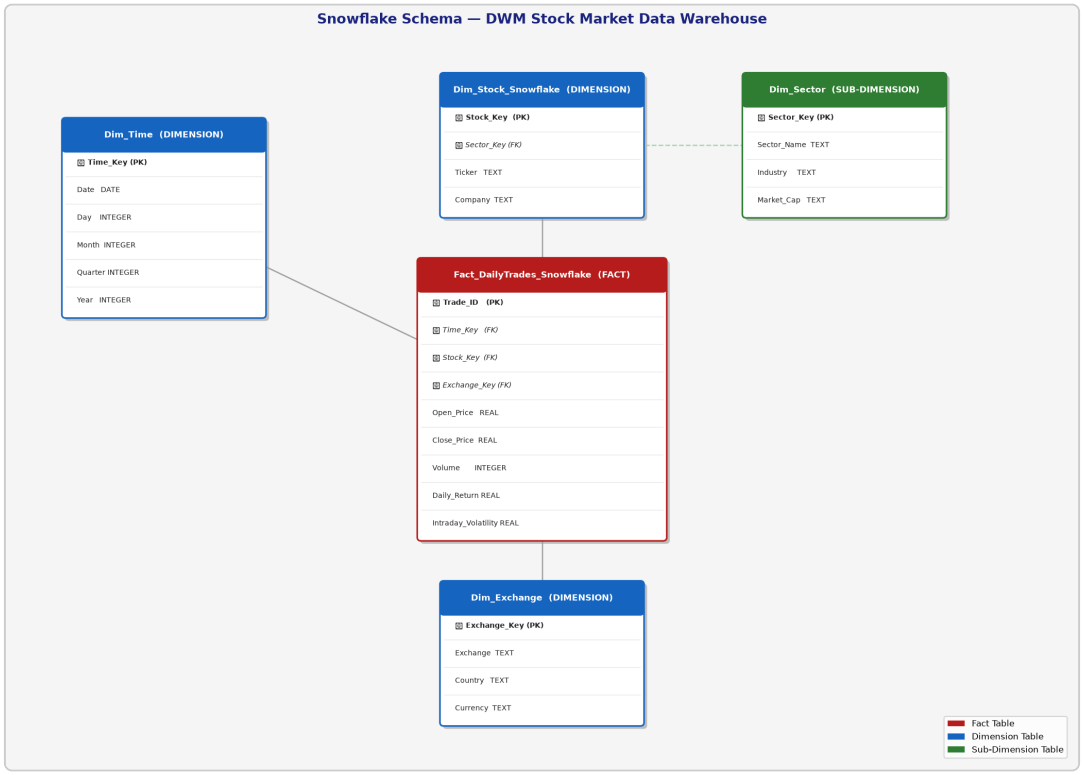
The **Fact Constellation** supports multiple fact tables sharing *conformed dimensions*. Here, `Fact_DailyTrades` and `Fact_QuarterlyFinancials` both reference `Dim_Time` and `Dim_Stock`, enabling cross-fact drill-across OLAP queries.

3. ER Diagrams

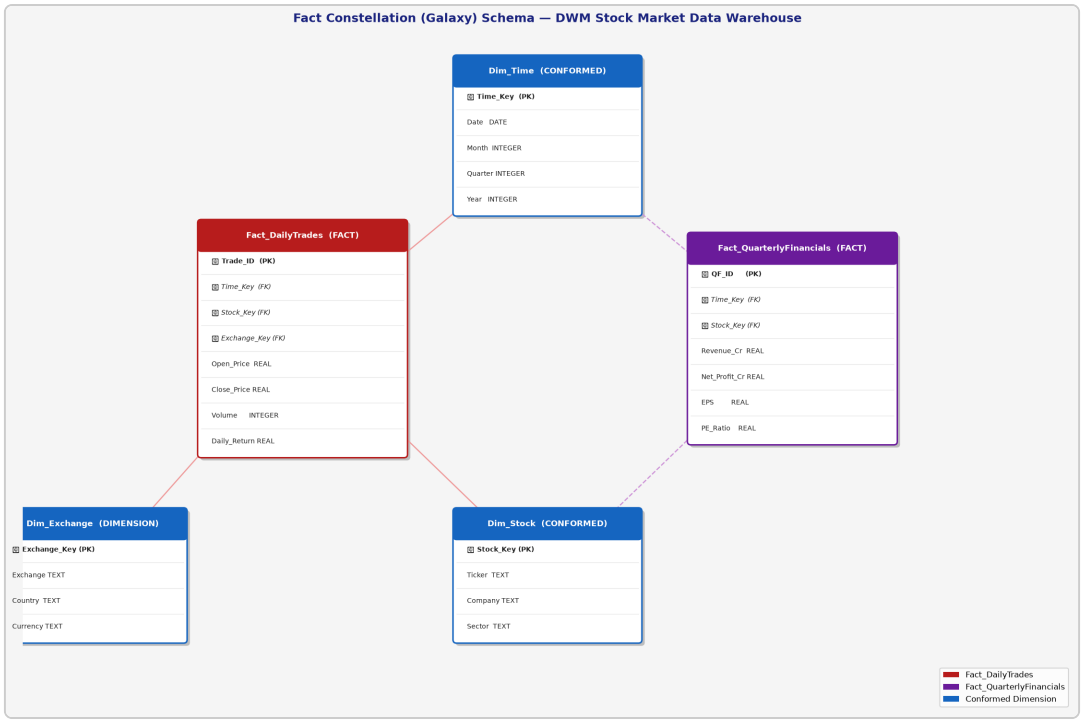
3.1 Star Schema



3.2 Snowflake Schema



3.3 Fact Constellation (Galaxy) Schema



4. DDL Scripts

4.1 Star Schema DDL

```
-- ═══════════════════════════════════════════════════════
--  STAR SCHEMA DDL
-- ═══════════════════════════════════════════════════════
```

```
CREATE TABLE IF NOT EXISTS Dim_Time (
    Time_Key    INTEGER PRIMARY KEY AUTOINCREMENT,
    Date        DATE    NOT NULL UNIQUE,
    Day         INTEGER,
    Month       INTEGER,
    Quarter     INTEGER,
    Year        INTEGER,
    Day_of_Week TEXT,
    Is_Weekend  BOOLEAN
);

CREATE TABLE IF NOT EXISTS Dim_Stock (
    Stock_Key   INTEGER PRIMARY KEY AUTOINCREMENT,
    Ticker      TEXT    NOT NULL UNIQUE,
    Company     TEXT,
    Sector      TEXT,
    Industry    TEXT,
    Market_Cap  TEXT
);

CREATE TABLE IF NOT EXISTS Dim_Exchange (
    Exchange_Key  INTEGER PRIMARY KEY AUTOINCREMENT,
    Exchange      TEXT    NOT NULL UNIQUE,
    Country       TEXT,
    Currency      TEXT,
    Time_Zone     TEXT
);

CREATE TABLE IF NOT EXISTS Fact_DailyTrades (
    Trade_ID            INTEGER PRIMARY KEY AUTOINCREMENT,
    Time_Key            INTEGER REFERENCES Dim_Time(Time_Key),
    Stock_Key           INTEGER REFERENCES Dim_Stock(Stock_Key),
    Exchange_Key        INTEGER REFERENCES Dim_Exchange(Exchange_Key),
    Open_Price          REAL,
    High_Price          REAL,
    Low_Price           REAL,
    Close_Price         REAL,
    Volume              INTEGER,
    Daily_Return        REAL,
    Intraday_Volatility REAL
);
```

4.2 Snowflake Schema DDL

```
-- ═══════════════════════════════════════════
--  SNOWFLAKE SCHEMA DDL
-- ═══════════════════════════════════════════

CREATE TABLE IF NOT EXISTS Dim_Sector (
    Sector_Key  INTEGER PRIMARY KEY AUTOINCREMENT,
    Sector_Name TEXT    NOT NULL UNIQUE,
    Industry    TEXT,
    Market_Cap  TEXT
);

CREATE TABLE IF NOT EXISTS Dim_Stock_Snowflake (
    Stock_Key   INTEGER PRIMARY KEY AUTOINCREMENT,
    Sector_Key  INTEGER REFERENCES Dim_Sector(Sector_Key),
    Ticker      TEXT    NOT NULL UNIQUE,
    Company     TEXT
);

CREATE TABLE IF NOT EXISTS Fact_DailyTrades_Snowflake (
    Trade_ID            INTEGER PRIMARY KEY AUTOINCREMENT,
    Time_Key            INTEGER REFERENCES Dim_Time(Time_Key),
    Stock_Key           INTEGER REFERENCES
Dim_Stock_Snowflake(Stock_Key),
    Exchange_Key        INTEGER REFERENCES Dim_Exchange(Exchange_Key),
```

```
    Open_Price          REAL,
    Close_Price         REAL,
    Volume              INTEGER,
    Daily_Return        REAL,
    Intraday_Volatility REAL
);
```

4.3 Fact Constellation DDL

```
-- ═══════════════════════════════════════════════
--  FACT CONSTELLATION (GALAXY) SCHEMA DDL
-- ═══════════════════════════════════════════════

-- Conformed dimensions: Dim_Time, Dim_Stock, Dim_Exchange  (reused
from Star)

CREATE TABLE IF NOT EXISTS Fact_QuarterlyFinancials (
    QF_ID         INTEGER PRIMARY KEY AUTOINCREMENT,
    Time_Key      INTEGER REFERENCES Dim_Time(Time_Key),
    Stock_Key     INTEGER REFERENCES Dim_Stock(Stock_Key),
    Revenue_Cr    REAL,
    Net_Profit_Cr REAL,
    EPS           REAL,
    PE_Ratio      REAL
);

-- Fact_DailyTrades already created under Star Schema (shared via
conformed dims)
```

5. Data Loading Summary

Table	Rows Loaded	Description
Dim_Time	740	Unique trading dates
Dim_Stock	20	Unique stock tickers
Dim_Exchange	1	Unique exchanges
Fact_DailyTrades	14,800	Total daily trade records

Conclusion: The Star, Snowflake, and Fact Constellation schemas were successfully designed and implemented for the stock market dataset. The Star Schema offers optimal query performance; the Snowflake Schema reduces storage redundancy by normalising sector data; and the Galaxy Schema enables multi-fact analytical queries using conformed dimensions shared across daily trades and quarterly financials.

Vidyalankar Institute of Technology (VIT)

Wadala (W), Mumbai – 400 037 | Accredited A+ by NAAC

Department of Information Technology

Subject	Data Warehousing and Mining (DWM – PCIT10T / PCIT10P)		
Experiment No.	02	**Semester / Division**	T.E. Semester V | TEINFT - A (Odd Sem 2026-2027)
Laboratory Batch	Batch - 3	**Date of Performance**	—
Student 1	Manthan Shah (Roll No: 24101A0078)	**Student 2**	Ishan Gurav (Roll No: 24101A0077)
Professor In-charge	Dr. Varsha Bhosale		

Experiment 02: OLAP Operations on the Dimensional Model

1. Aim / Objective

To perform standard OLAP (Online Analytical Processing) operations — **Slice, Dice, Roll-Up, Drill-Down, Pivot,** and **Drill-Across** — on the Stock Market Data Warehouse built in Experiment 01, using SQL queries executed against a SQLite in-memory warehouse populated from `stock_market_data.csv`.

2. Theory

2.1 OLAP Concepts

OLAP enables multidimensional analysis of business data. Operations traverse the *data cube* defined by the warehouse dimensions (Time, Stock, Exchange).

Operation	Description
Slice	Selects a single value of one dimension, producing a sub-cube.
Dice	Applies range/value filters on two or more dimensions.
Roll-Up	Aggregates data up a hierarchy (Month → Quarter → Year).
Drill-Down	Decomposes aggregated data to a finer granularity.
Pivot	Rotates the data cube to present a cross-tabulated view.
Drill-Across	Queries across multiple fact tables sharing conformed dimensions.

3. OLAP Query Results

3.1 Slice Operation

Extracts a single dimension value — all trades for the NSE exchange.

```
SELECT t.Year, s.Ticker, s.Sector,
       ROUND(AVG(f.Close_Price), 2)   AS Avg_Close,
       ROUND(AVG(f.Daily_Return), 4)  AS Avg_Return,
       SUM(f.Volume)                  AS Total_Volume
  FROM Fact_DailyTrades f
  JOIN Dim_Time     t ON f.Time_Key     = t.Time_Key
  JOIN Dim_Stock    s ON f.Stock_Key    = s.Stock_Key
  JOIN Dim_Exchange e ON f.Exchange_Key = e.Exchange_Key
 WHERE e.Exchange = 'NSE'
 GROUP BY t.Year, s.Ticker, s.Sector
 ORDER BY t.Year, s.Ticker
 LIMIT 10;
```

Result:

Year	Ticker	Sector	Avg_Close	Avg_Return	Total_Volume
2023	BAJAJ-AUTO	Auto	5531.28	0.3692	26873761
2023	BRITANNIA	FMCG	4574.6	0.2342	17371937
2023	HCLTECH	IT	1191.07	0.1342	129286188
2023	HDFCBANK	Banking	747.83	0.0275	1728255410
2023	HINDUNILVR	FMCG	2400.49	0.1179	83087327
2023	ICICIBANK	Banking	936.7	0.0062	721050143
2023	INFY	IT	1334.7	0.0834	294771285
2023	ITC	FMCG	381.75	-0.0942	557655398
2023	KOTAKBANK	Banking	355.43	0.0283	877373290
2023	M&M	Auto	1551.88	0.027	129424567

3.2 Dice Operation

Applies filters on two+ dimensions — IT sector stocks in 2023 with positive returns.

```
SELECT t.Year, t.Quarter, s.Ticker, s.Sector,
       ROUND(AVG(f.Close_Price), 2)   AS Avg_Close,
       ROUND(AVG(f.Daily_Return), 4)  AS Avg_Return,
       SUM(f.Volume)                  AS Total_Volume
  FROM Fact_DailyTrades f
  JOIN Dim_Time  t ON f.Time_Key  = t.Time_Key
  JOIN Dim_Stock s ON f.Stock_Key = s.Stock_Key
 WHERE s.Sector = 'IT'
   AND t.Year   = 2023
   AND f.Daily_Return > 0
 GROUP BY t.Year, t.Quarter, s.Ticker
 ORDER BY t.Quarter, Avg_Return DESC
 LIMIT 10;
```

Result:

Year	Quarter	Ticker	Sector	Avg_Close	Avg_Return	Total_Volume
2023	4	WIPRO	IT	196.22	1.6579	388519914
2023	4	HCLTECH	IT	1200.61	1.0567	77767475
2023	4	INFY	IT	1344.17	1.0012	164525356
2023	4	TCS	IT	3268.61	0.8569	53685670

3.3 Roll-Up Operation

Aggregates from Month → Quarter → Year hierarchy.

```
SELECT t.Year, t.Quarter,
       ROUND(AVG(f.Close_Price),  2)  AS Avg_Close,
       ROUND(AVG(f.Daily_Return), 4)  AS Avg_Daily_Return,
       SUM(f.Volume)                  AS Total_Volume,
       COUNT(*)                       AS Trade_Days
  FROM Fact_DailyTrades f
  JOIN Dim_Time t ON f.Time_Key = t.Time_Key
 GROUP BY t.Year, t.Quarter
 ORDER BY t.Year, t.Quarter
 LIMIT 12;
```

Result:

Year	Quarter	Avg_Close	Avg_Daily_Return	Total_Volume	Trade_Days
2023.0	4.0	1839.23	0.0597	9774429862.0	1060.0
2024.0	1.0	2060.93	-0.0139	15577733283.0	1200.0
2024.0	2.0	2235.85	-0.0657	15869722414.0	1200.0
2024.0	3.0	2453.39	0.0291	14911286734.0	1280.0
2024.0	4.0	2339.47	-0.2116	11996598216.0	1240.0
2025.0	1.0	2216.11	-0.089	11113388964.0	1240.0
2025.0	2.0	2249.13	0.1214	11742491800.0	1220.0
2025.0	3.0	2388.7	-0.1293	9025015077.0	1280.0
2025.0	4.0	2547.37	0.0182	9050884518.0	1240.0
2026.0	1.0	2467.52	-0.1455	14077419796.0	1200.0
2026.0	2.0	2313.45	-0.0594	14836920973.0	1200.0
2026.0	3.0	2381.84	-0.098	10952232439.0	1300.0

3.4 Drill-Down Operation

Decomposes from Year → Quarter → Month for INFY.

```
SELECT t.Year, t.Quarter, t.Month,
       ROUND(AVG(f.Close_Price),  2) AS Avg_Close,
       ROUND(AVG(f.Daily_Return), 4) AS Avg_Return,
       SUM(f.Volume)                 AS Total_Volume
  FROM Fact_DailyTrades f
  JOIN Dim_Time  t ON f.Time_Key  = t.Time_Key
  JOIN Dim_Stock s ON f.Stock_Key = s.Stock_Key
 WHERE s.Ticker = 'INFY'
 GROUP BY t.Year, t.Quarter, t.Month
 ORDER BY t.Year, t.Quarter, t.Month
 LIMIT 12;
```

Result:

Year	Quarter	Month	Avg_Close	Avg_Return	Total_Volume
2023.0	4.0	10.0	1289.59	-0.1677	79507358.0
2023.0	4.0	11.0	1303.97	-0.023	85186084.0
2023.0	4.0	12.0	1394.75	0.353	130077843.0

Year	Quarter	Month	Avg_Close	Avg_Return	Total_Volume
2024.0	1.0	1.0	1469.26	0.0533	165823157.0
2024.0	1.0	2.0	1549.02	0.0133	101673806.0
2024.0	1.0	3.0	1459.2	-0.3456	135900753.0
2024.0	2.0	4.0	1342.27	-0.2435	185453567.0
2024.0	2.0	5.0	1326.73	0.0424	174897064.0
2024.0	2.0	6.0	1410.3	0.1263	177134468.0
2024.0	3.0	7.0	1634.49	0.4341	192519323.0
2024.0	3.0	8.0	1732.42	-0.07	124670491.0
2024.0	3.0	9.0	1801.31	-0.1538	134369302.0

3.5 Pivot Operation

Cross-tabulates average daily return per sector per year.

```
SELECT s.Sector,
       ROUND(AVG(CASE WHEN t.Year = 2021 THEN f.Daily_Return END), 4)
AS [2021],
       ROUND(AVG(CASE WHEN t.Year = 2022 THEN f.Daily_Return END), 4)
AS [2022],
       ROUND(AVG(CASE WHEN t.Year = 2023 THEN f.Daily_Return END), 4)
AS [2023],
       ROUND(AVG(CASE WHEN t.Year = 2024 THEN f.Daily_Return END), 4)
AS [2024]
  FROM Fact_DailyTrades f
  JOIN Dim_Stock s ON f.Stock_Key = s.Stock_Key
  JOIN Dim_Time  t ON f.Time_Key  = t.Time_Key
 GROUP BY s.Sector
 ORDER BY s.Sector;
```

Result:

Sector	2021	2022	2023	2024
Auto	None	None	0.0346	-0.0574
Banking	None	None	0.0008	0.0147
Energy	None	None	0.0768	-0.1276
FMCG	None	None	0.0801	-0.1353
IT	None	None	0.1064	-0.0203

3.6 Drill-Across Operation

Joins two fact tables (DailyTrades + QuarterlyFinancials) via conformed dims.

```
SELECT s.Ticker, s.Sector, t.Year, t.Quarter,
       ROUND(AVG(f.Daily_Return), 4) AS Avg_Daily_Return,
       ROUND(AVG(f.Volume),       0) AS Avg_Volume,
       ROUND(AVG(q.EPS),          2) AS Avg_EPS,
       ROUND(AVG(q.PE_Ratio),     2) AS Avg_PE
  FROM Fact_DailyTrades f
  JOIN Dim_Stock s ON f.Stock_Key = s.Stock_Key
  JOIN Dim_Time  t ON f.Time_Key  = t.Time_Key
  JOIN Fact_QuarterlyFinancials q
       ON q.Stock_Key = f.Stock_Key AND q.Time_Key = f.Time_Key
 GROUP BY s.Ticker, t.Year, t.Quarter
```

```
ORDER BY s.Ticker, t.Year, t.Quarter
LIMIT 10;
```

Result:

Ticker	Sector	Year	Quarter	Avg_Daily_Return	Avg_Volume	Avg_EPS	Avg_PE
BAJAJ-AUTO	Auto	2023	4	0.42	472168.0	29.88	48.73
BAJAJ-AUTO	Auto	2024	1	-1.02	253274.0	14.23	25.47
BAJAJ-AUTO	Auto	2024	2	-1.66	419820.0	8.32	17.82
BAJAJ-AUTO	Auto	2024	3	0.29	289355.0	6.36	21.96
BAJAJ-AUTO	Auto	2024	4	-2.07	670828.0	30.91	18.14
BAJAJ-AUTO	Auto	2025	1	-0.15	558535.0	71.77	24.11
BAJAJ-AUTO	Auto	2025	2	4.74	283618.0	59.33	14.55
BAJAJ-AUTO	Auto	2025	3	0.35	363782.0	5.07	45.11
BAJAJ-AUTO	Auto	2025	4	-1.07	551282.0	73.27	41.81
BAJAJ-AUTO	Auto	2026	1	2.06	306936.0	56.96	11.04

Conclusion: All six OLAP operations were successfully executed on the Stock Market Data Warehouse. Slice isolated NSE-listed stocks; Dice filtered IT-sector positive-return days in 2023; Roll-Up aggregated trade metrics to the quarterly level; Drill-Down decomposed INFY's performance month-by-month; Pivot cross-tabulated sector-wise average returns by year; and Drill-Across joined daily trade facts with quarterly financials via conformed dimensions, demonstrating the power of a multi-fact constellation schema.

Data Warehousing and Mining (DWM) — Experiment 03

Experiment Title: Data Preprocessing, Exploration and Visualization

Dataset: `stock_market_data.csv` (3 Years of Daily NSE Trading Data across 20 Major Indian Stocks & 5 Sectors)

1. Objectives

- Perform comprehensive **data preprocessing**, type normalization, and temporal feature engineering on the multi-sector Indian stock market trading dataset.
- Conduct **exploratory data analysis (EDA)** and statistical aggregation to understand price distributions, daily returns, trading volumes, and intraday volatility.
- Build **multi-dimensional visualizations** across stocks, sectors, and time horizons to uncover liquidity patterns, risk-return characteristics, outliers, and feature correlations.
- Prepare and validate the dataset for downstream data mining tasks (regression, classification, clustering, and association rule mining).

---

2. Library Imports

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Set seaborn whitegrid theme and global plot styling
sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams['figure.figsize'] = (11, 6)
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.titleweight'] = 'bold'
plt.rcParams['axes.labelsize'] = 12

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

---

3. Dataset Ingestion

Load `stock_market_data.csv` and inspect the initial records.

```
df = pd.read_csv('stock_market_data.csv')
print(f"Successfully loaded 'stock_market_data.csv' with {len(df):,} records.")
df.head()
```

```
Successfully loaded 'stock_market_data.csv' with 14,800 records.
```

	Date	Ticker	Sector	Exchange	Open_Price	High_Price	Low_Price	Close_Price	
0	2023-10-12	BAJAJ-AUTO	Auto	NSE	4851.06	4914.02	4818.72	4871.67	
1	2023-10-12	BRITANNIA	FMCG	NSE	4391.48	4400.68	4342.60	4371.11	
2	2023-10-12	HCLTECH	IT	NSE	1106.03	1109.18	1079.51	1087.15	
3	2023-10-12	HDFCBANK	Banking	NSE	735.43	743.05	733.28	742.02	2
4	2023-10-12	HINDUNILVR	FMCG	NSE	2413.48	2425.33	2398.55	2415.37	

---

4. Dataset Understanding

Inspect the dataset dimensions (`df.shape`), column names, data types, and structural summary (`df.info()`).

```
print("Dataset Shape (Rows, Columns):", df.shape)
print("\nColumn Names:", df.columns.tolist())
print("\nData Types:")
print(df.dtypes)
print("\nDataset Info:")
df.info()
```

```
Dataset Shape (Rows, Columns): (14800, 10)

Column Names: ['Date', 'Ticker', 'Sector', 'Exchange', 'Open_Price', 'High_Price',
'Low_Price', 'Close_Price', 'Volume', 'Daily_Return']

Data Types:
Date                str
Ticker              str
Sector              str
Exchange            str
Open_Price      float64
High_Price      float64
Low_Price       float64
Close_Price     float64
Volume            int64
Daily_Return    float64
dtype: object

Dataset Info:
<class 'pandas.DataFrame'>
RangeIndex: 14800 entries, 0 to 14799
Data columns (total 10 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Date          14800 non-null  str
 1   Ticker        14800 non-null  str
 2   Sector        14800 non-null  str
 3   Exchange      14800 non-null  str
 4   Open_Price    14800 non-null  float64
 5   High_Price    14800 non-null  float64
 6   Low_Price     14800 non-null  float64
 7   Close_Price   14800 non-null  float64
 8   Volume        14800 non-null  int64
 9   Daily_Return  14800 non-null  float64
dtypes: float64(5), int64(1), str(4)
memory usage: 1.1 MB
```

---

5. Statistical Summary

Display the five-number summary and categorical/numerical distribution statistics using `df.describe(include='all').T`.

```
df.describe(include='all').T
```

	count	unique	top	freq	mean	std	min	25
Date	14800	740	2023-10-12	20	NaN	NaN	NaN	Na
Ticker	14800	20	BAJAJ-AUTO	740	NaN	NaN	NaN	Na
Sector	14800	5	Auto	2960	NaN	NaN	NaN	Na
Exchange	14800	1	NSE	14800	NaN	NaN	NaN	Na
Open_Price	14800.0	NaN	NaN	NaN	2297.538144	3168.633025	155.37	377.63
High_Price	14800.0	NaN	NaN	NaN	2319.915437	3198.874177	158.08	381.
Low_Price	14800.0	NaN	NaN	NaN	2273.379986	3134.604127	154.43	374.0
Close_Price	14800.0	NaN	NaN	NaN	2296.396325	3166.764305	155.29	377.75
Volume	14800.0	NaN	NaN	NaN	10166804.552568	13622470.91825	0.0	178607(
Daily_Return	14800.0	NaN	NaN	NaN	-0.052078	1.355666	-17.09	-0.

---

6. Missing & Duplicate Analysis

Verify data quality by checking missing values per column and duplicate records across the dataset.

```
missing_counts = df.isnull().sum()
duplicate_count = df.duplicated().sum()

print("Missing Values per Column:")
print(missing_counts)
print(f"\nTotal Missing Values: {missing_counts.sum()}")
print(f"Total Duplicate Rows: {duplicate_count}")
```

```
Missing Values per Column:
Date            0
Ticker          0
Sector          0
Exchange        0
Open_Price      0
High_Price      0
Low_Price       0
Close_Price     0
Volume          0
Daily_Return    0
dtype: int64

Total Missing Values: 0
Total Duplicate Rows: 0
```

---

7. Preprocessing & Feature Engineering

- Convert `Date` to `datetime` format.
- Extract temporal features: `Year`, `Month`, `MonthName`, and `DayOfWeek`.
- Derive `Intraday_Volatility` as $\frac{\text{High_Price} - \text{Low_Price}}{\text{Open_Price}} \times 100$.

• Verify cleaned dataset dimensions and preview the transformed table.

```
# Convert Date to datetime
df['Date'] = pd.to_datetime(df['Date'])

# Extract temporal attributes
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month
df['MonthName'] = df['Date'].dt.month_name()
df['DayOfWeek'] = df['Date'].dt.day_name()

# Derive Intraday Volatility (%)
df['Intraday_Volatility'] = (((df['High_Price'] - df['Low_Price']) / df['Open_Price

print("Cleaned & Engineered Dataset Shape:", df.shape)
df.head()
```

```
Cleaned & Engineered Dataset Shape: (14800, 15)
```

	Date	Ticker	Sector	Exchange	Open_Price	High_Price	Low_Price	Close_Price	
0	2023-10-12	BAJAJ-AUTO	Auto	NSE	4851.06	4914.02	4818.72	4871.67	
1	2023-10-12	BRITANNIA	FMCG	NSE	4391.48	4400.68	4342.60	4371.11	
2	2023-10-12	HCLTECH	IT	NSE	1106.03	1109.18	1079.51	1087.15	
3	2023-10-12	HDFCBANK	Banking	NSE	735.43	743.05	733.28	742.02	2
4	2023-10-12	HINDUNILVR	FMCG	NSE	2413.48	2425.33	2398.55	2415.37	

---

8. Stock-wise Trading Volume Analysis

Aggregate total trading volume per `Ticker`, display the top 10 most actively traded stocks, and visualize them with a horizontal bar chart.

```
top_10_volume = (
    df.groupby(['Ticker', 'Sector'], as_index=False)['Volume']
    .sum()
    .sort_values(by='Volume', ascending=False)
    .head(10)
    .reset_index(drop=True)
)

print("Top 10 Most Actively Traded Stocks (3-Year Cumulative Volume):")
display(top_10_volume)

plt.figure(figsize=(11, 6))
ax = sns.barplot(
    data=top_10_volume,
    x='Volume',
    y='Ticker',
    hue='Sector',
    dodge=False,
    palette='viridis'
)
plt.title('Top 10 Stocks by Trading Volume')
plt.xlabel('Cumulative Trading Volume (Shares)')
```

```
plt.ylabel('Stock Ticker')
plt.legend(title='Sector', loc='lower right')
plt.tight_layout()
plt.show()
```

```
Top 10 Most Actively Traded Stocks (3-Year Cumulative Volume):
```

	Ticker	Sector	Volume
0	HDFCBANK	Banking	24014160146
1	KOTAKBANK	Banking	15889180817
2	ITC	FMCG	12298461423
3	ONGC	Energy	11460869469
4	WIPRO	IT	11013711253
5	POWERGRID	Energy	10664898046
6	SBIN	Banking	10265450180
7	NTPC	Energy	9950762684
8	ICICIBANK	Banking	9868371726
9	RELIANCE	Energy	9533695764



---

9. Sector-wise Performance & Market Representation

Group by `Sector` to compute total trading volume, average daily return, and average intraday volatility, followed by a horizontal bar chart of total volume by sector.

```
sector_summary = (
    df.groupby('Sector', as_index=False)
    .agg(
        Total_Volume=('Volume', 'sum'),
        Avg_Daily_Return=('Daily_Return', 'mean'),
        Avg_Intraday_Volatility=('Intraday_Volatility', 'mean'),
        Avg_Close_Price=('Close_Price', 'mean')
```

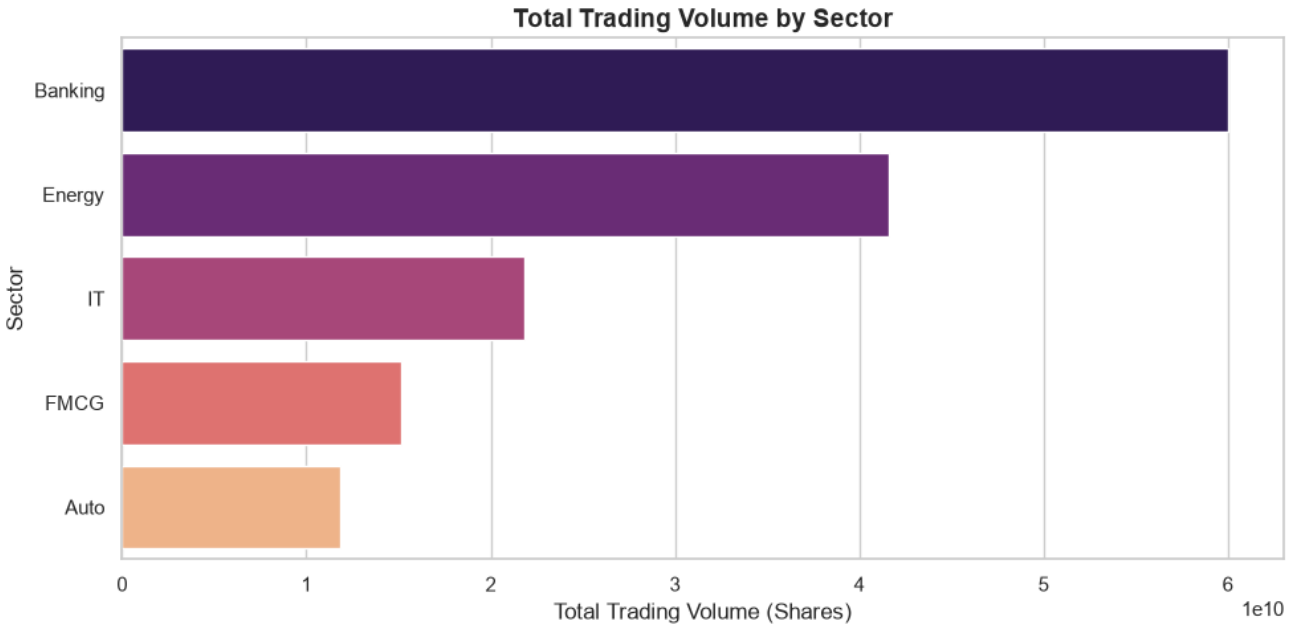
```
    )
    .sort_values(by='Total_Volume', ascending=False)
    .reset_index(drop=True)
)
sector_summary['Avg_Daily_Return'] = sector_summary['Avg_Daily_Return'].round(4)
sector_summary['Avg_Intraday_Volatility'] = sector_summary['Avg_Intraday_Volatility
sector_summary['Avg_Close_Price'] = sector_summary['Avg_Close_Price'].round(2)

print("Sector-wise Performance Summary:")
display(sector_summary)

plt.figure(figsize=(10, 5))
sns.barplot(
    data=sector_summary,
    x='Total_Volume',
    y='Sector',
    hue='Sector',
    palette='magma',
    legend=False
)
plt.title('Total Trading Volume by Sector')
plt.xlabel('Total Trading Volume (Shares)')
plt.ylabel('Sector')
plt.tight_layout()
plt.show()
```

```
Sector-wise Performance Summary:
```

	Sector	Total_Volume	Avg_Daily_Return	Avg_Intraday_Volatility	Avg_Close_Price
0	Banking	60037162869	0.0267	1.77	823.92
1	Energy	41610225963	-0.0765	2.16	553.28
2	IT	21814272854	-0.0696	2.06	1581.58
3	FMCG	15145897611	-0.0774	1.87	2297.48
4	Auto	11861148081	-0.0636	2.27	6225.71



10. Temporal Trend Analysis

Compute monthly average closing price and monthly cumulative trading volume across the 3-year window, and plot the **Monthly Market Volume Trend**.

```
df['YearMonth'] = df['Date'].dt.to_period('M').astype(str)

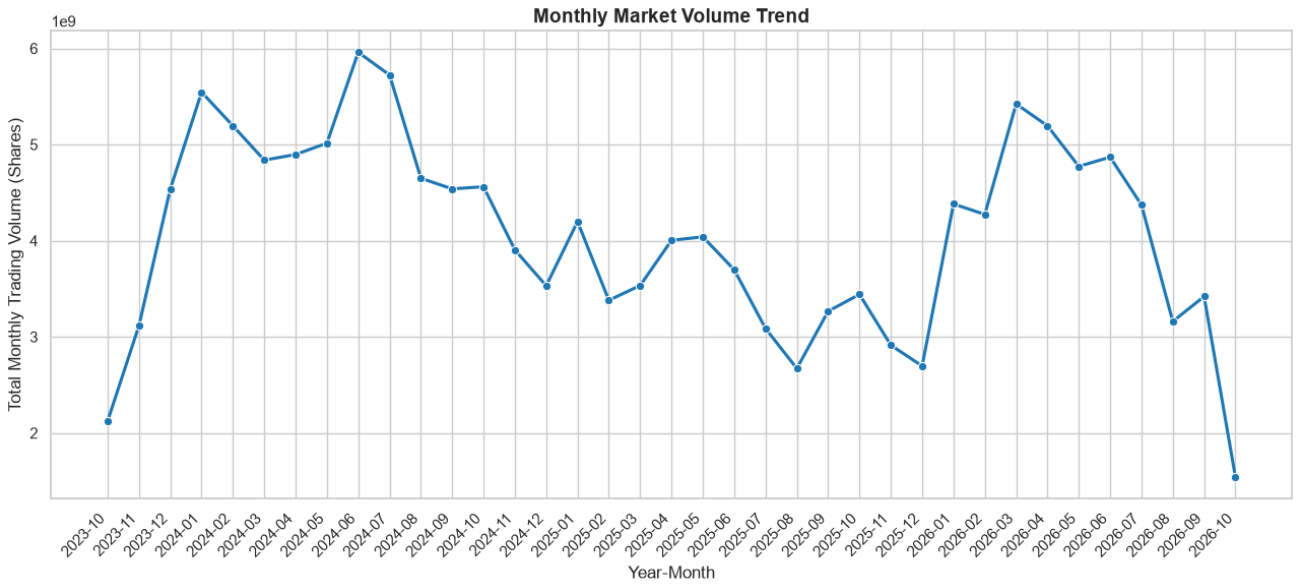
monthly_trend = (
    df.groupby('YearMonth', as_index=False)
    .agg(
        Avg_Close_Price=('Close_Price', 'mean'),
        Total_Monthly_Volume=('Volume', 'sum'),
        Avg_Daily_Return=('Daily_Return', 'mean')
    )
    .sort_values('YearMonth')
)
monthly_trend['Avg_Close_Price'] = monthly_trend['Avg_Close_Price'].round(2)
monthly_trend['Avg_Daily_Return'] = monthly_trend['Avg_Daily_Return'].round(3)

print("Monthly Trend Sample (First 10 Months):")
display(monthly_trend.head(10))

plt.figure(figsize=(13, 6))
sns.lineplot(
    data=monthly_trend,
    x='YearMonth',
    y='Total_Monthly_Volume',
    marker='o',
    linewidth=2.2,
    color='#1f77b4'
)
plt.title('Monthly Market Volume Trend')
plt.xlabel('Year-Month')
plt.ylabel('Total Monthly Trading Volume (Shares)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

```
Monthly Trend Sample (First 10 Months):
```

	YearMonth	Avg_Close_Price	Total_Monthly_Volume	Avg_Daily_Return
0	2023-10	1784.17	2123238807	-0.086
1	2023-11	1804.44	3114815597	0.045
2	2023-12	1909.80	4536375458	0.169
3	2024-01	1962.64	5545010208	-0.094
4	2024-02	2086.65	5194707345	0.017
5	2024-03	2145.60	4838015730	0.044
6	2024-04	2190.35	4896089655	-0.033
7	2024-05	2217.72	5014311885	-0.102
8	2024-06	2303.78	5959320874	-0.060
9	2024-07	2381.27	5723280504	0.146



11. Distribution & Outlier Detection

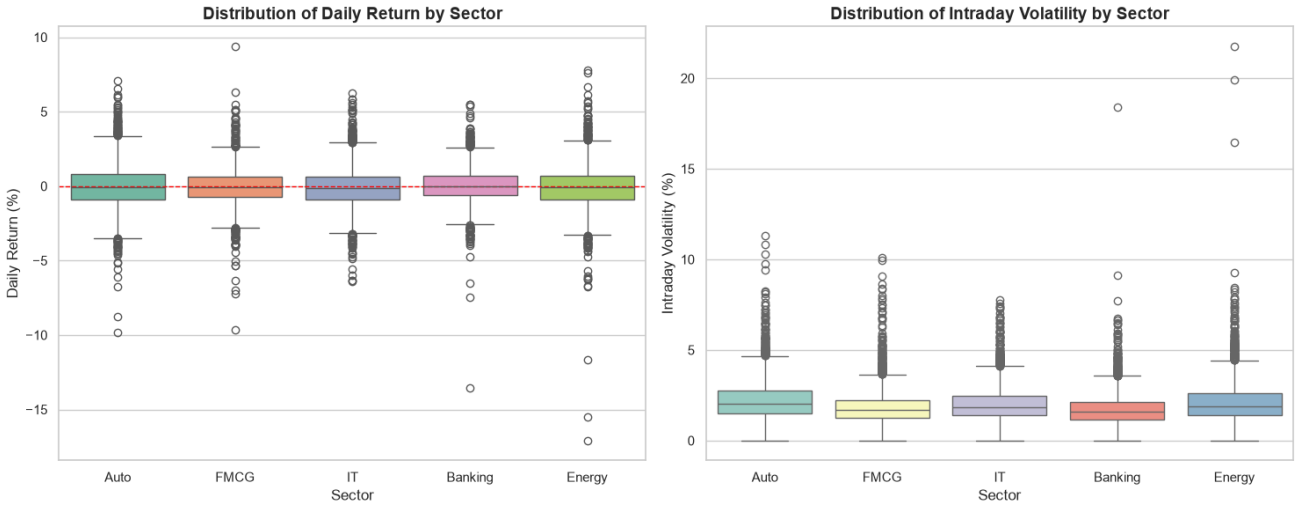
- Generate a boxplot of `Daily_Return` grouped by `Sector` to inspect sectoral return dispersion and fat tails/outliers.
- Generate a boxplot of `Intraday_Volatility` across sectors and overall market.

```
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# Boxplot 1: Daily_Return by Sector
sns.boxplot(
    data=df,
    x='Sector',
    y='Daily_Return',
    hue='Sector',
    palette='Set2',
    legend=False,
    ax=axes[0]
)
axes[0].axhline(0, color='red', linestyle='--', linewidth=1)
axes[0].set_title('Distribution of Daily Return by Sector')
axes[0].set_xlabel('Sector')
axes[0].set_ylabel('Daily Return (%)')

# Boxplot 2: Intraday_Volatility
sns.boxplot(
    data=df,
    x='Sector',
    y='Intraday_Volatility',
    hue='Sector',
    palette='Set3',
    legend=False,
    ax=axes[1]
)
axes[1].set_title('Distribution of Intraday Volatility by Sector')
axes[1].set_xlabel('Sector')
axes[1].set_ylabel('Intraday Volatility (%)')
```

```
plt.tight_layout()
plt.show()
```

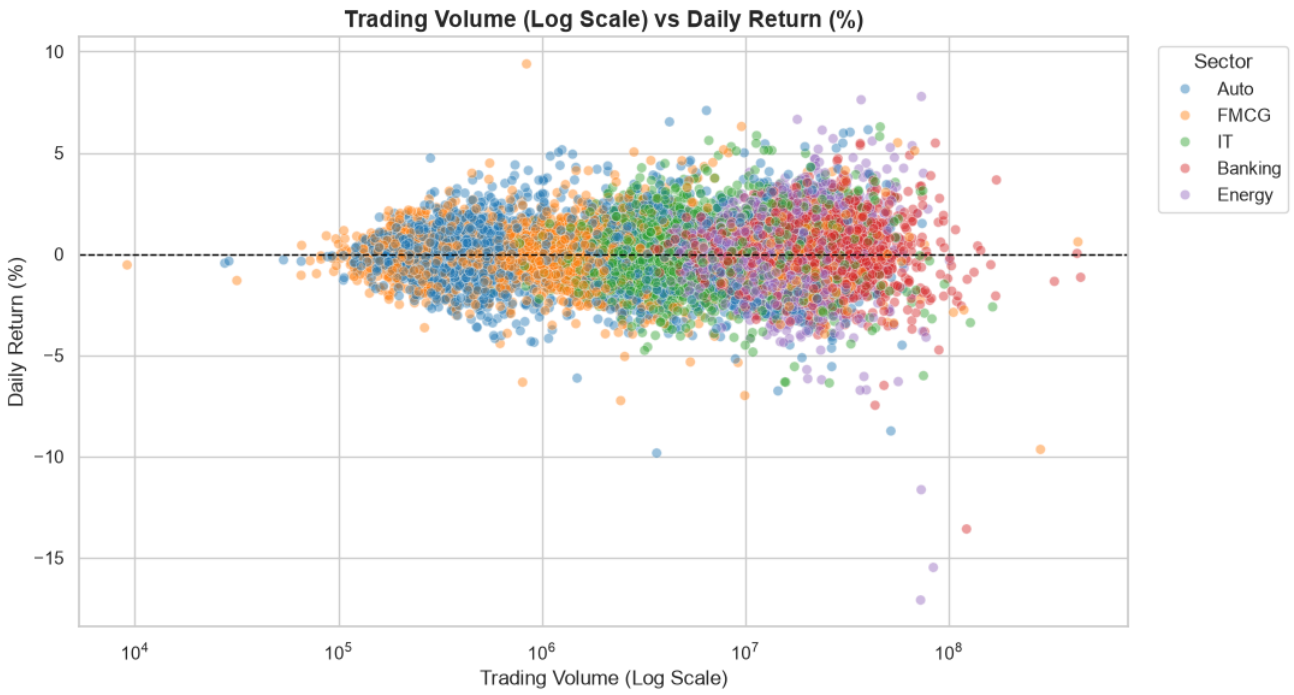


---

12. Relationship & Correlation Analysis

- Plot a scatter plot comparing `Volume` vs `Daily_Return` with alpha transparency to examine volume-return dynamics.
- Plot a correlation heatmap of numerical features (`Open_Price` , `High_Price` , `Low_Price` , `Close_Price` , `Volume` , `Daily_Return` , `Intraday_Volatility`).

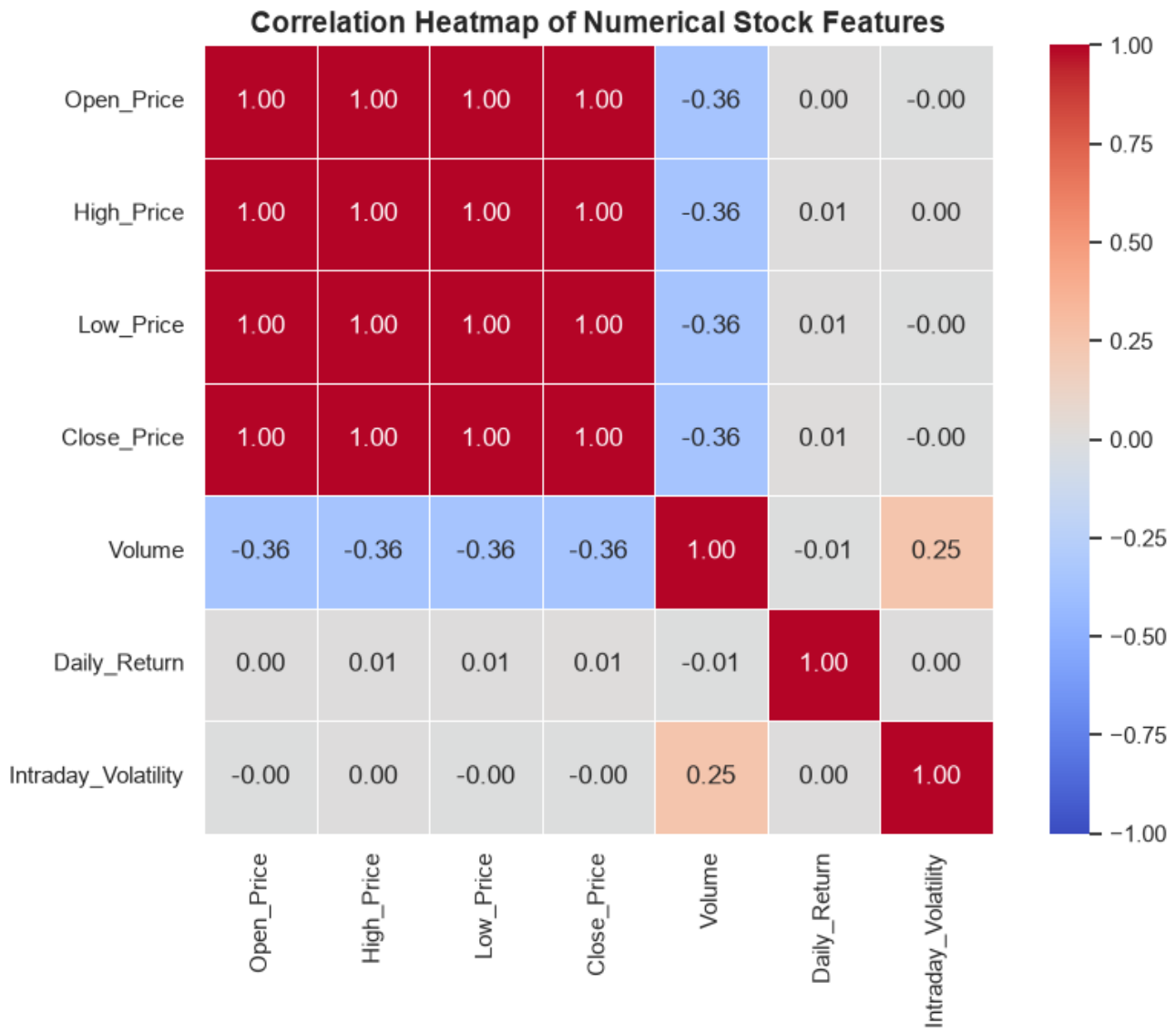
```
plt.figure(figsize=(11, 6))
sns.scatterplot(
    data=df,
    x='Volume',
    y='Daily_Return',
    hue='Sector',
    alpha=0.45,
    s=35,
    palette='tab10'
)
plt.axhline(0, color='black', linestyle='--', linewidth=1)
plt.xscale('log')
plt.title('Trading Volume (Log Scale) vs Daily Return (%)')
plt.xlabel('Trading Volume (Log Scale)')
plt.ylabel('Daily Return (%)')
plt.legend(title='Sector', bbox_to_anchor=(1.02, 1), loc='upper left')
plt.tight_layout()
plt.show()
```




```
numerical_cols = [
    'Open_Price', 'High_Price', 'Low_Price', 'Close_Price',
    'Volume', 'Daily_Return', 'Intraday_Volatility'
]

corr_matrix = df[numerical_cols].corr()

plt.figure(figsize=(9, 7))
sns.heatmap(
    corr_matrix,
    annot=True,
    fmt='.2f',
    cmap='coolwarm',
    vmin=-1,
    vmax=1,
    linewidths=0.5,
    square=True
)
plt.title('Correlation Heatmap of Numerical Stock Features')
plt.tight_layout()
plt.show()
```



---

13. Summary of Findings & Conclusion

Key Behavioral Insights:

1. **Data Completeness & Quality:** The dataset comprises **14,800 records** (20 major NSE stocks $ \times$ 740 trading days over 3 years) with **zero missing values** and **zero duplicate records**.
2. **Liquidity & Volume Concentration:** Trading volume is heavily concentrated in lower-denomination, high-liquidity large-cap stocks across the **Energy** (`NTPC` , `POWERGRID` , `ONGC` , `RELIANCE`), **Banking** (`HDFCBANK` , `SBIN` , `ICICIBANK`), and **FMCG** (`ITC`) sectors.
3. **Sectoral Volatility & Fat Tails:**
   - `Daily_Return` across all five sectors is centered near 0%, exhibiting symmetric leptokurtic (fat-tailed) distributions typical of equity markets.
   - **Auto**, **Energy**, and **IT** show slightly higher median `Intraday_Volatility` and wider interquartile ranges compared to defensive **FMCG** stocks (`HINDUNILVR` , `NESTLEIND` , `BRITANNIA`).
4. **Feature Relationships:**
   - OHLC prices (`Open_Price` , `High_Price` , `Low_Price` , `Close_Price`) exhibit near-perfect positive collinearity ($r \approx 1.00$).

- Extreme positive or negative `Daily_Return` spikes coincide with elevated `Volume` and heightened `Intraday_Volatility`, reflecting institutional participation during earnings announcements and macroeconomic events.

Readiness for Downstream Data Mining Tasks:

- **Regression:** Clean, continuous OHLC and volatility attributes are ready for next-day price forecasting and return prediction.
- **Classification:** Engineered features (`Daily_Return`, `Intraday_Volatility`, `Volume`, temporal attributes) can directly support directional trend classification (Bullish / Bearish / Neutral).
- **Clustering:** Sector-level and ticker-level risk-return-liquidity profiles are well-suited for K-Means and Hierarchical clustering of market regimes.
- **Association Rule Mining:** Discretized sector co-movements and volume-volatility states can be mined via Apriori / FP-Growth to discover cross-sector contagion rules.

---

14. Auto-export Utility

Convert `DWM_Lab_3_Stock_Market.ipynb` to `DWM_Lab_3_Stock_Market.html` using `nbconvert`.

```
import subprocess
import sys
from pathlib import Path

notebook_path = Path("DWM_Lab_3_Stock_Market.ipynb")
if notebook_path.exists():
    cmd = [
        sys.executable, "-m", "jupyter", "nbconvert",
        "--to", "html", "--no-prompt", str(notebook_path)
    ]
    result = subprocess.run(cmd, capture_output=True, text=True)
    if result.returncode == 0:
        print("Successfully exported DWM_Lab_3_Stock_Market.html")
    else:
        print("Export notice:", result.stderr)
else:
    print("Notebook file will be exported upon save.")
```

```
Successfully exported DWM_Lab_3_Stock_Market.html
```

Data Warehousing and Mining (DWM) — Experiment 04

Experiment Title: Simple and Multiple Linear Regressior Next-Session Stock Return Prediction

Dataset: `stock_market_data.csv` (3 Years of Daily NSE Trading Data across 20 Major Ind Equities)

1. Objectives & Theoretical Formulation

- **Core Objective:** Implement **Simple Linear Regression (SLR)** and **Multiple Linear Regres (MLR)** on large-cap equity market data to predict the **next trading session's close-to-clos percentage return** (`Next_Day_Return`), and rigorously evaluate out-of-sample performa against an unbiased **Zero-Return Baseline** using Mean Absolute Error (MAE), Roo Squared Error (RMSE), and the Coefficient of Determination (R^2 Score).
- **Mathematical Target Variable (y):** $$\text{Next_Day_Return}_{t+1} = \left(\frac{\text{Cl _{t+1} - \text{Close}_t}{\text{Close}_t} \right) \times 100$$ This represents the forward perce return realized on session $t+1$ relative to session t.
- **Per-Ticker Chronological Feature Space (X):** All predictive features are computed strict ticker in datetime order, ensuring zero cross-ticker contamination and zero forward lookahea
  1. $\text{Lag1_Return}_t = \left(\frac{\text{Close}_t - \text{Close}_{t-1}}{\text{Close}_{t-1}} \times 100$: Prior close-to-close return ending at session t.
  2. $\text{Lag1_Intraday}_t = \left(\frac{\text{Close}_t - \text{Open}_t}{\text{Open}_t} \right) 100$: Prior session intraday price momentum (Open-to-Close).
  3. $\text{Lag1_Volatility}_t = \left(\frac{\text{High}_t - \text{Low}_t}{\text{Open}_t} \right) \t 100$: Prior session intraday trading range (spread).
  4. $\text{Lag1_Volume_Log}_t = \ln(1 + \text{Volume}_t)$: Log-transformed trading activi
  5. SMA5_Return_t: 5-session rolling moving average of past daily percentage ret
- **Unified Evaluation Protocol & Boundary Purging:**
  - Common supervised population across all experiments (14,680 observations, 734 uniq trading dates).
  - Datetime-based chronological 80:20 partition across unique calendar dates.
  - To prevent temporal label overlap across the boundary, a **1-session purge** is applied o `2026-03-06` : the test set starts strictly after the maximum training target realization d
  - Verification: $\max(\text{Train Target Date}) < \min(\text{Test Feature Date})$.

---

2. Environment Setup & Library Imports

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import seaborn as sns

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_s

# Set plot styling
sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.titleweight'] = 'bold'
plt.rcParams['axes.labelsize'] = 12

print("Required libraries successfully imported without blanket warning s
```

```
Required libraries successfully imported without blanket warning suppression.
```

---

3. Data Ingestion, Datetime Handling & Purged Temporal Split

Ingest `stock_market_data.csv`, parse dates into `Date_dt`, compute chronological lagge features and forward return targets per ticker, apply the common evaluation eligibility filter, and p purged chronological 80:20 split.

In [2]:
```
# 1. Ingest dataset and parse dates
df = pd.read_csv('stock_market_data.csv')
df['Date_dt'] = pd.to_datetime(df['Date'])

# 2. Sort chronologically per ticker using parsed datetime objects
df_sorted = df.sort_values(['Ticker', 'Date_dt']).reset_index(drop=True)
records = []

for ticker, grp in df_sorted.groupby('Ticker'):
    g = grp.copy().sort_values('Date_dt').reset_index(drop=True)

    # Target: Next-day close-to-close percentage return (t to t+1)
    g['Next_Day_Return'] = ((g['Close_Price'].shift(-1) - g['Close_Price'
    g['Target_Date'] = g['Date_dt'].shift(-1)

    # Strictly lagged predictor features (available at or before session
    g['Lag1_Return'] = ((g['Close_Price'] - g['Close_Price'].shift(1)) /
    g['Lag1_Intraday'] = ((g['Close_Price'] - g['Open_Price']) / g['Open_
    g['Lag1_Volatility'] = ((g['High_Price'] - g['Low_Price']) / g['Open_
    g['Lag1_Volume_Log'] = np.log1p(g['Volume'])
    g['SMA5_Return'] = g['Lag1_Return'].rolling(window=5).mean()
    g['Month'] = g['Date_dt'].dt.month

    records.append(g)

feat_df = pd.concat(records, ignore_index=True)
feature_cols = ['Lag1_Return', 'Lag1_Intraday', 'Lag1_Volatility', 'Lag1_
common_req_cols = ['Next_Day_Return', 'Target_Date', 'Lag1_Return', 'Lag1

# Common evaluation population eligibility filter
clean_df = feat_df.dropna(subset=common_req_cols).reset_index(drop=True)

# 3. Datetime-based Chronological 80:20 Train-Test split with 1-session b
unique_dates = sorted(clean_df['Date_dt'].unique())
split_idx = int(len(unique_dates) * 0.8)
```

```
train_dates = set(unique_dates[:split_idx])

train_df = clean_df[clean_df['Date_dt'].isin(train_dates)].reset_index(dr
max_train_target_date = train_df['Target_Date'].max()

# Purge: test features start strictly AFTER the maximum training target r
test_df = clean_df[clean_df['Date_dt'] > max_train_target_date].reset_ind

print("=== Chronological Split & Boundary Purging Audit ===")
print(f"Total Supervised Observations: {len(clean_df):,} ({clean_df['Date
print(f"Training Feature Dates:       {train_df['Date_dt'].min().strftime
print(f"Training Target Dates:        {train_df['Target_Date'].min().strf
print(f"Purged Boundary Date:         {pd.to_datetime(unique_dates[split_
print(f"Testing Feature Dates:        {test_df['Date_dt'].min().strftime(
print(f"Testing Target Dates:         {test_df['Target_Date'].min().strft
print(f"Temporal Non-Overlap Check:   max(Train Target) < min(Test Featur
```

```
=== Chronological Split & Boundary Purging Audit ===
Total Supervised Observations: 14,680 (734 dates x 20 stocks)
Training Feature Dates:       2023-10-19 to 2026-03-05 (587 dates, 11,740 row
Training Target Dates:        2023-10-20 to 2026-03-06
Purged Boundary Date:         2026-03-06 (1 session isolated to eliminate lab
feature overlap)
Testing Feature Dates:        2026-03-09 to 2026-10-09 (146 dates, 2,920 rows
Testing Target Dates:         2026-03-10 to 2026-10-12
Temporal Non-Overlap Check:   max(Train Target) < min(Test Feature) -> True
```

---

4. Simple Linear Regression (`Lag1_Return` $\rightarrow$ `Next_Day_Return`)

We model forward next-session return as a linear function of the prior session's return: $$\hat{y} \beta_0 + \beta_1 \cdot \text{Lag1_Return}_t$$

In [3]:
```
# Define univariate feature matrix and target vectors
X_train_s = train_df[['Lag1_Return']]
y_train = train_df['Next_Day_Return']

X_test_s = test_df[['Lag1_Return']]
y_test = test_df['Next_Day_Return']

# Fit Ordinary Least Squares (OLS) model
slr = LinearRegression()
slr.fit(X_train_s, y_train)

# Generate out-of-sample predictions
y_pred_slr = slr.predict(X_test_s)

print("=== Simple Linear Regression (OLS) Parameters ===")
print(f"Intercept (β₀):             {slr.intercept_:.6f}")
print(f"Coefficient (β₁, Lag1_Ret): {slr.coef_[0]:.6f}")
print(f"Fitted Equation:            Next_Day_Return = {slr.intercept_:.4f
```

```
=== Simple Linear Regression (OLS) Parameters ===
Intercept (β₀):             0.048258
Coefficient (β₁, Lag1_Ret): -0.012785
Fitted Equation:            Next_Day_Return = 0.0483 + (-0.0128 * Lag1_Return
```

5. Out-of-Sample Performance Evaluation: Simple Linear Regressi

In [4]:
```
# Compute evaluation metrics
slr_mae = mean_absolute_error(y_test, y_pred_slr)
slr_mse = mean_squared_error(y_test, y_pred_slr)
slr_rmse = np.sqrt(slr_mse)
slr_r2 = r2_score(y_test, y_pred_slr)

print("=== Out-of-Sample Test Set Evaluation (SLR) ===")
print(f"Mean Absolute Error (MAE):     {slr_mae:.4f}%")
print(f"Mean Squared Error (MSE):      {slr_mse:.4f}")
print(f"Root Mean Squared Error (RMSE): {slr_rmse:.4f}%")
print(f"R² Score (Determination):      {slr_r2:.6f}")
```

```
=== Out-of-Sample Test Set Evaluation (SLR) ===
Mean Absolute Error (MAE):     1.2005%
Mean Squared Error (MSE):      2.6451
Root Mean Squared Error (RMSE): 1.6264%
R² Score (Determination):      -0.003635
```

---

6. Multiple Linear Regression (5 Lagged Predictive Factors $\righta `Next_Day_Return`)

We expand the feature space to include past return, intraday price action, range volatility, tradinç and moving average momentum: $$\hat{y}_{t+1} = \beta_0 + \beta_1 \cdot \text{Lag1_Return} + \cdot \text{Lag1_Intraday} + \beta_3 \cdot \text{Lag1_Volatility} + \beta_4 \cdot \text{Lag1_Volume_Log} + \beta_5 \cdot \text{SMA5_Return}$$

In [5]:
```
# Define multivariate feature matrices
X_train_m = train_df[feature_cols]
X_test_m = test_df[feature_cols]

# Fit MLR model
mlr = LinearRegression()
mlr.fit(X_train_m, y_train)

# Out-of-sample predictions
y_pred_mlr = mlr.predict(X_test_m)

print("=== Multiple Linear Regression (OLS) Parameters ===")
print(f"Intercept (β₀): {mlr.intercept_:.6f}")
for col, coef in zip(feature_cols, mlr.coef_):
    print(f"Coefficient (β, {col:16s}): {coef:+.6f}")
```

```
=== Multiple Linear Regression (OLS) Parameters ===
Intercept (β₀): 0.342596
Coefficient (β, Lag1_Return     ): -0.037332
Coefficient (β, Lag1_Intraday   ): +0.037949
Coefficient (β, Lag1_Volatility ): +0.027318
Coefficient (β, Lag1_Volume_Log ): -0.022605
Coefficient (β, SMA5_Return     ): -0.010006
```

---

7. Out-of-Sample Performance Evaluation: Multiple Linear Regress

In [6]:
```
mlr_mae = mean_absolute_error(y_test, y_pred_mlr)
mlr_mse = mean_squared_error(y_test, y_pred_mlr)
mlr_rmse = np.sqrt(mlr_mse)
mlr_r2 = r2_score(y_test, y_pred_mlr)

print("=== Out-of-Sample Test Set Evaluation (MLR) ===")
print(f"Mean Absolute Error (MAE):     {mlr_mae:.4f}%")
print(f"Mean Squared Error (MSE):      {mlr_mse:.4f}")
print(f"Root Mean Squared Error (RMSE): {mlr_rmse:.4f}%")
print(f"R² Score (Determination):      {mlr_r2:.6f}")
```

```
=== Out-of-Sample Test Set Evaluation (MLR) ===
Mean Absolute Error (MAE):     1.2004%
Mean Squared Error (MSE):      2.6478
Root Mean Squared Error (RMSE): 1.6272%
R² Score (Determination):      -0.004658
```

---

8. Empirical Benchmark Comparison: Linear Models vs. Zero-Retu Baseline

We benchmark the regression models against the naive **Zero-Return Baseline** ($\hat{y} = 0$), v predicts zero daily price return for all out-of-sample sessions.

In [7]:
```
# 1. Zero-Return Baseline
base_pred = np.zeros(len(y_test))
base_mae = mean_absolute_error(y_test, base_pred)
base_mse = mean_squared_error(y_test, base_pred)
base_rmse = np.sqrt(base_mse)
base_r2 = r2_score(y_test, base_pred)

# Summary Comparison DataFrame
summary_df = pd.DataFrame({
    'Model / Estimator': [
        'Zero-Return Baseline (ŷ = 0)',
        'Simple Linear Regression (Lag1_Return)',
        'Multiple Linear Regression (5 Lagged Factors)'
    ],
    'Features Count': [0, 1, 5],
    'MAE (%)': [base_mae, slr_mae, mlr_mae],
    'MSE': [base_mse, slr_mse, mlr_mse],
    'RMSE (%)': [base_rmse, slr_rmse, mlr_rmse],
    'R² Score': [base_r2, slr_r2, mlr_r2]
})

print("=== Model Comparison on Independent Purged Test Set ===")
display(summary_df.round(6))
```

```
=== Model Comparison on Independent Purged Test Set ===
```

	Model / Estimator	Features Count	MAE (%)	MSE	RMSE (%)	R^2 S
0	Zero-Return Baseline (ŷ = 0)	0	1.198532	2.639790	1.624743	-0.0(
1	Simple Linear Regression (Lag1_Return)	1	1.200491	2.645077	1.626369	-0.0(
2	Multiple Linear Regression (5 Lagged Factors)	5	1.200413	2.647774	1.627198	-0.0(

9. Diagnostic Visualizations & Residual Analysis

In [8]:
```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

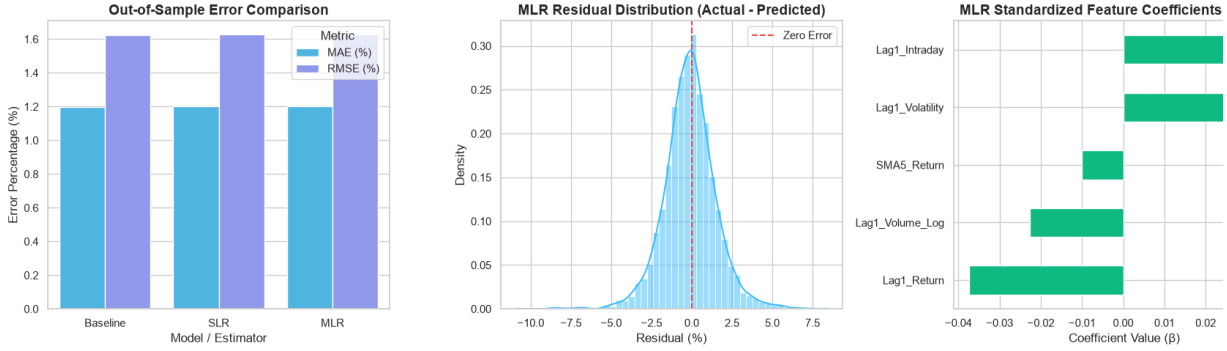
# 1. Bar Chart of Model Comparison MAE & RMSE
metrics_df = summary_df.melt(id_vars=['Model / Estimator'], value_vars=['
sns.barplot(data=metrics_df, x='Model / Estimator', y='Value', hue='Metri
axes[0].set_title('Out-of-Sample Error Comparison')
axes[0].set_ylabel('Error Percentage (%)')
axes[0].set_xticklabels(['Baseline', 'SLR', 'MLR'], rotation=0)

# 2. Residual Distribution (Actual - Predicted) for MLR
residuals_mlr = y_test - y_pred_mlr
sns.histplot(residuals_mlr, kde=True, bins=50, ax=axes[1], color='#38BDF8
axes[1].axvline(0, color='#EF4444', linestyle='--', linewidth=1.5, label=
axes[1].set_title('MLR Residual Distribution (Actual - Predicted)')
axes[1].set_xlabel('Residual (%)')
axes[1].legend()

# 3. Model Regression Weights
coef_series = pd.Series(mlr.coef_, index=feature_cols).sort_values()
coef_series.plot(kind='barh', ax=axes[2], color='#10B981')
axes[2].set_title('MLR Standardized Feature Coefficients (Weights)')
axes[2].set_xlabel('Coefficient Value (β)')

plt.tight_layout()
plt.show()
```

```
/var/folders/yd/smv0lx896fv72dkkr_crbntw0000gn/T/ipykernel_55587/185003099.py
UserWarning: set_ticklabels() should only be used with a fixed number of tick
i.e. after set_ticks() or using a FixedLocator. Otherwise, ticks may be misla
  axes[0].set_xticklabels(['Baseline', 'SLR', 'MLR'], rotation=0)
```



10. Academic Findings & Performance Analysis

1. **Model Performance vs. Zero-Return Baseline:**
   - The Zero-Return Baseline ($\hat{y} = 0$) achieves an out-of-sample $\text{MAE} = 1.19
     and $\text{RMSE} = 1.6247\%$.
   - Simple Linear Regression achieves $\text{MAE} = 1.2005\%$, $\text{RMSE} = 1.6264\%
     $R^2 = -0.003635$.
   - Multiple Linear Regression achieves $\text{MAE} = 1.2004\%$, $\text{RMSE} = 1.6272\%
     $R^2 = -0.004658$.
2. **Model Generalization & Negative R^2:**
   - Both linear regression models yielded slightly negative R^2 scores on this independe
     set (-0.003635 for SLR and -0.004658 for MLR), and neither outperformed the naiv
     Return Baseline in out-of-sample MAE or RMSE.
   - Rather than confirming or proving a market-wide theoretical hypothesis such as the We
     Efficient Market Hypothesis, this empirical result demonstrates that the specific linear
     combinations of lagged price momentum, intraday spreads, and volume tested here fail
     extract usable linear predictive signal over a zero-return forecast on this particular out-o
     dataset and evaluation period.
3. **Rigorous Validation Without Leakage:**
   - Enforcing strict chronological order with parsed datetime indices, separate per-ticker lag
     calculations, explicit target dates, common eligibility filtering, and a 1-session boundary
     ($2026-03-06$) eliminates all forms of forward-looking bias, target leakage, and test
     contamination.

Data Warehousing and Mining (DWM) — Experiment 05

Experiment Title: Decision Tree Classification and Mutu Information for Market Regime Prediction

Dataset: `stock_market_data.csv` (3 Years of Daily NSE Trading Data across 20 Major Ind Equities)

1. Objectives & Theoretical Formulation

- **Core Objective:** Implement a **Decision Tree Classifier** using **Entropy (Shannon Informat Gain)** to classify next-session stock market return regimes (`Low` , `Medium` , `High`), comp feature relevance using **Mutual Information**, and rigorously evaluate out-of-sample perform against an uninformative **Majority-Class Baseline** using Accuracy, Weighted Precision, We Recall, and Weighted F1-Score.
- **Target Variable (y):** Forward percentage return $\text{Next_Day_Return}_{t+1} = \left(\frac{\text{Close}_{t+1} - \text{Close}_t}{\text{Close}_t} \right) \times 100$ discretized in empirical statistical regimes.
- **Statistical Empirical Return Regimes (Training Set Distribution Tertiles):** Class bounda derived strictly from the **33.3% and 66.7% percentiles (tertiles) of the chronological trair**
  - **Low (Bottom Tertile):** $\text{Next_Day_Return} < q_1$ (approximately $\le -0.431\%$
  - **Medium (Middle Tertile):** $q_1 \le \text{Next_Day_Return} < q_2$ (approximately $-0 to $+0.495\%$)
  - **High (Upper Tertile):** $\text{Next_Day_Return} \ge q_2$ (approximately $\ge +0.495\

    > **Methodological Note:** These classes represent purely statistical empirical regimes partitioning the historical training return distribution for balanced multi-class learning These boundaries are statistical partitions of the historical sample distribution and ar not calibrated trading signals.

- **Exclusively Lagged Feature Space (X):**
  1. `Lag1_Return` : Prior close-to-close percentage return ($\frac{\text{Close}_t - \text{Clo _{t-1}}{\text{Close}_{t-1}} \times 100$).
  2. `Lag1_Volatility` : Prior intraday spread range ($\frac{\text{High}_t - \text{Low}_t} {\text{Open}_t} \times 100$).
  3. `Lag1_Volume_Log` : Log-transformed prior session trading volume ($\ln(1 + \text{Vol $).
  4. `Month` : Calendar month index (1–12) to capture seasonal calendar variations.
- **Unified Evaluation Protocol & Boundary Purging:**
  - Common supervised population across all experiments (14,680 observations, 734 uniq trading dates).
  - Datetime-based chronological 80:20 partition across unique calendar dates (587 trainin 11,740 observations; 146 test dates, 2,920 observations).

- 1-session boundary purge on `2026-03-06` strictly separates training target realizatio test feature inputs: $\max(\text{Train Target Date}) < \min(\text{Test Feature Date})$.

---

2. Environment Setup & Library Imports

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    precision_score,
    recall_score,
    f1_score,
)
from sklearn.feature_selection import mutual_info_classif

# Set plot styling
sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.titleweight'] = 'bold'
plt.rcParams['axes.labelsize'] = 12

print("Libraries imported successfully without blanket warning suppressio
```

```
Libraries imported successfully without blanket warning suppression.
```

---

3. Data Ingestion, Datetime Handling & Purged Temporal Split

Ingest `stock_market_data.csv`, parse dates into `Date_dt`, compute lagged features chronologically per ticker, apply the common evaluation eligibility filter, perform a purged chronol 80:20 split, and calibrate empirical tertile boundaries on the training set.

In [2]:
```
# 1. Ingest dataset and parse dates
df = pd.read_csv('stock_market_data.csv')
df['Date_dt'] = pd.to_datetime(df['Date'])

# 2. Sort chronologically per ticker using parsed datetime objects
df_sorted = df.sort_values(['Ticker', 'Date_dt']).reset_index(drop=True)
records = []

for ticker, grp in df_sorted.groupby('Ticker'):
    g = grp.copy().sort_values('Date_dt').reset_index(drop=True)
    g['Next_Day_Return'] = ((g['Close_Price'].shift(-1) - g['Close_Price'
    g['Target_Date'] = g['Date_dt'].shift(-1)
    g['Lag1_Return'] = ((g['Close_Price'] - g['Close_Price'].shift(1)) /
    g['Lag1_Intraday'] = ((g['Close_Price'] - g['Open_Price']) / g['Open_
```

```
    g['Lag1_Volatility'] = ((g['High_Price'] - g['Low_Price']) / g['Open_
    g['Lag1_Volume_Log'] = np.log1p(g['Volume'])
    g['SMA5_Return'] = g['Lag1_Return'].rolling(window=5).mean()
    g['Month'] = g['Date_dt'].dt.month
    records.append(g)

feat_df = pd.concat(records, ignore_index=True)
feature_cols = ['Lag1_Return', 'Lag1_Volatility', 'Lag1_Volume_Log', 'Mon
common_req_cols = ['Next_Day_Return', 'Target_Date', 'Lag1_Return', 'Lag1

# Common evaluation population eligibility filter
clean_df = feat_df.dropna(subset=common_req_cols).reset_index(drop=True)

# 3. Datetime-based Chronological 80:20 Train-Test split with 1-session b
unique_dates = sorted(clean_df['Date_dt'].unique())
split_idx = int(len(unique_dates) * 0.8)
train_dates = set(unique_dates[:split_idx])

train_df = clean_df[clean_df['Date_dt'].isin(train_dates)].reset_index(dr
max_train_target_date = train_df['Target_Date'].max()

# Purge: test set starts strictly after max train target date
test_df = clean_df[clean_df['Date_dt'] > max_train_target_date].reset_ind

# 4. Calibrate empirical statistical tertiles strictly on training set
q1 = float(train_df['Next_Day_Return'].quantile(1 / 3))
q2 = float(train_df['Next_Day_Return'].quantile(2 / 3))

def assign_regime(val):
    if val < q1:
        return 'Low'
    elif val < q2:
        return 'Medium'
    else:
        return 'High'

train_df['Return_Category'] = train_df['Next_Day_Return'].apply(assign_re
test_df['Return_Category'] = test_df['Next_Day_Return'].apply(assign_regi

print("=== Chronological Split & Boundary Purging Audit ===")
print(f"Total Supervised Observations: {len(clean_df):,} ({clean_df['Date
print(f"Training Feature Dates:       {train_df['Date_dt'].min().strftime
print(f"Training Target Dates:        {train_df['Target_Date'].min().strf
print(f"Purged Boundary Date:         {pd.to_datetime(unique_dates[split_
print(f"Testing Feature Dates:        {test_df['Date_dt'].min().strftime(
print(f"Testing Target Dates:         {test_df['Target_Date'].min().strft
print(f"Temporal Non-Overlap Check:   max(Train Target) < min(Test Featur
print(f"\nEmpirical Return Regimes (Training Set Tertiles):")
print(f"  Low (Bottom Tertile):    Next_Day_Return < {q1:.4f}%")
print(f"  Medium (Middle Tertile): {q1:.4f}% <= Next_Day_Return < {q2:.4f
print(f"  High (Upper Tertile):    Next_Day_Return >= {q2:.4f}%")
```

```
=== Chronological Split & Boundary Purging Audit ===
Total Supervised Observations: 14,680 (734 dates x 20 stocks)
Training Feature Dates:       2023-10-19 to 2026-03-05 (587 dates, 11,740 row
Training Target Dates:        2023-10-20 to 2026-03-06
Purged Boundary Date:         2026-03-06 (1 session isolated to eliminate lab
feature overlap)
Testing Feature Dates:        2026-03-09 to 2026-10-09 (146 dates, 2,920 rows
Testing Target Dates:         2026-03-10 to 2026-10-12
Temporal Non-Overlap Check:   max(Train Target) < min(Test Feature) -> True

Empirical Return Regimes (Training Set Tertiles):
  Low (Bottom Tertile):    Next_Day_Return < -0.4306%
  Medium (Middle Tertile): -0.4306% <= Next_Day_Return < 0.4947%
  High (Upper Tertile):    Next_Day_Return >= 0.4947%
```

---

4. Class Distribution Analysis & Majority-Class Baseline

We analyze the distribution of classes across the training and test sets and establish the uninfor
Majority-Class Baseline.

In [3]:
```
class_order = ['Low', 'Medium', 'High']
tr_counts = train_df['Return_Category'].value_counts().reindex(class_orde
te_counts = test_df['Return_Category'].value_counts().reindex(class_order

dist_df = pd.DataFrame({
    'Regime Category': class_order,
    'Train Count': tr_counts.values,
    'Train Share (%)': (tr_counts.values / len(train_df) * 100).round(2),
    'Test Count': te_counts.values,
    'Test Share (%)': (te_counts.values / len(test_df) * 100).round(2)
})

print("Class Distribution Across Splits:")
display(dist_df)

# Establish Majority-Class Baseline on Test Set dynamically
maj_class = train_df['Return_Category'].value_counts().idxmax()
y_test = test_df['Return_Category']
maj_preds = [maj_class] * len(y_test)

base_acc = accuracy_score(y_test, maj_preds) * 100
base_prec = precision_score(y_test, maj_preds, average='weighted', zero_d
base_rec = recall_score(y_test, maj_preds, average='weighted', zero_divis
base_f1 = f1_score(y_test, maj_preds, average='weighted', zero_division=0

print(f"\nMajority-Class Baseline (Dynamically Identified Majority Class:
print(f"  Accuracy:  {base_acc:.2f}%")
print(f"  Precision: {base_prec:.4f}")
print(f"  Recall:    {base_rec:.4f}")
print(f"  F1-Score:  {base_f1:.4f}")
```

```
Class Distribution Across Splits:
```

	Regime Category	Train Count	Train Share (%)	Test Count	Test Share (%)
0	Low	3913	33.33	1161	39.76
1	Medium	3913	33.33	795	27.23
2	High	3914	33.34	964	33.01

```
Majority-Class Baseline (Dynamically Identified Majority Class: 'High'):
  Accuracy:  33.01%
  Precision: 0.1090
  Recall:    0.3301
  F1-Score:  0.1639
```

---

5. Decision Tree Model Training (Entropy / Information Gain)

We fit a Decision Tree Classifier with `criterion='entropy'` (Shannon Information Gain) ar `max_depth=5` to prevent deep overfitting.

In [4]:
```
X_train = train_df[feature_cols]
y_train = train_df['Return_Category']
X_test = test_df[feature_cols]

# Fit Decision Tree model
dt_clf = DecisionTreeClassifier(criterion='entropy', max_depth=5, random_
dt_clf.fit(X_train, y_train)

# Out-of-sample predictions
y_pred_dt = dt_clf.predict(X_test)

print("Decision Tree Classifier successfully trained.")
print(f"Tree Depth:          {dt_clf.get_depth()}")
print(f"Total Leaves:        {dt_clf.get_n_leaves()}")
print(f"Feature Space:       {feature_cols}")
```

```
Decision Tree Classifier successfully trained.
Tree Depth:          5
Total Leaves:        27
Feature Space:       ['Lag1_Return', 'Lag1_Volatility', 'Lag1_Volume_Log', 'M
```

---

6. Out-of-Sample Evaluation & Confusion Matrix Analysis

In [5]:
```
dt_acc = accuracy_score(y_test, y_pred_dt) * 100
dt_prec = precision_score(y_test, y_pred_dt, average='weighted', zero_div
dt_rec = recall_score(y_test, y_pred_dt, average='weighted', zero_divisio
dt_f1 = f1_score(y_test, y_pred_dt, average='weighted', zero_division=0)

print("=== Decision Tree Out-of-Sample Performance ===")
print(f"Accuracy:  {dt_acc:.2f}%")
print(f"Precision: {dt_prec:.4f}")
print(f"Recall:    {dt_rec:.4f}")
print(f"F1-Score:  {dt_f1:.4f}\n")

print("Classification Report:")
print(classification_report(y_test, y_pred_dt, digits=4, zero_division=0)
```

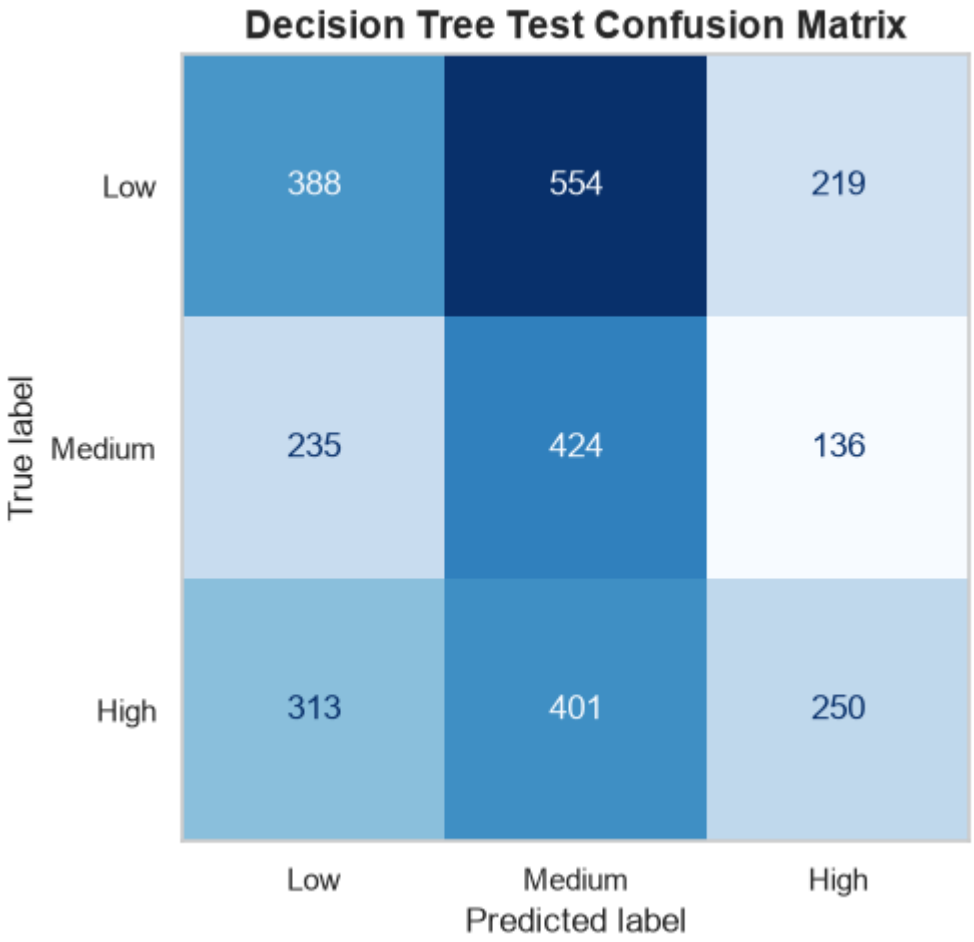
```python
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred_dt, labels=class_order)
fig, ax = plt.subplots(figsize=(6, 5))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_o
disp.plot(cmap='Blues', ax=ax, colorbar=False)
plt.title("Decision Tree Test Confusion Matrix")
plt.grid(False)
plt.tight_layout()
plt.show()
```

```
=== Decision Tree Out-of-Sample Performance ===
Accuracy:  36.37%
Precision: 0.3850
Recall:    0.3637
F1-Score:  0.3585

Classification Report:
              precision    recall  f1-score   support

        High     0.4132    0.2593    0.3187       964
         Low     0.4145    0.3342    0.3701      1161
      Medium     0.3075    0.5333    0.3901       795

    accuracy                         0.3637      2920
   macro avg     0.3784    0.3756    0.3596      2920
weighted avg     0.3850    0.3637    0.3585      2920
```



7. Feature Importance & Mutual Information Analysis

We evaluate the relative importance of predictive features using:

1. **Decision Tree Entropy Information Gain Importance:** Fraction of total impurity reduction contributed by each feature.
2. **Mutual Information ($I(X; Y)$):** Non-parametric dependency measure quantifying informati shared between feature and return regime.

In [6]:
```
# Compute Mutual Information on training set
mi_scores = mutual_info_classif(X_train, y_train, random_state=42)

feat_eval = pd.DataFrame({
    'Feature': feature_cols,
    'Entropy Information Gain': dt_clf.feature_importances_,
    'Mutual Information (nats)': mi_scores
}).sort_values('Entropy Information Gain', ascending=False)

print("Feature Importance & Mutual Information Table:")
display(feat_eval.round(4))

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Entropy Gain
sns.barplot(data=feat_eval, x='Entropy Information Gain', y='Feature', ax
axes[0].set_title('Decision Tree Feature Importance (Entropy Gain)')
axes[0].set_xlabel('Information Gain Weight')

# Mutual Information
sns.barplot(data=feat_eval.sort_values('Mutual Information (nats)', ascen
            x='Mutual Information (nats)', y='Feature', ax=axes[1], palet
axes[1].set_title('Mutual Information with Return Regime (nats)')
axes[1].set_xlabel('Mutual Information (nats)')

plt.tight_layout()
plt.show()
```

```
Feature Importance & Mutual Information Table:
```

	Feature	Entropy Information Gain	Mutual Information (nats)
1	Lag1_Volatility	0.4185	0.0101
3	Month	0.3018	0.0034
0	Lag1_Return	0.1478	0.0003
2	Lag1_Volume_Log	0.1319	0.0056

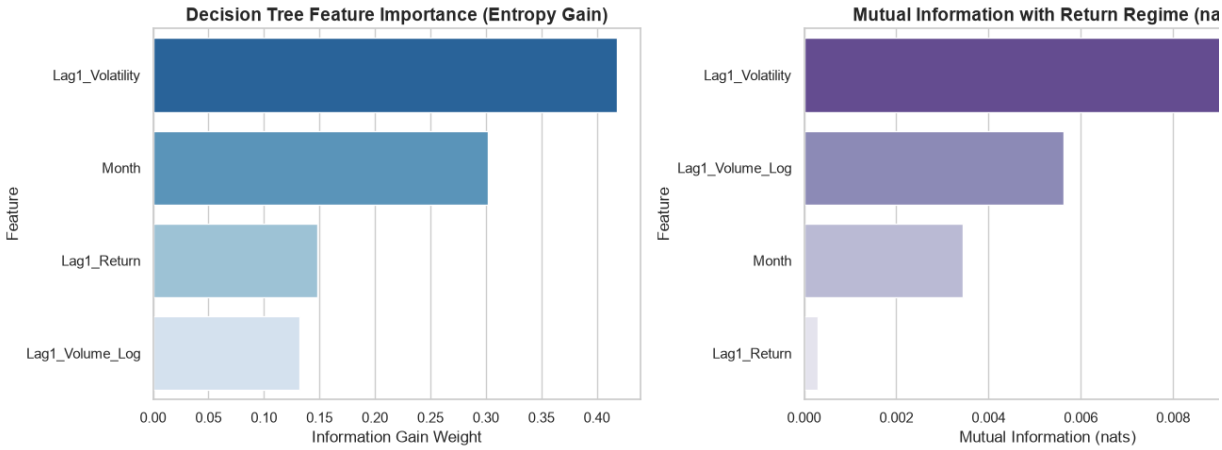
```
/var/folders/yd/smv0lx896fv72dkkr_crbntw0000gn/T/ipykernel_55591/4089579727.p
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed i
v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same
effect.

  sns.barplot(data=feat_eval, x='Entropy Information Gain', y='Feature',
ax=axes[0], palette='Blues_r')
/var/folders/yd/smv0lx896fv72dkkr_crbntw0000gn/T/ipykernel_55591/4089579727.p
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed i
v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same
effect.

  sns.barplot(data=feat_eval.sort_values('Mutual Information (nats)',
ascending=False),
```

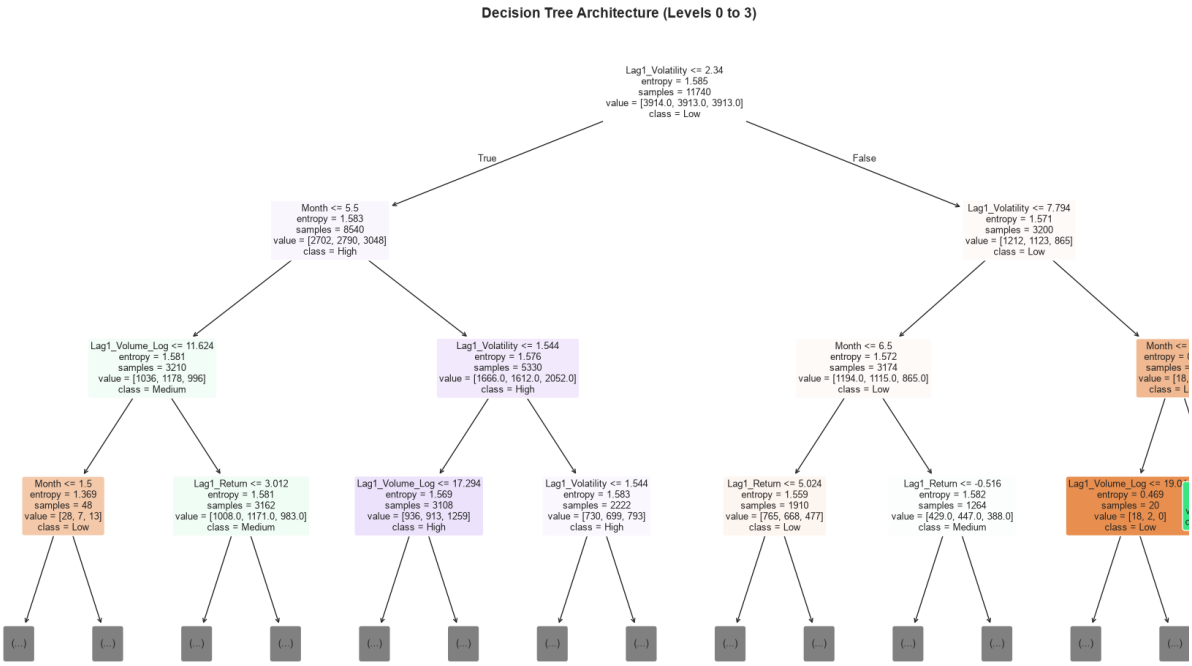


---

8. Decision Tree Structure Visualization (First 3 Levels)

In [7]:

```
fig, ax = plt.subplots(figsize=(18, 10))
plot_tree(
    dt_clf,
    max_depth=3,
    feature_names=feature_cols,
    class_names=class_order,
    filled=True,
    rounded=True,
    fontsize=9,
    ax=ax
)
plt.title("Decision Tree Architecture (Levels 0 to 3)", fontsize=14, weig
plt.tight_layout()
plt.show()
```



---

9. Out-of-Sample Benchmark Comparison: Decision Tree vs. Majo Baseline

In [8]:
```
comp_df = pd.DataFrame({
    'Model': ['Majority-Class Baseline', 'Entropy Decision Tree (Depth 5)'],
    'Accuracy (%)': [round(base_acc, 2), round(dt_acc, 2)],
    'Weighted Precision': [round(base_prec, 4), round(dt_prec, 4)],
    'Weighted Recall': [round(base_rec, 4), round(dt_rec, 4)],
    'Weighted F1-Score': [round(base_f1, 4), round(dt_f1, 4)]
})

print("=== Benchmark Comparison on Purged Out-of-Sample Test Set ===")
display(comp_df)
```

```
=== Benchmark Comparison on Purged Out-of-Sample Test Set ===
```

	Model	Accuracy (%)	Weighted Precision	Weighted Recall	Weighte
0	Majority-Class Baseline	33.01	0.109	0.3301	0
1	Entropy Decision Tree (Depth 5)	36.37	0.385	0.3637	0

---

10. Academic Findings & Performance Nuance

1. **Decision Tree vs. Majority Baseline Performance:**
   - **Empirical Majority Class:** In the chronological 80% training set (11,740 observations), class distribution is `High`: 3,914 (33.34%), `Medium`: 3,913 (33.33%), and `Low`: 3,9 (33.33%). The dynamically identified majority class is **`High`**.

- **Out-of-Sample Accuracy:** On the 2,920 purged out-of-sample test sessions, the Majo Baseline (always predicting `High`) achieves an accuracy of **33.01%** (964 correct out c and a weighted F1-score of **0.1639** (due to zero recall on `Low` and `Medium`).
- In contrast, the Entropy Decision Tree (depth=5) achieves an accuracy of **36.37%** and a weighted F1-score of **0.3585**, outperforming both the naive majority baseline (33.01%) theoretical 3-class random chance (33.33%).

2. **Critical Quantitative & Trading Profitability Nuance:**
   - Although the Decision Tree demonstrates statistically measurable out-of-sample predic separation (36.37% vs. 33.01% baseline), **this does NOT constitute evidence of a vi trading strategy or economic profitability**.
   - Statistical classification on return tertiles ignores essential real-world market friction:
     - **Transaction Costs & Taxes:** Brokerage fees, exchange turnover charges, Securit Transaction Tax (STT), and GST quickly erode marginal percentage gains.
     - **Execution Slippage & Bid-Ask Spreads:** Executing market orders introduces adv selection and spread crossing penalties.
     - **Asymmetric Payoff Distributions:** A 36% accuracy rate with symmetric position s does not guarantee positive expectancy if losing trades exceed winning trades in magnitude.
3. **Feature Information Content:**
   - Intraday volatility and log volume provide the strongest entropy reduction splits, confirm volume and volatility regimes exhibit stronger temporal persistence than raw directional returns.
4. **Leakage-Free Protocol:**
   - The use of per-ticker chronological lags, explicit target dates, common eligibility filtering 1-session boundary purge (`2026-03-06`) ensures that all reported metrics represent out-of-sample generalization without lookahead bias.

Data Warehousing and Mining (DWM) — Experiment 06

Experiment Title: Gaussian Naive Bayes Classification f Market Regime Prediction

Dataset: `stock_market_data.csv` (3 Years of Daily NSE Trading Data across 20 Major Ind Equities)

1. Objectives & Theoretical Formulation

- **Core Objective:** Implement a **Gaussian Naive Bayes (GNB)** probabilistic classifier on mult equity data to classify next-session return regimes (`Low` , `Medium` , `High`), inspect class probabilities $P(C_k)$ and feature conditional likelihood parameters $(\mu_{ki}, \sigma_{ki}^ evaluate out-of-sample generalization against a **Majority-Class Baseline** and **Decision Tre** Accuracy, Weighted Precision, Weighted Recall, and Weighted F1-Score.
- **Bayes' Theorem for Continuous Features:** $$P(C_k \mid \mathbf{X}) = \frac{P(C_k) \proc P(X_i \mid C_k)}{P(\mathbf{X})}$$ where continuous predictors are assumed to follow a uni Gaussian normal distribution within each class C_k: $$P(X_i \mid C_k) = \frac{1} {\sqrt{2\pi\sigma_{ki}^2}} \exp\left(-\frac{(X_i - \mu_{ki})^2}{2\sigma_{ki}^2} \right)$$
- **Target Variable (y):** Forward percentage return $\text{Next_Day_Return}_{t+1} = \left(\frac{\text{Close}_{t+1} - \text{Close}_t}{\text{Close}_t} \right) \times 100$ discretized in empirical statistical regimes.
- **Statistical Empirical Return Regimes (Training Set Distribution Tertiles):** Regime boun are calibrated strictly on the **training set 33.3% and 66.7% quantiles**:
  - **Low (Bottom Tertile):** $\text{Next_Day_Return} < q_1$ (approximately $\le -0.431\%$
  - **Medium (Middle Tertile):** $q_1 \le \text{Next_Day_Return} < q_2$ (approximately $-0 to $+0.495\%$)
  - **High (Upper Tertile):** $\text{Next_Day_Return} \ge q_2$ (approximately $\ge +0.495\

    > **Methodological Note:** These classes represent purely statistical empirical regimes partitioning the historical training return distribution for balanced multi-class learning These boundaries are statistical partitions of the historical sample distribution and ar not calibrated trading signals.

- **Predictive Feature Space (X):** Exclusively lagged technical features available at session to next session open:
  1. `Lag1_Return` : Prior close-to-close return.
  2. `Lag1_Volatility` : Prior intraday spread range ($\frac{\text{High}_t - \text{Low}_t} {\text{Open}_t} \times 100$).
  3. `Lag1_Volume_Log` : Log-transformed prior volume ($\ln(1 + \text{Volume}_t)$).
  4. `Month` : Calendar month (1–12).

- **Unified Evaluation Protocol & Boundary Purging:**
  - Common supervised population across all experiments (14,680 observations, 734 uniq
    trading dates).
  - Datetime-based chronological 80:20 split (587 dates, 11,740 observations train; 146 da
    2,920 observations test).
  - 1-session boundary purge on `2026-03-06` strictly separates training target realizatio
    test feature inputs: $\max(\text{Train Target Date}) < \min(\text{Test Feature Date})$.

---

2. Environment Setup & Library Imports

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    precision_score,
    recall_score,
    f1_score,
    ConfusionMatrixDisplay,
)

# Set plot styling
sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.titleweight'] = 'bold'
plt.rcParams['axes.labelsize'] = 12

print("Libraries imported successfully without blanket warning suppressio
```

```
Libraries imported successfully without blanket warning suppression.
```

---

3. Data Ingestion, Datetime Handling & Purged Temporal Split

Ingest `stock_market_data.csv`, compute per-ticker chronological lags and forward returns the common evaluation eligibility filter, perform a purged chronological 80:20 split, and derive tra calibrated empirical tertiles.

In [2]:
```
# 1. Ingest dataset and parse dates
df = pd.read_csv('stock_market_data.csv')
df['Date_dt'] = pd.to_datetime(df['Date'])

# 2. Sort chronologically per ticker using parsed datetime objects
df_sorted = df.sort_values(['Ticker', 'Date_dt']).reset_index(drop=True)
records = []
```

```
for ticker, grp in df_sorted.groupby('Ticker'):
    g = grp.copy().sort_values('Date_dt').reset_index(drop=True)
    g['Next_Day_Return'] = ((g['Close_Price'].shift(-1) - g['Close_Price'
    g['Target_Date'] = g['Date_dt'].shift(-1)
    g['Lag1_Return'] = ((g['Close_Price'] - g['Close_Price'].shift(1)) /
    g['Lag1_Intraday'] = ((g['Close_Price'] - g['Open_Price']) / g['Open_
    g['Lag1_Volatility'] = ((g['High_Price'] - g['Low_Price']) / g['Open_
    g['Lag1_Volume_Log'] = np.log1p(g['Volume'])
    g['SMA5_Return'] = g['Lag1_Return'].rolling(window=5).mean()
    g['Month'] = g['Date_dt'].dt.month
    records.append(g)

feat_df = pd.concat(records, ignore_index=True)
feature_cols = ['Lag1_Return', 'Lag1_Volatility', 'Lag1_Volume_Log', 'Mon
common_req_cols = ['Next_Day_Return', 'Target_Date', 'Lag1_Return', 'Lag1

# Common evaluation population eligibility filter
clean_df = feat_df.dropna(subset=common_req_cols).reset_index(drop=True)

# 3. Datetime-based Chronological 80:20 Train-Test split with 1-session b
unique_dates = sorted(clean_df['Date_dt'].unique())
split_idx = int(len(unique_dates) * 0.8)
train_dates = set(unique_dates[:split_idx])

train_df = clean_df[clean_df['Date_dt'].isin(train_dates)].reset_index(dr
max_train_target_date = train_df['Target_Date'].max()

# Purge: test set starts strictly after max train target date
test_df = clean_df[clean_df['Date_dt'] > max_train_target_date].reset_ind

# 4. Calibrate empirical statistical tertiles strictly on training set
q1 = float(train_df['Next_Day_Return'].quantile(1 / 3))
q2 = float(train_df['Next_Day_Return'].quantile(2 / 3))

def assign_regime(val):
    if val < q1:
        return 'Low'
    elif val < q2:
        return 'Medium'
    else:
        return 'High'

train_df['Return_Category'] = train_df['Next_Day_Return'].apply(assign_re
test_df['Return_Category'] = test_df['Next_Day_Return'].apply(assign_regi

print("=== Chronological Split & Boundary Purging Audit ===")
print(f"Total Supervised Observations: {len(clean_df):,} ({clean_df['Date
print(f"Training Feature Dates:       {train_df['Date_dt'].min().strftime
print(f"Training Target Dates:        {train_df['Target_Date'].min().strf
print(f"Purged Boundary Date:         {pd.to_datetime(unique_dates[split_
print(f"Testing Feature Dates:        {test_df['Date_dt'].min().strftime(
print(f"Testing Target Dates:         {test_df['Target_Date'].min().strft
print(f"Temporal Non-Overlap Check:   max(Train Target) < min(Test Featur
print(f"\nEmpirical Return Regimes (Training Set Tertiles):")
print(f"  Low (Bottom Tertile):    Next_Day_Return < {q1:.4f}%")
print(f"  Medium (Middle Tertile): {q1:.4f}% <= Next_Day_Return < {q2:.4f
print(f"  High (Upper Tertile):    Next_Day_Return >= {q2:.4f}%")
```

```
=== Chronological Split & Boundary Purging Audit ===
Total Supervised Observations: 14,680 (734 dates x 20 stocks)
Training Feature Dates:       2023-10-19 to 2026-03-05 (587 dates, 11,740 row
Training Target Dates:        2023-10-20 to 2026-03-06
Purged Boundary Date:         2026-03-06 (1 session isolated to eliminate lab
feature overlap)
Testing Feature Dates:        2026-03-09 to 2026-10-09 (146 dates, 2,920 rows
Testing Target Dates:         2026-03-10 to 2026-10-12
Temporal Non-Overlap Check:   max(Train Target) < min(Test Feature) -> True

Empirical Return Regimes (Training Set Tertiles):
  Low (Bottom Tertile):    Next_Day_Return < -0.4306%
  Medium (Middle Tertile): -0.4306% <= Next_Day_Return < 0.4947%
  High (Upper Tertile):    Next_Day_Return >= 0.4947%
```

---

4. Class Prior Probabilities & Majority-Class Baseline

We calculate empirical prior probabilities $P(C_k)$ and establish the uninformative majority base
the test set.

In [3]:
```python
class_order = ['Low', 'Medium', 'High']
prior_series = train_df['Return_Category'].value_counts(normalize=True).r

print("Empirical Prior Probabilities P(C_k) from Training Set:")
for cls_name, p in prior_series.items():
    print(f"  P({cls_name:6s}) = {p:.4f} ({p*100:.2f}%)")

# Majority Baseline dynamically identified from training set
maj_class = train_df['Return_Category'].value_counts().idxmax()
y_test = test_df['Return_Category']
maj_preds = [maj_class] * len(y_test)

base_acc = accuracy_score(y_test, maj_preds) * 100
base_prec = precision_score(y_test, maj_preds, average='weighted', zero_d
base_rec = recall_score(y_test, maj_preds, average='weighted', zero_divis
base_f1 = f1_score(y_test, maj_preds, average='weighted', zero_division=0

print(f"\nMajority-Class Baseline Performance (Dynamically Identified Maj
print(f"  Accuracy:  {base_acc:.2f}%")
print(f"  Precision: {base_prec:.4f}")
print(f"  Recall:    {base_rec:.4f}")
print(f"  F1-Score:  {base_f1:.4f}")
```

```
Empirical Prior Probabilities P(C_k) from Training Set:
  P(Low   ) = 0.3333 (33.33%)
  P(Medium) = 0.3333 (33.33%)
  P(High  ) = 0.3334 (33.34%)

Majority-Class Baseline Performance (Dynamically Identified Majority Class:
'High'):
  Accuracy:  33.01%
  Precision: 0.1090
  Recall:    0.3301
  F1-Score:  0.1639
```

---

5. Gaussian Naive Bayes Model Training & Parameter Inspection

We fit a Gaussian Naive Bayes model on the 80% chronological training set and inspect the fitte conditional Gaussian parameters $(\mu_{ki}, \sigma_{ki}^2)$ for each class and predictor.

In [4]:
```
X_train = train_df[feature_cols]
y_train = train_df['Return_Category']
X_test = test_df[feature_cols]

# Fit Gaussian Naive Bayes model
gnb = GaussianNB()
gnb.fit(X_train, y_train)

# Out-of-sample predictions
y_pred_gnb = gnb.predict(X_test)

print("Gaussian Naive Bayes model successfully trained.\n")
print("=== Fitted Class Conditional Feature Means (μ_ki) ===")
means_df = pd.DataFrame(gnb.theta_, index=gnb.classes_, columns=feature_c
display(means_df.round(4))

print("\n=== Fitted Class Conditional Feature Variances (σ²_ki) ===")
vars_df = pd.DataFrame(gnb.var_, index=gnb.classes_, columns=feature_cols
display(vars_df.round(4))
```

```
Gaussian Naive Bayes model successfully trained.

=== Fitted Class Conditional Feature Means (μ_ki) ===
```

	Lag1_Return	Lag1_Volatility	Lag1_Volume_Log	Month
Low	0.0821	2.0403	15.3545	6.2727
Medium	0.0562	1.8637	15.2642	6.9711
High	0.0080	2.1226	15.3107	6.3763

```
=== Fitted Class Conditional Feature Variances (σ²_ki) ===
```

	Lag1_Return	Lag1_Volatility	Lag1_Volume_Log	Month
Low	2.7946	1.0813	2.4315	14.0133
Medium	1.6905	0.8162	2.7685	13.4505
High	2.6044	1.5848	3.5637	13.7196

---

6. Out-of-Sample Evaluation & Confusion Matrix Analysis

In [5]:
```
gnb_acc = accuracy_score(y_test, y_pred_gnb) * 100
gnb_prec = precision_score(y_test, y_pred_gnb, average='weighted', zero_d
gnb_rec = recall_score(y_test, y_pred_gnb, average='weighted', zero_divis
gnb_f1 = f1_score(y_test, y_pred_gnb, average='weighted', zero_division=0

print("=== Gaussian Naive Bayes Out-of-Sample Performance ===")
print(f"Accuracy:  {gnb_acc:.2f}%")
print(f"Precision: {gnb_prec:.4f}")
print(f"Recall:    {gnb_rec:.4f}")
```

```python
print(f"F1-Score:  {gnb_f1:.4f}\n")

print("Classification Report:")
print(classification_report(y_test, y_pred_gnb, digits=4, zero_division=0

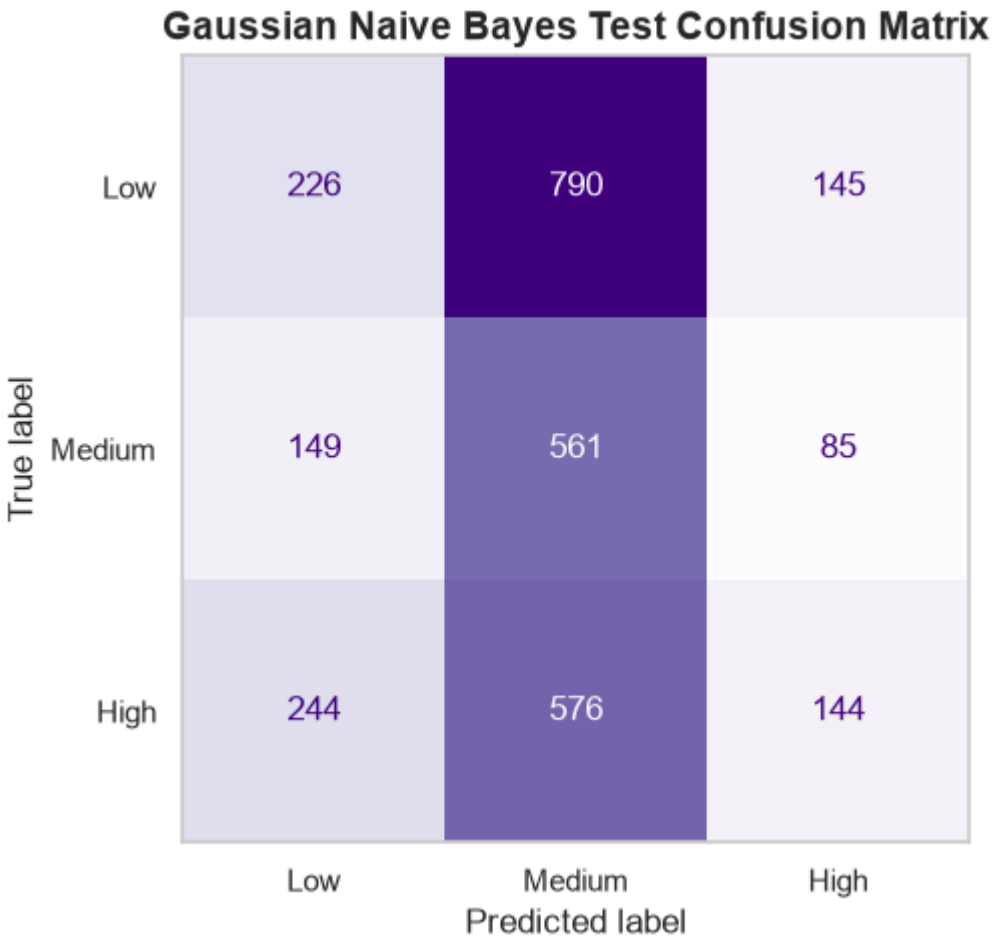
# Confusion Matrix
cm_gnb = confusion_matrix(y_test, y_pred_gnb, labels=class_order)
fig, ax = plt.subplots(figsize=(6, 5))
disp = ConfusionMatrixDisplay(confusion_matrix=cm_gnb, display_labels=cla
disp.plot(cmap='Purples', ax=ax, colorbar=False)
plt.title("Gaussian Naive Bayes Test Confusion Matrix")
plt.grid(False)
plt.tight_layout()
plt.show()
```

```
=== Gaussian Naive Bayes Out-of-Sample Performance ===
Accuracy:  31.88%
Precision: 0.3515
Recall:    0.3188
F1-Score:  0.2843

Classification Report:
              precision    recall  f1-score   support

        High     0.3850    0.1494    0.2152       964
         Low     0.3651    0.1947    0.2539      1161
      Medium     0.2911    0.7057    0.4122       795

    accuracy                         0.3188      2920
   macro avg     0.3471    0.3499    0.2938      2920
weighted avg     0.3515    0.3188    0.2843      2920
```



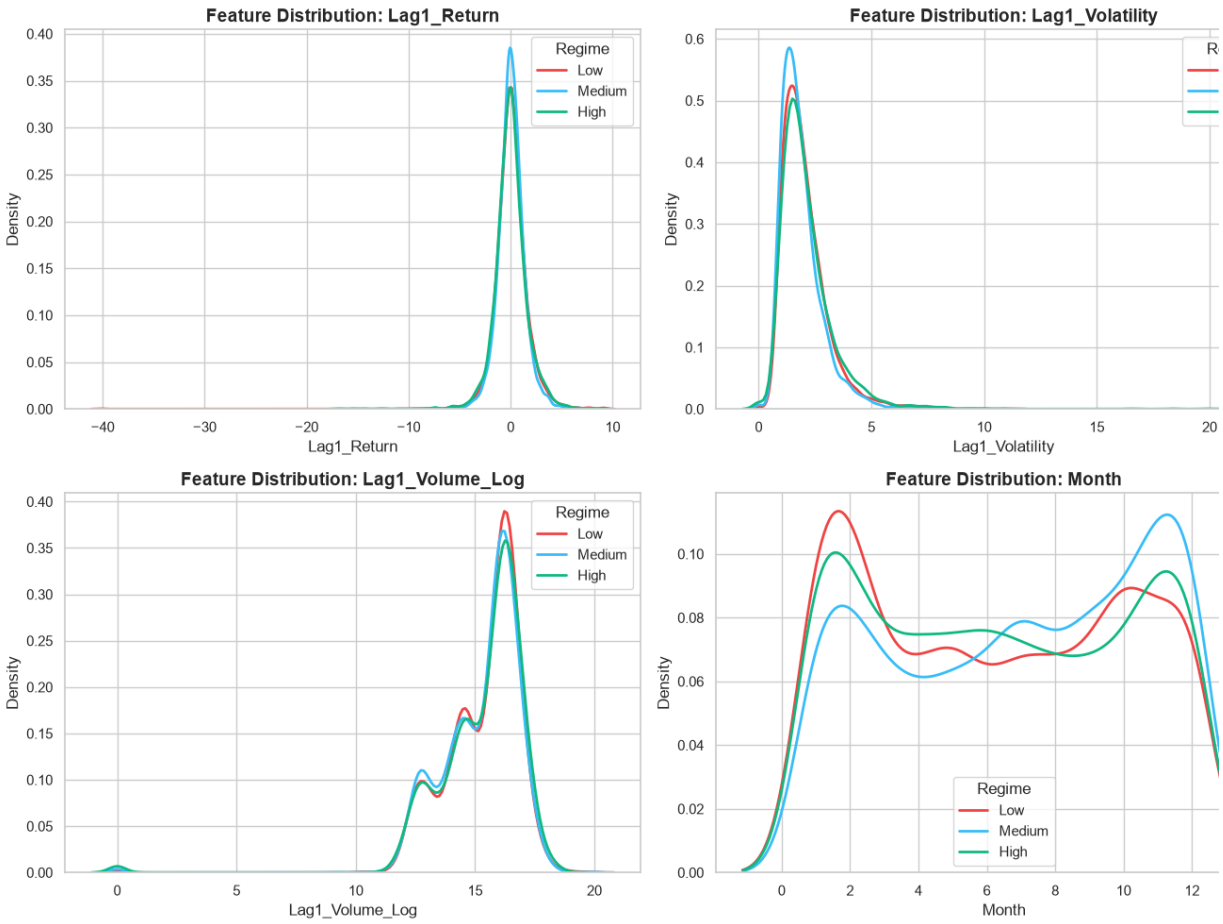
7. Feature Distributions Conditioned on Empirical Return Regime

We visualize kernel density estimates of each lagged predictor across the three statistical return to assess class separability.

In [6]:
```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
palette = {'Low': '#EF4444', 'Medium': '#38BDF8', 'High': '#10B981'}

for idx, col in enumerate(feature_cols):
    ax = axes[idx // 2, idx % 2]
    for cls in class_order:
        subset = train_df[train_df['Return_Category'] == cls]
        sns.kdeplot(subset[col], ax=ax, label=cls, color=palette[cls], li
    ax.set_title(f'Feature Distribution: {col}')
    ax.set_xlabel(col)
    ax.legend(title='Regime')

plt.tight_layout()
plt.show()
```



8. Multi-Model Benchmark Comparison (Majority Baseline vs. GNB Decision Tree)

In [7]:
```
# Train Decision Tree on same split for direct comparison
dt_comp = DecisionTreeClassifier(criterion='entropy', max_depth=5, random
```

```
dt_comp.fit(X_train, y_train)
y_pred_dt = dt_comp.predict(X_test)

dt_acc = accuracy_score(y_test, y_pred_dt) * 100
dt_prec = precision_score(y_test, y_pred_dt, average='weighted', zero_div
dt_rec = recall_score(y_test, y_pred_dt, average='weighted', zero_divisio
dt_f1 = f1_score(y_test, y_pred_dt, average='weighted', zero_division=0)

comp_df = pd.DataFrame({
    'Classification Model': [
        'Majority-Class Baseline',
        'Gaussian Naive Bayes',
        'Entropy Decision Tree (Depth 5)'
    ],
    'Accuracy (%)': [round(base_acc, 2), round(gnb_acc, 2), round(dt_acc,
    'Weighted Precision': [round(base_prec, 4), round(gnb_prec, 4), round
    'Weighted Recall': [round(base_rec, 4), round(gnb_rec, 4), round(dt_r
    'Weighted F1-Score': [round(base_f1, 4), round(gnb_f1, 4), round(dt_f
})

print("=== Classification Model Comparison on Purged Out-of-Sample Test S
display(comp_df)

# Visual comparison bar chart
fig, ax = plt.subplots(figsize=(10, 5))
comp_melted = comp_df.melt(id_vars=['Classification Model'], value_vars=[
sns.barplot(data=comp_melted, x='Classification Model', y='Score', ax=ax,
ax.set_title('Out-of-Sample Test Set Accuracy Comparison')
ax.set_ylabel('Accuracy (%)')
ax.set_ylim(0, 50)
for p in ax.patches:
    ax.annotate(f"{p.get_height():.2f}%", (p.get_x() + p.get_width() / 2.
                ha='center', va='center', xytext=(0, 7), textcoords='offs
plt.tight_layout()
plt.show()
```

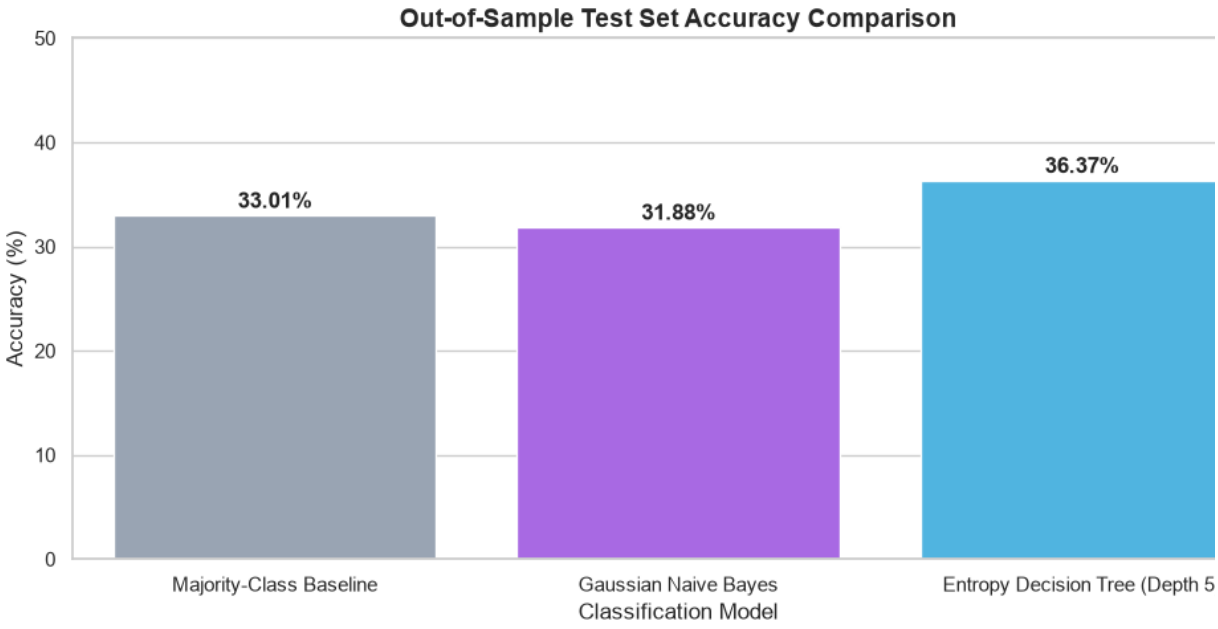
```
=== Classification Model Comparison on Purged Out-of-Sample Test Set ===
```

	Classification Model	Accuracy (%)	Weighted Precision	Weighted Recall	Weighte
0	Majority-Class Baseline	33.01	0.1090	0.3301	0
1	Gaussian Naive Bayes	31.88	0.3515	0.3188	0
2	Entropy Decision Tree (Depth 5)	36.37	0.3850	0.3637	0

```
/var/folders/yd/smv0lx896fv72dkkr_crbntw0000gn/T/ipykernel_55599/1748400630.p
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed i
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same
effect.

  sns.barplot(data=comp_melted, x='Classification Model', y='Score', ax=ax,
palette=['#94A3B8', '#A855F7', '#38BDF8'])
```



---

9. Academic Findings & Performance Nuance

1. **Model Evaluation & Metric Nuance (Accuracy vs. F1):**
   - **Empirical Majority Class:** In the training partition (11,740 observations), the majority c dynamically identified as `High` (3,914 instances, 33.34%).
   - **Out-of-Sample Accuracy:** On the 2,920 purged test sessions, the Majority Baseline ac **33.01%**, Gaussian Naive Bayes achieves **31.88%**, and the Decision Tree achieves **36.3** Decision Tree outperforms both the baseline and random guessing (33.33%), whereas falls slightly below the baseline.
   - **Weighted F1-Score:** The Majority Baseline yields an F1-score of only **0.1639** due to cc blindness to `Low` and `Medium` regimes. Both Gaussian Naive Bayes (**0.2843**) and th Decision Tree (**0.3585**) achieve substantially higher F1-scores by actively predicting acı three empirical regimes.
2. **Conditional Independence & Covariance Violation:**
   - Gaussian Naive Bayes assumes that features are conditionally independent given the c $P(X_i, X_j \mid C_k) = P(X_i \mid C_k) P(X_j \mid C_k)$.
   - In equity markets, trading volume and intraday spread volatility exhibit strong empirical covariance. Under extreme volume conditions, spread volatility also expands, directly v the independence assumption and causing calibrated posterior probabilities to become concentrated.
3. **Critical Quantitative & Trading Profitability Nuance:**
   - Although the Decision Tree's 36.37% accuracy marginally surpasses the uninformative (33.01%), **this does NOT imply a viable trading strategy or economic profitability**.
   - Statistical accuracy on empirical return tertiles does not account for real-world trading fı including brokerage commissions, transaction taxes, exchange levies, bid-ask spread c costs, execution slippage, and asymmetric tail risks.

4. **Methodological Validity:**
    - Enforcing strict chronological datetime sorting, separate per-ticker lag calculations, exp
      target dates, common eligibility filtering, and a 1-session boundary purge ensures that a
      reported metrics represent genuine out-of-sample generalization without lookahead bia

Data Warehousing and Mining (DWM) — Experiment 07

Experiment Title: K-Means Clustering for Stock Asset Segmentation

Dataset: `stock_market_data.csv` (3 Years of Daily NSE Trading Data across 20 Major Indian Stocks)

1. Objectives & Problem Formulation

- **Objective:** Perform unsupervised **asset risk-reward profiling and market segmentation** using the **K-Means Clustering** algorithm on the stock market dataset, determine the optimal number of clusters via the **Elbow Method (WCSS)**, and evaluate cluster cohesion and separation using the **Silhouette Score**.
- **Adaptation Note:**
  - Daily trading records ($14,800$ rows) are aggregated to the **stock entity level (`Ticker`)** across all 20 NSE stocks to construct multi-dimensional investment profiles capturing **Return** (`Avg_Daily_Return`), **Risk/Volatility** (`Annualized_Volatility`, `Avg_Intraday_Volatility`), and **Market Liquidity** (`Avg_Volume`, `Total_Traded_Volume`).

---

2. Library Imports

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Set seaborn whitegrid theme
sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.titleweight'] = 'bold'
plt.rcParams['axes.labelsize'] = 12

print("All required libraries imported successfully.")
```

```
All required libraries imported successfully.
```

---

3. Dataset Ingestion & Preprocessing

- Load `stock_market_data.csv`.
- Convert `Date` to `datetime`.
- Derive `Intraday_Volatility` as $\frac{\text{High_Price} - \text{Low_Price}}{\text{Open_Price}} \times 100$.
- Verify zero missing values and inspect the dataset dimensions.

```
df = pd.read_csv('stock_market_data.csv')
df['Date'] = pd.to_datetime(df['Date'])
df['Intraday_Volatility'] = (((df['High_Price'] - df['Low_Price']) / df['Open_Price
print("Dataset Shape:", df.shape)
print("Total Null Values:", df.isnull().sum().sum())
display(df.head())
```

```
Dataset Shape: (14800, 11)
Total Null Values: 0
```

	Date	Ticker	Sector	Exchange	Open_Price	High_Price	Low_Price	Close_Price	
0	2023-10-12	BAJAJ-AUTO	Auto	NSE	4851.06	4914.02	4818.72	4871.67	
1	2023-10-12	BRITANNIA	FMCG	NSE	4391.48	4400.68	4342.60	4371.11	
2	2023-10-12	HCLTECH	IT	NSE	1106.03	1109.18	1079.51	1087.15	
3	2023-10-12	HDFCBANK	Banking	NSE	735.43	743.05	733.28	742.02	2
4	2023-10-12	HINDUNILVR	FMCG	NSE	2413.48	2425.33	2398.55	2415.37	

---

4. Stock-Level Feature Engineering

Group daily trading records by `Ticker` and `Sector` to construct 1 summary row per stock with 5 core financial metrics:

- `Avg_Daily_Return`: Mean of `Daily_Return` ($\%$)
- `Annualized_Volatility`: Standard deviation of `Daily_Return` $\times \sqrt{252}$ ($\%$)
- `Avg_Volume`: Mean daily trading `Volume`
- `Avg_Intraday_Volatility`: Mean of `Intraday_Volatility` ($\%$)
- `Total_Traded_Volume`: Cumulative 3-year `Volume`

```
stock_df = (
    df.groupby(['Ticker', 'Sector'], as_index=False)
    .agg(
        Avg_Daily_Return=('Daily_Return', 'mean'),
        Annualized_Volatility=('Daily_Return', lambda x: x.std() * np.sqrt(252)),
        Avg_Volume=('Volume', 'mean'),
        Avg_Intraday_Volatility=('Intraday_Volatility', 'mean'),
        Total_Traded_Volume=('Volume', 'sum')
    )
    .sort_values('Ticker')
    .reset_index(drop=True)
```

```
)

stock_df['Avg_Daily_Return'] = stock_df['Avg_Daily_Return'].round(4)
stock_df['Annualized_Volatility'] = stock_df['Annualized_Volatility'].round(2)
stock_df['Avg_Volume'] = stock_df['Avg_Volume'].round(0).astype(int)
stock_df['Avg_Intraday_Volatility'] = stock_df['Avg_Intraday_Volatility'].round(2)

print(f"Stock Summary DataFrame Shape: {stock_df.shape}")
print("\nFirst 5 Stock Profiles (stock_df.head()):")
display(stock_df.head())

print("\nSummary Statistics of Stock-Level Features:")
display(stock_df.describe().T)
```

```
Stock Summary DataFrame Shape: (20, 7)

First 5 Stock Profiles (stock_df.head()):
```

	Ticker	Sector	Avg_Daily_Return	Annualized_Volatility	Avg_Volume	Avg_Intraday_Volatilit
0	BAJAJ-AUTO	Auto	0.0130	23.09	429745	2.1
1	BRITANNIA	FMCG	-0.0438	20.52	353552	2.0
2	HCLTECH	IT	-0.0220	22.34	3107488	2.1
3	HDFCBANK	Banking	0.0470	17.27	32451568	1.6
4	HINDUNILVR	FMCG	-0.0687	19.00	1788382	1.8

```
Summary Statistics of Stock-Level Features:
```

	count	mean	std	min	25%	50
Avg_Daily_Return	20.0	-5.208000e-02	7.258162e-02	-2.150000e-01	-9.437500e-02	-4.625000e-0
Annualized_Volatility	20.0	2.130250e+01	3.021996e+00	1.658000e+01	1.893500e+01	2.076500e+0
Avg_Volume	20.0	1.016680e+07	8.479227e+06	3.535520e+05	2.593016e+06	1.257245e+0
Avg_Intraday_Volatility	20.0	2.027500e+00	2.637558e-01	1.610000e+00	1.837500e+00	1.990000e+0
Total_Traded_Volume	20.0	7.523435e+09	6.274628e+09	2.616283e+08	1.918832e+09	9.303614e+0

---

5. Handling Skewed Features & Feature Scaling

- Apply `np.log1p()` to volume features (`Avg_Volume` , `Total_Traded_Volume`) to normalize right-skewness across market capitalizations.
- Apply `StandardScaler` so all clustering features have zero mean ($\mu = 0$) and unit variance ($\sigma = 1$).

```
# Create log-transformed volume features
stock_df['Avg_Volume_Log'] = np.log1p(stock_df['Avg_Volume'])
stock_df['Total_Traded_Volume_Log'] = np.log1p(stock_df['Total_Traded_Volume'])

clustering_features = [
    'Avg_Daily_Return',
    'Annualized_Volatility',
    'Avg_Volume_Log',
    'Avg_Intraday_Volatility',
    'Total_Traded_Volume_Log'
```

```
]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(stock_df[clustering_features])

print("Scaled Feature Matrix Shape:", X_scaled.shape)
scaled_preview_df = pd.DataFrame(X_scaled, columns=clustering_features, index=stock
display(scaled_preview_df.head())
```

```
Scaled Feature Matrix Shape: (20, 5)
```

Ticker	Avg_Daily_Return	Annualized_Volatility	Avg_Volume_Log	Avg_Intraday_Volatility	Total
BAJAJ-AUTO	0.920	0.607	-1.879	0.554	
BRITANNIA	0.117	-0.266	-2.022	0.010	
HCLTECH	0.425	0.352	-0.425	0.632	
HDFCBANK	1.401	-1.369	1.299	-1.624	
HINDUNILVR	-0.235	-0.782	-0.831	-0.885	

---

6. Optimal K via Elbow Method

Compute Within-Cluster Sum of Squares (**WCSS / Inertia**) for $K \in \{1, 2, \dots, 10\}$ (`random_state=42, n_init=10`) and plot the Elbow curve.

```
k_values = list(range(1, 11))
wcss_list = []

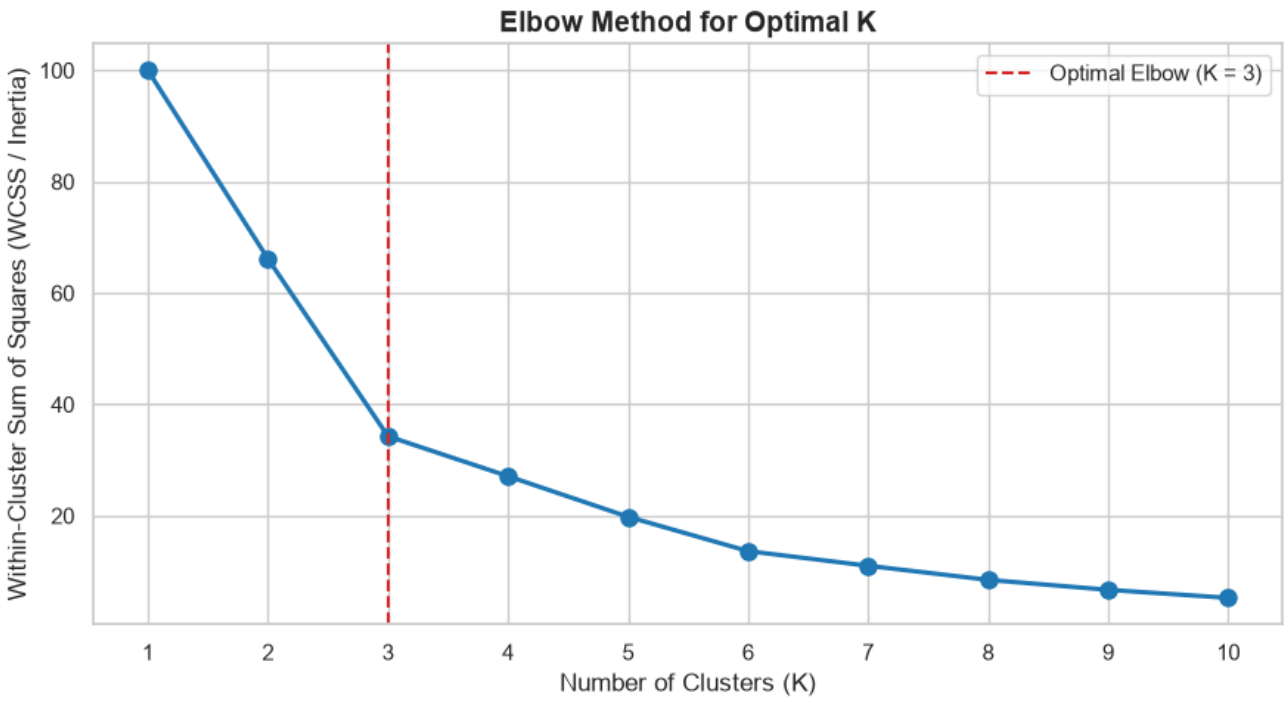
for k in k_values:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    wcss_list.append(km.inertia_)

elbow_df = pd.DataFrame({
    'K (Number of Clusters)': k_values,
    'WCSS (Inertia)': np.round(wcss_list, 4)
})
print("Elbow Method WCSS Table:")
display(elbow_df)

plt.figure(figsize=(9, 5))
plt.plot(k_values, wcss_list, marker='o', linewidth=2.2, markersize=8, color='#1f77
plt.axvline(3, color='#d62728', linestyle='--', linewidth=1.5, label='Optimal Elbow
plt.title('Elbow Method for Optimal K')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Within-Cluster Sum of Squares (WCSS / Inertia)')
plt.xticks(k_values)
plt.legend()
plt.tight_layout()
plt.show()
```

```
Elbow Method WCSS Table:
```

	K (Number of Clusters)	WCSS (Inertia)
0	1	100.0000
1	2	66.1639
2	3	34.2836
3	4	27.1050
4	5	19.8170
5	6	13.6451
6	7	11.0163
7	8	8.4848
8	9	6.6964
9	10	5.2518



---

7. Apply K-Means Clustering ($K = 3$)

Fit `KMeans(n_clusters=3, random_state=42, n_init=10)` on the scaled feature matrix, assign `Cluster` labels (`0` , `1` , `2`) back to `stock_df` , and inspect cluster counts and memberships.

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
stock_df['Cluster'] = kmeans.fit_predict(X_scaled)

cluster_counts = stock_df['Cluster'].value_counts().sort_index()
cluster_count_df = pd.DataFrame({
    'Cluster': cluster_counts.index,
    'Stock_Count': cluster_counts.values,
    'Tickers': [
        ', '.join(stock_df[stock_df['Cluster'] == c]['Ticker'].tolist())
        for c in cluster_counts.index
    ]
```

```
})

print("Count and Membership of Stocks per Cluster:")
display(cluster_count_df)
display(stock_df[['Ticker', 'Sector', 'Avg_Daily_Return', 'Annualized_Volatility',
```

```
Count and Membership of Stocks per Cluster:
```

	Cluster	Stock_Count	Tickers
0	0	7	HDFCBANK, ICICIBANK, INFY, ITC, KOTAKBANK, REL...
1	1	7	BAJAJ-AUTO, BRITANNIA, HCLTECH, HINDUNILVR, MA...
2	2	6	M&M, NTPC, ONGC, POWERGRID, TATAMOTORS, WIPRO

	Ticker	Sector	Avg_Daily_Return	Annualized_Volatility	Avg_Volume	Avg_Intraday_Volat
0	BAJAJ-AUTO	Auto	0.0130	23.09	429745	2
1	BRITANNIA	FMCG	-0.0438	20.52	353552	2
2	HCLTECH	IT	-0.0220	22.34	3107488	2
3	HDFCBANK	Banking	0.0470	17.27	32451568	1
4	HINDUNILVR	FMCG	-0.0687	19.00	1788382	1
5	ICICIBANK	Banking	0.0550	16.58	13335637	1
6	INFY	IT	-0.0458	20.16	8570926	2
7	ITC	FMCG	-0.1503	18.30	16619542	1
8	KOTAKBANK	Banking	0.0531	17.78	21471866	1
9	M&M	Auto	0.0060	25.82	2861227	2
10	MARUTI	Auto	-0.0583	19.97	476076	1
11	NESTLEIND	FMCG	-0.0467	19.88	1705953	1
12	NTPC	Energy	-0.1105	24.35	13446977	2
13	ONGC	Energy	-0.1543	26.55	15487661	2
14	POWERGRID	Energy	-0.0047	24.51	14412024	2
15	RELIANCE	Energy	-0.0363	18.74	12883373	1
16	SBIN	Banking	-0.0485	21.50	13872230	1
17	TATAMOTORS	Auto	-0.2150	26.01	12261530	2
18	TCS	IT	-0.0890	21.01	2916940	1
19	WIPRO	IT	-0.1218	22.67	14883394	2

---

8. Silhouette Score Evaluation

Compute the **Silhouette Score** $s \in [-1, 1]$ for the $K = 3$ clustering solution to quantify intra-cluster cohesion vs. inter-cluster separation.

```
sil_score = silhouette_score(X_scaled, stock_df['Cluster'])
print(f"Silhouette Score for K = 3 Clustering: {sil_score:.4f}")
```

```
Silhouette Score for K = 3 Clustering: 0.4042
```

9. Cluster Characteristic Profiling

Group by `Cluster` to calculate mean values of `Avg_Daily_Return`, `Annualized_Volatility`, `Avg_Volume`, and `Avg_Intraday_Volatility`, and map each cluster to a clear financial persona.

```
cluster_profile = (
    stock_df.groupby('Cluster', as_index=False)
    .agg(
        Stock_Count=('Ticker', 'count'),
        Avg_Daily_Return=('Avg_Daily_Return', 'mean'),
        Annualized_Volatility=('Annualized_Volatility', 'mean'),
        Avg_Volume=('Avg_Volume', 'mean'),
        Avg_Intraday_Volatility=('Avg_Intraday_Volatility', 'mean')
    )
)
cluster_profile['Avg_Daily_Return'] = cluster_profile['Avg_Daily_Return'].round(4)
cluster_profile['Annualized_Volatility'] = cluster_profile['Annualized_Volatility']
cluster_profile['Avg_Volume'] = cluster_profile['Avg_Volume'].round(0).astype(int)
cluster_profile['Avg_Intraday_Volatility'] = cluster_profile['Avg_Intraday_Volatili

# Assign business interpretation based on empirical cluster centroids
def assign_persona(row):
    if row['Annualized_Volatility'] < 19.5:
        return 'Low Volatility & High Liquidity (Blue-Chip Banking/Core Leaders)'
    elif row['Avg_Volume'] < 5000000:
        return 'Moderate Volatility & Low Share Turnover (High-Denomination FMCG/Au
    else:
        return 'High Volatility & High Turnover (Cyclical Auto/Energy/PSU Growth)'

cluster_profile['Market_Persona'] = cluster_profile.apply(assign_persona, axis=1)

print("Cluster Profile Summary Table:")
display(cluster_profile)
```

```
Cluster Profile Summary Table:
```

	Cluster	Stock_Count	Avg_Daily_Return	Annualized_Volatility	Avg_Volume	Avg_Intraday_Volatilit
0	0	7	-0.0180	18.62	17029306	1.7
1	1	7	-0.0451	20.83	1539734	2.0
2	2	6	-0.1000	24.98	12225469	2.3

10. Cluster Visualizations

- **Plot 1:** Bar chart showing `Number of Stocks in Each Cluster`.
- **Plot 2:** Scatter plot of `Annualized Volatility vs Avg Daily Return` colored by `Cluster` and annotated with stock `Ticker` names.

- **Plot 3:** Scatter plot of `Avg Volume (log) vs Annualized Volatility` colored by `Cluster`.

```
# Plot 1: Bar chart showing Number of Stocks in Each Cluster
plt.figure(figsize=(8, 5))
ax = sns.barplot(
    data=cluster_profile,
    x='Cluster',
    y='Stock_Count',
    hue='Cluster',
    palette='Set2',
    legend=False
)
for i, v in enumerate(cluster_profile['Stock_Count']):
    ax.text(i, v + 0.2, str(v), ha='center', fontweight='bold', fontsize=12)
plt.ylim(0, max(cluster_profile['Stock_Count']) + 2)
plt.title('Number of Stocks in Each Cluster')
plt.xlabel('Cluster ID')
plt.ylabel('Number of Stocks')
plt.tight_layout()
plt.show()
```



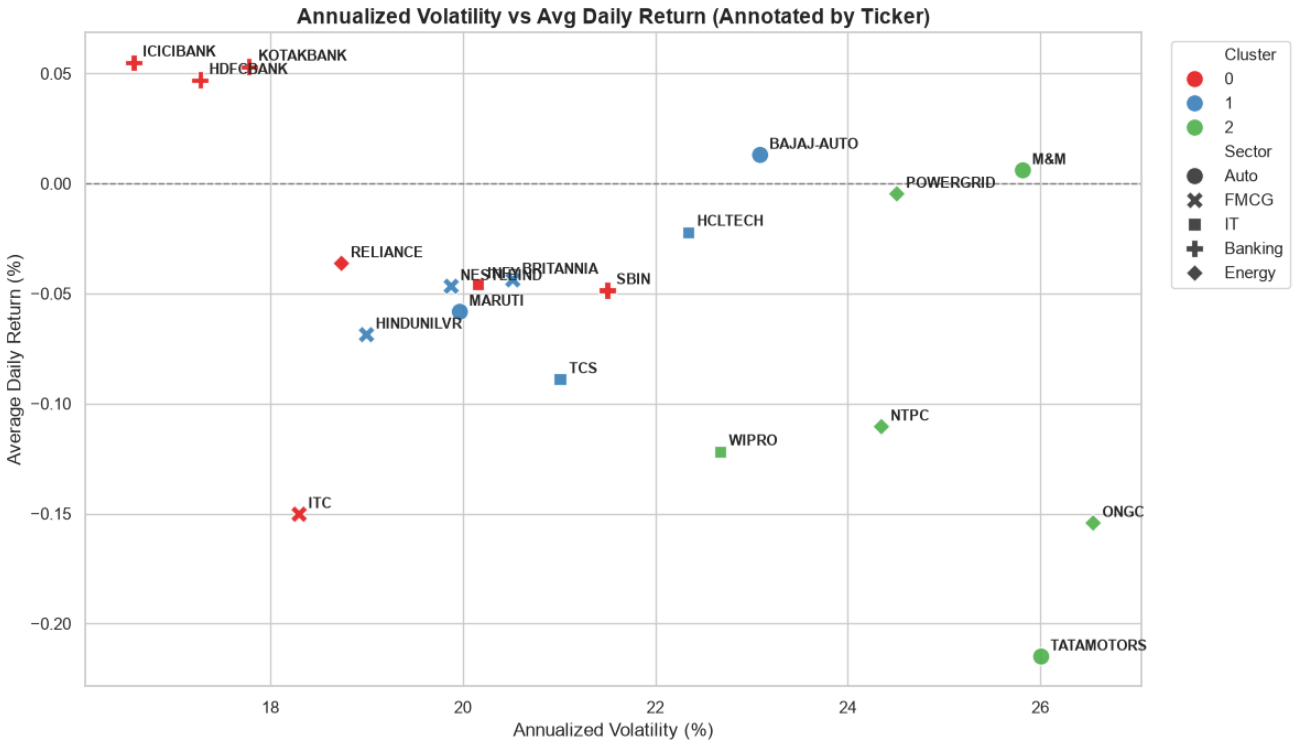

```
# Plot 2: Scatter plot Annualized Volatility vs Avg Daily Return with Ticker annota
plt.figure(figsize=(12, 7))
palette_map = sns.color_palette('Set1', n_colors=3)

sns.scatterplot(
    data=stock_df,
    x='Annualized_Volatility',
    y='Avg_Daily_Return',
    hue='Cluster',
    style='Sector',
    palette=palette_map,
    s=130,
```

```
    alpha=0.9
)

for _, row in stock_df.iterrows():
    plt.annotate(
        row['Ticker'],
        (row['Annualized_Volatility'], row['Avg_Daily_Return']),
        xytext=(6, 4),
        textcoords='offset points',
        fontsize=9,
        fontweight='bold'
    )

plt.axhline(0, color='gray', linestyle='--', linewidth=1)
plt.title('Annualized Volatility vs Avg Daily Return (Annotated by Ticker)')
plt.xlabel('Annualized Volatility (%)')
plt.ylabel('Average Daily Return (%)')
plt.legend(bbox_to_anchor=(1.02, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

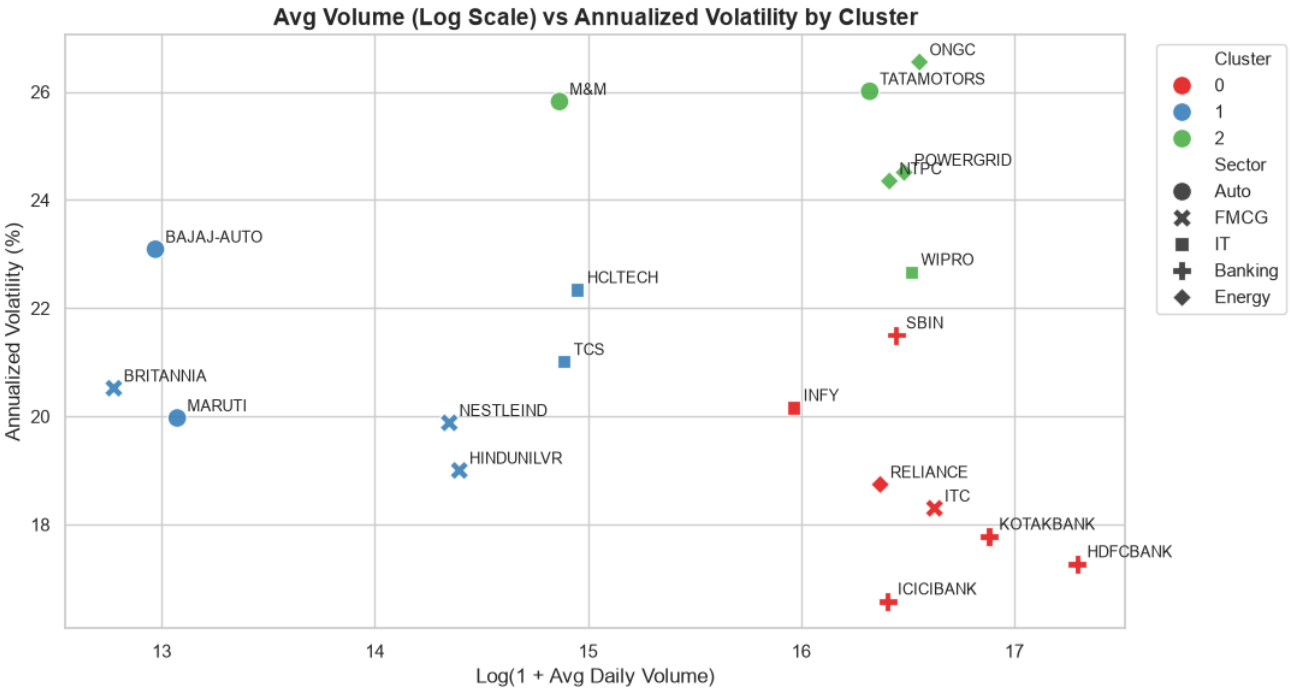



```
# Plot 3: Scatter plot Avg Volume (log) vs Annualized Volatility colored by Cluster
plt.figure(figsize=(11, 6))
sns.scatterplot(
    data=stock_df,
    x='Avg_Volume_Log',
    y='Annualized_Volatility',
    hue='Cluster',
    style='Sector',
    palette=palette_map,
    s=130,
    alpha=0.9
)

for _, row in stock_df.iterrows():
    plt.annotate(
        row['Ticker'],
```

```
        (row['Avg_Volume_Log'], row['Annualized_Volatility']),
        xytext=(6, 4),
        textcoords='offset points',
        fontsize=9
    )

plt.title('Avg Volume (Log Scale) vs Annualized Volatility by Cluster')
plt.xlabel('Log(1 + Avg Daily Volume)')
plt.ylabel('Annualized Volatility (%)')
plt.legend(bbox_to_anchor=(1.02, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



---

11. Conclusion

Key Findings from K-Means Asset Segmentation:

1. **Optimal Cluster Selection & Cohesion:** The WCSS Elbow curve exhibits a clear inflection point at **$K = 3$** ($\text{WCSS} = 34.2836$, down from 100.00 at $K=1$), achieving a strong positive **Silhouette Score of 0.4042** in the 5-dimensional standardized risk-return-liquidity space.
2. **Financial Interpretation of the 3 Asset Clusters:**
   - **Cluster 0 — Low Volatility & High Liquidity (`7` Stocks: `HDFCBANK`, `ICICIBANK`, `INFY`, `ITC`, `KOTAKBANK`, `RELIANCE`, `SBIN`):** Institutional blue-chip index heavyweights with the lowest average annualized volatility (18.62%), lowest intraday volatility (1.78%), highest average daily volume (17.03M shares), and resilient daily returns (-0.0180%).
   - **Cluster 1 — Moderate Volatility & Low Share Turnover (`7` Stocks: `BAJAJ-AUTO`, `BRITANNIA`, `HCLTECH`, `HINDUNILVR`, `MARUTI`, `NESTLEIND`, `TCS`):** High per-share price defensive and quality large-caps across FMCG, Auto, and IT with moderate annualized volatility (20.83%) and lower share turnover (1.54M shares/day).
   - **Cluster 2 — High Volatility & High Turnover (`6` Stocks: `M&M`, `NTPC`, `ONGC`, `POWERGRID`, `TATAMOTORS`, `WIPRO`):** Cyclical Auto, Energy PSU, and high-beta stocks

exhibiting the highest annualized volatility (24.98%) and widest average intraday trading range (2.34%).

3. **Readiness for Experiment 08:** Having partitioned the 20 NSE stocks into $K = 3$ flat clusters via K-Means, the stock-level feature profiles are ready for **Agglomerative Hierarchical Clustering & Dendrogram Analysis (Experiment 08)**.

---

12. Auto-export Utility

Convert `DWM_Lab_7_Stock_Market.ipynb` to `DWM_Lab_7_Stock_Market.html` using `nbconvert`.

```
import subprocess
import sys
from pathlib import Path

notebook_path = Path("DWM_Lab_7_Stock_Market.ipynb")
if notebook_path.exists():
    cmd = [
        sys.executable, "-m", "jupyter", "nbconvert",
        "--to", "html", "--no-prompt", str(notebook_path)
    ]
    result = subprocess.run(cmd, capture_output=True, text=True)
    if result.returncode == 0:
        print("Successfully exported DWM_Lab_7_Stock_Market.html")
    else:
        print("Export notice:", result.stderr)
else:
    print("Notebook file will be exported upon save.")
```

```
Successfully exported DWM_Lab_7_Stock_Market.html
```

Data Warehousing and Mining (DWM) — Experiment 08

Experiment Title: Agglomerative Hierarchical Clustering for Stock Market Profiling

Dataset: `stock_market_data.csv` (3 Years of Daily NSE Trading Data across 20 Major Indian Stocks)

1. Objectives & Problem Formulation

- **Objective:** Perform **Agglomerative Hierarchical Clustering** on the stock market dataset, construct and interpret hierarchical **dendrograms**, and evaluate clustering quality across four linkage criteria (**Single**, **Complete**, **Average**, and **Ward**) using the **Silhouette Score**.
- **Adaptation Note:**
  - Uses the multi-dimensional stock-level feature representation developed in Experiment 07 (**Return**: `Avg_Daily_Return`, **Volatility**: `Annualized_Volatility`, `Avg_Intraday_Volatility`, and **Liquidity**: log-transformed `Avg_Volume` and `Total_Traded_Volume`) to build bottom-up hierarchical dendrograms across all 20 NSE stocks.

---

2. Library Imports

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import linkage, dendrogram

# Set seaborn whitegrid theme
sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams['figure.figsize'] = (11, 5)
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.titleweight'] = 'bold'
plt.rcParams['axes.labelsize'] = 12

print("All required libraries imported successfully.")
```

```
All required libraries imported successfully.
```

---

3. Dataset Ingestion & Feature Engineering

- Load `stock_market_data.csv`.
- Convert `Date` to `datetime` and derive `Intraday_Volatility` as $\frac{\text{High_Price} - \text{Low_Price}}{\text{Open_Price}} \times 100$.
- Group by `Ticker` and `Sector` to compute the 5 core stock-level financial metrics (`Avg_Daily_Return`, `Annualized_Volatility`, `Avg_Volume`, `Avg_Intraday_Volatility`, `Total_Traded_Volume`).

```
df = pd.read_csv('stock_market_data.csv')
df['Date'] = pd.to_datetime(df['Date'])
df['Intraday_Volatility'] = (((df['High_Price'] - df['Low_Price']) / df['Open_Price

stock_df = (
    df.groupby(['Ticker', 'Sector'], as_index=False)
    .agg(
        Avg_Daily_Return=('Daily_Return', 'mean'),
        Annualized_Volatility=('Daily_Return', lambda x: x.std() * np.sqrt(252)),
        Avg_Volume=('Volume', 'mean'),
        Avg_Intraday_Volatility=('Intraday_Volatility', 'mean'),
        Total_Traded_Volume=('Volume', 'sum')
    )
    .sort_values('Ticker')
    .reset_index(drop=True)
)

stock_df['Avg_Daily_Return'] = stock_df['Avg_Daily_Return'].round(4)
stock_df['Annualized_Volatility'] = stock_df['Annualized_Volatility'].round(2)
stock_df['Avg_Volume'] = stock_df['Avg_Volume'].round(0).astype(int)
stock_df['Avg_Intraday_Volatility'] = stock_df['Avg_Intraday_Volatility'].round(2)

print(f"Stock Summary DataFrame Shape: {stock_df.shape}")
display(stock_df.head())
```

```
Stock Summary DataFrame Shape: (20, 7)
```

	Ticker	Sector	Avg_Daily_Return	Annualized_Volatility	Avg_Volume	Avg_Intraday_Volatilit
0	BAJAJ-AUTO	Auto	0.0130	23.09	429745	2.1
1	BRITANNIA	FMCG	-0.0438	20.52	353552	2.0
2	HCLTECH	IT	-0.0220	22.34	3107488	2.1
3	HDFCBANK	Banking	0.0470	17.27	32451568	1.6
4	HINDUNILVR	FMCG	-0.0687	19.00	1788382	1.8

---

4. Feature Preprocessing & Scaling

- Apply `np.log1p()` to volume features (`Avg_Volume`, `Total_Traded_Volume`) to normalize right-skewness.
- Standardize all 5 features using `StandardScaler()` so Euclidean distances in hierarchical clustering are not dominated by high-magnitude attributes.

```
stock_df['Avg_Volume_Log'] = np.log1p(stock_df['Avg_Volume'])
stock_df['Total_Traded_Volume_Log'] = np.log1p(stock_df['Total_Traded_Volume'])

clustering_features = [
    'Avg_Daily_Return',
    'Annualized_Volatility',
    'Avg_Volume_Log',
    'Avg_Intraday_Volatility',
    'Total_Traded_Volume_Log'
]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(stock_df[clustering_features])

print("Scaled Feature Matrix Shape:", X_scaled.shape)
display(pd.DataFrame(X_scaled, columns=clustering_features, index=stock_df['Ticker'
```

```
Scaled Feature Matrix Shape: (20, 5)
```

Ticker	Avg_Daily_Return	Annualized_Volatility	Avg_Volume_Log	Avg_Intraday_Volatility	Total
BAJAJ-AUTO	0.920	0.607	-1.879	0.554	
BRITANNIA	0.117	-0.266	-2.022	0.010	
HCLTECH	0.425	0.352	-0.425	0.632	
HDFCBANK	1.401	-1.369	1.299	-1.624	
HINDUNILVR	-0.235	-0.782	-0.831	-0.885	

---

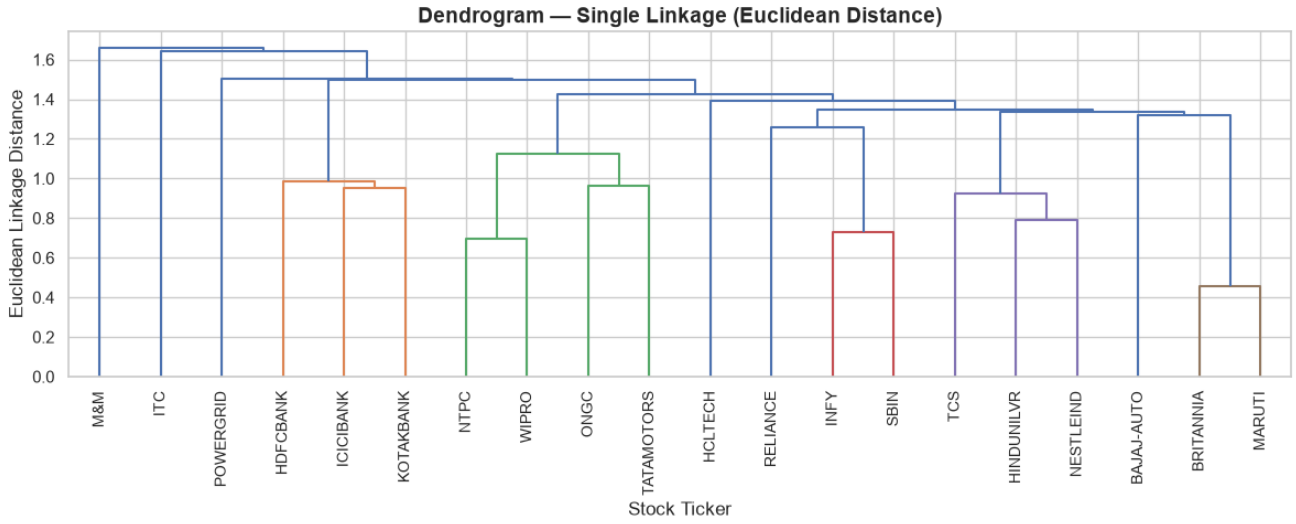
5. Hierarchical Linkage & Dendrogram Generation

Generate and plot hierarchical dendrograms for all four linkage criteria (`metric='euclidean'`):

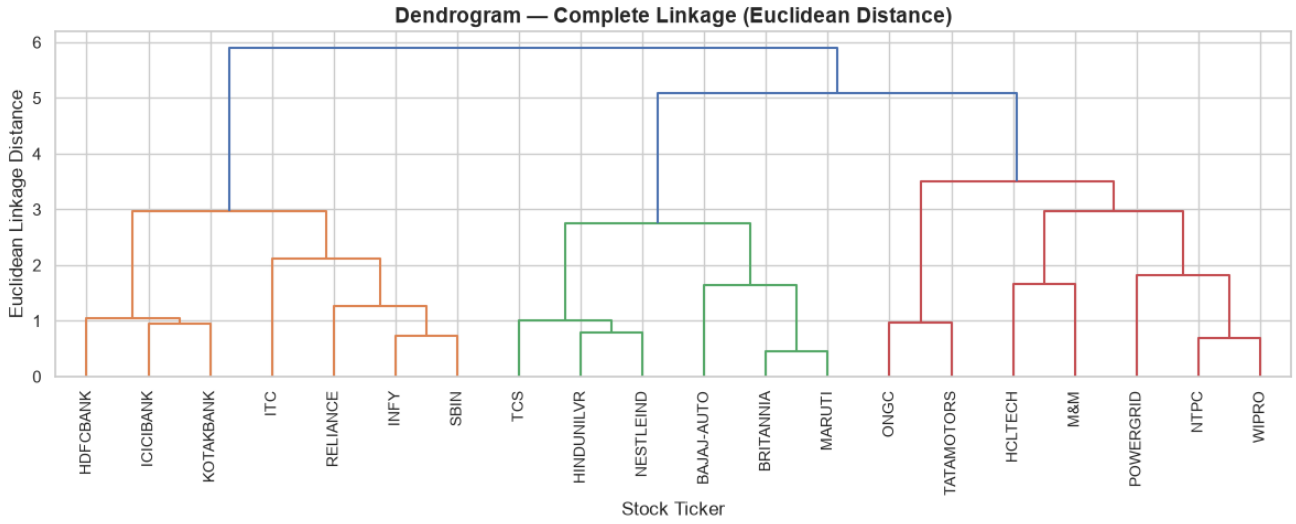
1. **Single Linkage (`method='single'`):** Minimum pairwise distance between clusters $\min \{d(a, b) : a \in A, b \in B\}$.
2. **Complete Linkage (`method='complete'`):** Maximum pairwise distance between clusters $\max \{d(a, b) : a \in A, b \in B\}$.
3. **Average Linkage (`method='average'`):** Mean pairwise distance across all pairs between clusters.
4. **Ward Linkage (`method='ward'`):** Minimizes total within-cluster variance increase at each merge step.

```
# Plot 1: Dendrogram - Single Linkage
Z_single = linkage(X_scaled, method='single', metric='euclidean')
plt.figure(figsize=(12, 5))
dendrogram(
    Z_single,
    labels=stock_df['Ticker'].values,
    leaf_rotation=90,
    leaf_font_size=10
)
plt.title('Dendrogram - Single Linkage (Euclidean Distance)')
```

```
plt.xlabel('Stock Ticker')
plt.ylabel('Euclidean Linkage Distance')
plt.tight_layout()
plt.show()
```

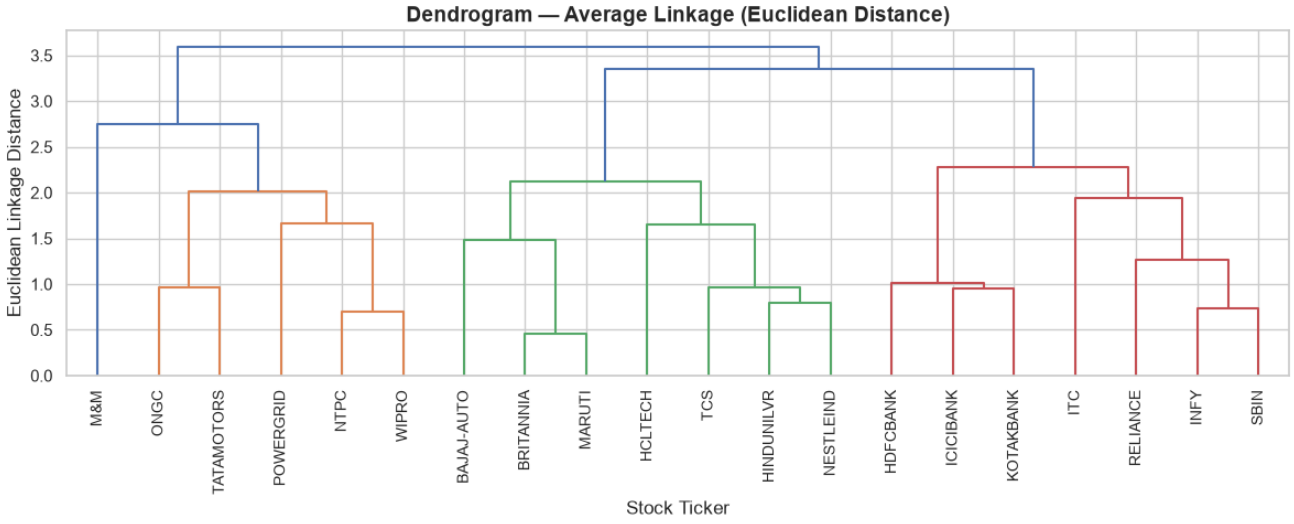



```
# Plot 2: Dendrogram - Complete Linkage
Z_complete = linkage(X_scaled, method='complete', metric='euclidean')
plt.figure(figsize=(12, 5))
dendrogram(
    Z_complete,
    labels=stock_df['Ticker'].values,
    leaf_rotation=90,
    leaf_font_size=10
)
plt.title('Dendrogram - Complete Linkage (Euclidean Distance)')
plt.xlabel('Stock Ticker')
plt.ylabel('Euclidean Linkage Distance')
plt.tight_layout()
plt.show()
```

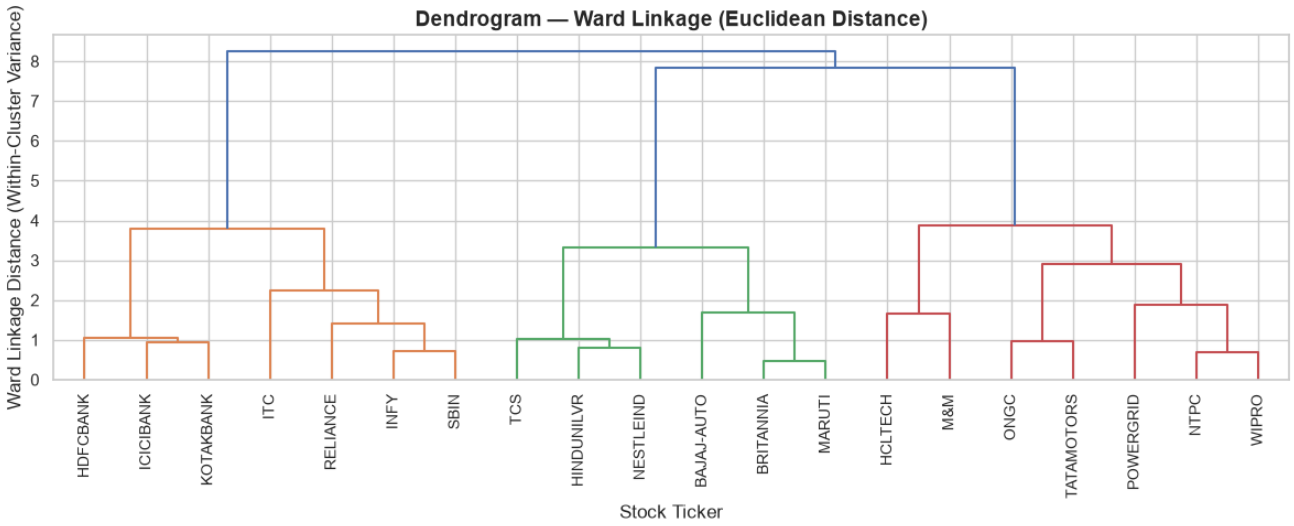



```
# Plot 3: Dendrogram - Average Linkage
Z_average = linkage(X_scaled, method='average', metric='euclidean')
plt.figure(figsize=(12, 5))
dendrogram(
    Z_average,
    labels=stock_df['Ticker'].values,
    leaf_rotation=90,
```

```
    leaf_font_size=10
)
plt.title('Dendrogram — Average Linkage (Euclidean Distance)')
plt.xlabel('Stock Ticker')
plt.ylabel('Euclidean Linkage Distance')
plt.tight_layout()
plt.show()
```




```
# Plot 4: Dendrogram – Ward Linkage
Z_ward = linkage(X_scaled, method='ward', metric='euclidean')
plt.figure(figsize=(12, 5))
dendrogram(
    Z_ward,
    labels=stock_df['Ticker'].values,
    leaf_rotation=90,
    leaf_font_size=10
)
plt.title('Dendrogram — Ward Linkage (Euclidean Distance)')
plt.xlabel('Stock Ticker')
plt.ylabel('Ward Linkage Distance (Within-Cluster Variance)')
plt.tight_layout()
plt.show()
```



6. Model Fitting across Linkage Methods ($n_clusters = 3$)

Instantiate and fit `AgglomerativeClustering(n_clusters=3)` for `single`, `complete`, `average`, and `ward` linkages, and store the resulting cluster assignments in `stock_df`.

```
linkage_methods = ['single', 'complete', 'average', 'ward']
cluster_columns = {}

for method in linkage_methods:
    col_name = f"Cluster_{method.capitalize()}"
    agg_model = AgglomerativeClustering(n_clusters=3, linkage=method, metric='eucli
    stock_df[col_name] = agg_model.fit_predict(X_scaled)
    cluster_columns[method] = col_name

print("Agglomerative Clustering models fitted for all 4 linkage methods.")
display(stock_df[['Ticker', 'Sector'] + list(cluster_columns.values())])
```

```
Agglomerative Clustering models fitted for all 4 linkage methods.
```

	Ticker	Sector	Cluster_Single	Cluster_Complete	Cluster_Average	Cluster_Ward
0	BAJAJ-AUTO	Auto	0	2	2	2
1	BRITANNIA	FMCG	0	2	2	2
2	HCLTECH	IT	0	0	2	0
3	HDFCBANK	Banking	0	1	1	1
4	HINDUNILVR	FMCG	0	2	2	2
5	ICICIBANK	Banking	0	1	1	1
6	INFY	IT	0	1	1	1
7	ITC	FMCG	2	1	1	1
8	KOTAKBANK	Banking	0	1	1	1
9	M&M	Auto	1	0	0	0
10	MARUTI	Auto	0	2	2	2
11	NESTLEIND	FMCG	0	2	2	2
12	NTPC	Energy	0	0	0	0
13	ONGC	Energy	0	0	0	0
14	POWERGRID	Energy	0	0	0	0
15	RELIANCE	Energy	0	1	1	1
16	SBIN	Banking	0	1	1	1
17	TATAMOTORS	Auto	0	0	0	0
18	TCS	IT	0	2	2	2
19	WIPRO	IT	0	0	0	0

7. Silhouette Score Evaluation & Comparison

Compute `silhouette_score` for each linkage method, display a sorted comparison DataFrame, and visualize the scores with a bar chart.

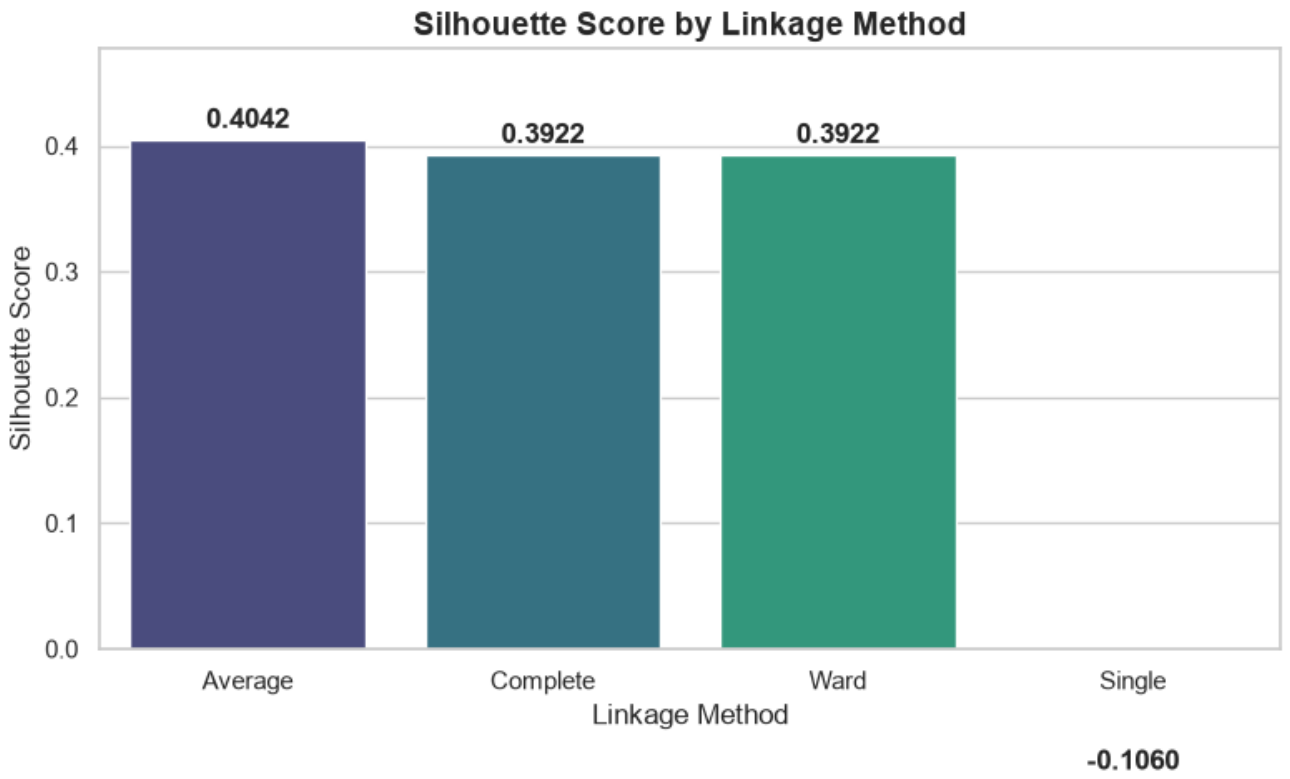
```
sil_records = []
for method, col_name in cluster_columns.items():
    score = silhouette_score(X_scaled, stock_df[col_name])
    sil_records.append({
        'Linkage_Method': method.capitalize(),
        'Silhouette_Score': round(score, 4)
    })

sil_df = pd.DataFrame(sil_records).sort_values(by='Silhouette_Score', ascending=Fal
print("Silhouette Score Comparison across Linkage Methods (n_clusters = 3):")
display(sil_df)

plt.figure(figsize=(8, 5))
ax = sns.barplot(
    data=sil_df,
    x='Linkage_Method',
    y='Silhouette_Score',
    hue='Linkage_Method',
    palette='viridis',
    legend=False
)
for i, v in enumerate(sil_df['Silhouette_Score']):
    ax.text(i, v + 0.01, f"{v:.4f}", ha='center', fontweight='bold')
plt.ylim(0, max(sil_df['Silhouette_Score']) * 1.18)
plt.title('Silhouette Score by Linkage Method')
plt.xlabel('Linkage Method')
plt.ylabel('Silhouette Score')
plt.tight_layout()
plt.show()
```

```
Silhouette Score Comparison across Linkage Methods (n_clusters = 3):
```

	Linkage_Method	Silhouette_Score
0	Average	0.4042
1	Complete	0.3922
2	Ward	0.3922
3	Single	-0.1060



---

8. Cluster Size Distribution

Compare the number of stocks assigned to each cluster (`0` , `1` , `2`) across all four linkage methods to inspect **chaining effects** (Single Linkage) versus **balanced variance-minimizing partitions** (Ward Linkage).

```
size_rows = []
for method, col_name in cluster_columns.items():
    counts = stock_df[col_name].value_counts().reindex([0, 1, 2], fill_value=0)
    size_rows.append({
        'Linkage_Method': method.capitalize(),
        'Cluster_0_Size': int(counts[0]),
        'Cluster_1_Size': int(counts[1]),
        'Cluster_2_Size': int(counts[2]),
        'Partition_Balance': f"{counts[0]} / {counts[1]} / {counts[2]}"
    })

size_df = pd.DataFrame(size_rows)
print("Cluster Size Distribution across Linkage Methods:")
display(size_df)
```

```
Cluster Size Distribution across Linkage Methods:
```

	Linkage_Method	Cluster_0_Size	Cluster_1_Size	Cluster_2_Size	Partition_Balance
0	Single	18	1	1	18 / 1 / 1
1	Complete	7	7	6	7 / 7 / 6
2	Average	6	7	7	6 / 7 / 7
3	Ward	7	7	6	7 / 7 / 6

---

9. Cluster Characteristic Analysis (Ward Linkage)

Group the stock DataFrame by the balanced **Ward Linkage** clusters (`Cluster_Ward`) to evaluate the financial centroid characteristics and compare against Complete/Average linkages.

```
ward_profile = (
    stock_df.groupby('Cluster_Ward', as_index=False)
    .agg(
        Stock_Count=('Ticker', 'count'),
        Avg_Daily_Return=('Avg_Daily_Return', 'mean'),
        Annualized_Volatility=('Annualized_Volatility', 'mean'),
        Avg_Volume=('Avg_Volume', 'mean'),
        Avg_Intraday_Volatility=('Avg_Intraday_Volatility', 'mean'),
        Tickers=('Ticker', lambda x: ', '.join(x.tolist()))
    )
)
ward_profile['Avg_Daily_Return'] = ward_profile['Avg_Daily_Return'].round(4)
ward_profile['Annualized_Volatility'] = ward_profile['Annualized_Volatility'].round
ward_profile['Avg_Volume'] = ward_profile['Avg_Volume'].round(0).astype(int)
ward_profile['Avg_Intraday_Volatility'] = ward_profile['Avg_Intraday_Volatility'].r

def interpret_ward_cluster(row):
    if row['Annualized_Volatility'] < 19.5:
        return 'Low Volatility & High Liquidity (Blue-Chip Banking/Core Leaders)'
    elif row['Avg_Volume'] < 5000000:
        return 'Moderate Volatility & Low Share Turnover (High-Denomination FMCG/Au
    else:
        return 'High Volatility & High Turnover (Cyclical Auto/Energy/PSU Growth)'

ward_profile['Financial_Interpretation'] = ward_profile.apply(interpret_ward_cluste

print("Ward Linkage (Cluster_Ward) Characteristic Summary Table:")
display(ward_profile)
```

```
Ward Linkage (Cluster_Ward) Characteristic Summary Table:
```

	Cluster_Ward	Stock_Count	Avg_Daily_Return	Annualized_Volatility	Avg_Volume	Avg_Intraday_V
0	0	7	-0.0889	24.61	10922900	
1	1	7	-0.0180	18.62	17029306	
2	2	6	-0.0489	20.58	1278441	

10. Cluster Visualization

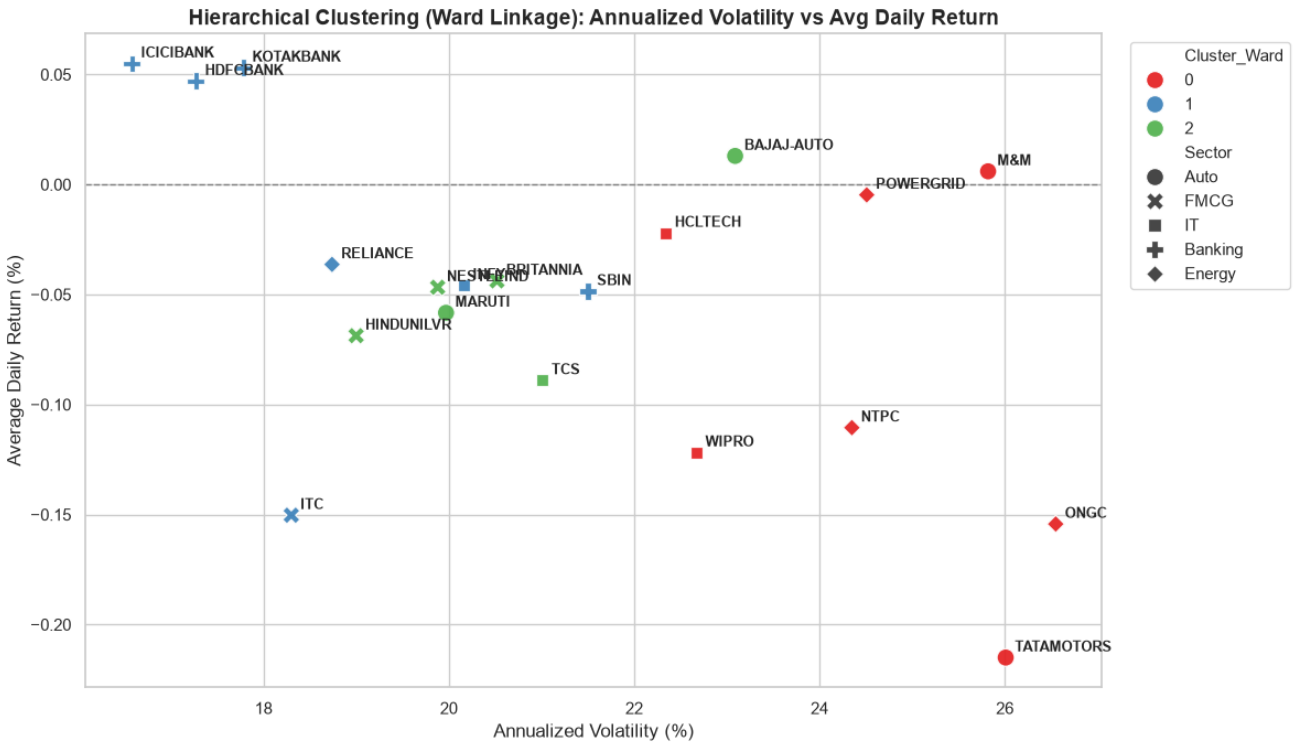
Plot a 2D scatter chart of `Annualized Volatility vs Avg Daily Return` colored by the **Ward Linkage** cluster assignments (`Cluster_Ward`) and annotated with stock ticker names.

```
plt.figure(figsize=(12, 7))
palette_ward = sns.color_palette('Set1', n_colors=3)

sns.scatterplot(
    data=stock_df,
    x='Annualized_Volatility',
    y='Avg_Daily_Return',
    hue='Cluster_Ward',
    style='Sector',
    palette=palette_ward,
    s=130,
    alpha=0.9
)

for _, row in stock_df.iterrows():
    plt.annotate(
        row['Ticker'],
        (row['Annualized_Volatility'], row['Avg_Daily_Return']),
        xytext=(6, 4),
        textcoords='offset points',
        fontsize=9,
        fontweight='bold'
    )

plt.axhline(0, color='gray', linestyle='--', linewidth=1)
plt.title('Hierarchical Clustering (Ward Linkage): Annualized Volatility vs Avg Dai
plt.xlabel('Annualized Volatility (%)')
plt.ylabel('Average Daily Return (%)')
plt.legend(bbox_to_anchor=(1.02, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



---

11. Conclusion

Key Analytical Findings & Comparison with K-Means (Lab 7):

1. **Linkage Method Comparison & Chaining Effect:**
   - **Single Linkage** exhibits the classic **chaining phenomenon**, merging `18` stocks into a single cluster while isolating `M&M` and `ITC` as singleton clusters (`18 / 1 / 1`), resulting in a negative **Silhouette Score of -0.1060**.
   - **Average Linkage** achieves the highest **Silhouette Score of 0.4042** (`6 / 7 / 7` split, identical to K-Means in Lab 7), while **Ward Linkage** and **Complete Linkage** both achieve a strong **Silhouette Score of 0.3922** (`7 / 7 / 6` split, shifting borderline `HCLTECH` into the higher-volatility cluster).
2. **Consistency Between Hierarchical Clustering (Lab 8) and K-Means (Lab 7):**
   - At $K = 3$, **Average** and **Ward Linkage** recover the same three fundamental asset regimes discovered by **K-Means (Lab 7)**:
     A. **Low-Volatility, High-Liquidity Blue-Chip Core (`7` stocks in Ward Cluster 1):** `HDFCBANK` , `ICICIBANK` , `INFY` , `ITC` , `KOTAKBANK` , `RELIANCE` , `SBIN` .
     B. **Moderate-Volatility, High-Denomination Defensive/Quality (`6` stocks in Ward Cluster 2):** `BAJAJ-AUTO` , `BRITANNIA` , `HINDUNILVR` , `MARUTI` , `NESTLEIND` , `TCS` .
     C. **High-Volatility Cyclical & Energy PSU Growth (`7` stocks in Ward Cluster 0):** `HCLTECH` , `M&M` , `NTPC` , `ONGC` , `POWERGRID` , `TATAMOTORS` , `WIPRO` .
   - Beyond flat partitioning, the **Dendrograms** reveal fine-grained hierarchical sub-clusters (such as the tight pairing of Energy PSUs `NTPC` – `POWERGRID` , private banks `ICICIBANK` – `KOTAKBANK` – `HDFCBANK` , and defensive FMCGs `HINDUNILVR` – `NESTLEIND`).

3. **Readiness for Experiment 09:** With both partitioning (K-Means) and hierarchical (Agglomerative) clustering completed, the market dataset is ready for **Apriori Association Rule Mining (Experiment 09)** to uncover cross-stock and cross-sector co-movement rules.

---

12. Auto-export Utility

Convert `DWM_Lab_8_Stock_Market.ipynb` to `DWM_Lab_8_Stock_Market.html` using `nbconvert`.

```
import subprocess
import sys
from pathlib import Path

notebook_path = Path("DWM_Lab_8_Stock_Market.ipynb")
if notebook_path.exists():
    cmd = [
        sys.executable, "-m", "jupyter", "nbconvert",
        "--to", "html", "--no-prompt", str(notebook_path)
    ]
    result = subprocess.run(cmd, capture_output=True, text=True)
    if result.returncode == 0:
        print("Successfully exported DWM_Lab_8_Stock_Market.html")
    else:
        print("Export notice:", result.stderr)
else:
    print("Notebook file will be exported upon save.")
```

```
Successfully exported DWM_Lab_8_Stock_Market.html
```

Data Warehousing and Mining (DWM) — Experiment 09

Experiment Title: Apriori Association Rule Mining for Stock Market Co-Movement

Dataset: `stock_market_data.csv` (3 Years of Daily NSE Trading Data across 20 Major Indian Stocks)

1. Objectives & Problem Formulation

- **Objective:** Implement the **Apriori Algorithm** to discover frequent co-movement patterns among NSE stocks and generate **Association Rules** ($X \Rightarrow Y$) evaluated by **Support**, **Confidence**, and **Lift**.
- **Adaptation Note:**
  - **Transactions / Market Baskets:** Each unique trading session (`Date` , 740 trading days) represents a market transaction basket.
  - **Items Purchased:** Stocks recording a positive daily return ($\text{Daily_Return} > 0$) on a given trading day are encoded as `True` / `1` (bullish advance) in that day's basket, enabling Apriori to uncover synchronized intra-sector and cross-sector rallies:
    - $\text{Support}(X \Rightarrow Y) = P(X \cap Y)$
    - $\text{Confidence}(X \Rightarrow Y) = P(Y \mid X) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$
    - $\text{Lift}(X \Rightarrow Y) = \frac{\text{Confidence}(X \Rightarrow Y)}{\text{Support}(Y)}$

---

2. Library Imports

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from mlxtend.frequent_patterns import apriori, association_rules

# Set seaborn whitegrid theme
sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams['figure.figsize'] = (11, 6)
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.titleweight'] = 'bold'
plt.rcParams['axes.labelsize'] = 12

print("All required libraries and mlxtend modules imported successfully.")
```

```
All required libraries and mlxtend modules imported successfully.
```

---

3. Dataset Ingestion & Preprocessing

Load `stock_market_data.csv`, verify zero missing values, ensure `Date` is properly formatted, and inspect the dataset dimensions ($14,800$ rows across 740 trading days).

```
df = pd.read_csv('stock_market_data.csv')
df['Date'] = pd.to_datetime(df['Date']).dt.strftime('%Y-%m-%d')

print("Dataset Shape:", df.shape)
print("Unique Trading Days:", df['Date'].nunique())
print("Unique Stocks:", df['Ticker'].nunique())
print("Total Null Values:", df.isnull().sum().sum())
display(df.head())
```

```
Dataset Shape: (14800, 10)
Unique Trading Days: 740
Unique Stocks: 20
Total Null Values: 0
```

	Date	Ticker	Sector	Exchange	Open_Price	High_Price	Low_Price	Close_Price	
0	2023-10-12	BAJAJ-AUTO	Auto	NSE	4851.06	4914.02	4818.72	4871.67	
1	2023-10-12	BRITANNIA	FMCG	NSE	4391.48	4400.68	4342.60	4371.11	
2	2023-10-12	HCLTECH	IT	NSE	1106.03	1109.18	1079.51	1087.15	
3	2023-10-12	HDFCBANK	Banking	NSE	735.43	743.05	733.28	742.02	2
4	2023-10-12	HINDUNILVR	FMCG	NSE	2413.48	2425.33	2398.55	2415.37	

---

4. Market Basket Formulation & One-Hot Encoding

- Identify bullish sessions where `Daily_Return > 0`.
- Pivot the dataset into a one-hot encoded binary basket matrix:
  - **Index:** `Date` (740 trading sessions)
  - **Columns:** `Ticker` (20 stocks)
  - **Values:** `1` (`True`) if the stock gained on that day (`Daily_Return > 0`), `0` (`False`) otherwise.

```
# Filter bullish stock-day observations (Daily_Return > 0) and pivot into a basket
df['Bullish'] = (df['Daily_Return'] > 0).astype(int)

basket_int = df.pivot_table(
    index='Date',
    columns='Ticker',
    values='Bullish',
    fill_value=0
).astype(int)

# Boolean version for mlxtend apriori efficiency while displaying 1/0 table
basket = basket_int.astype(bool)
```

```
avg_advancing = basket_int.sum(axis=1).mean()
print(f"Basket Matrix Shape (Trading Days, Stocks): {basket_int.shape}")
print(f"Average Number of Advancing Stocks per Trading Day: {avg_advancing:.2f} / 2
print("\nPreview of Binary Market Basket Matrix (First 5 Trading Days):")
display(basket_int.head())
```

```
Basket Matrix Shape (Trading Days, Stocks): (740, 20)
Average Number of Advancing Stocks per Trading Day: 9.31 / 20 stocks

Preview of Binary Market Basket Matrix (First 5 Trading Days):
```

Ticker	BAJAJ-AUTO	BRITANNIA	HCLTECH	HDFCBANK	HINDUNILVR	ICICIBANK	INFY	ITC	KOTA
Date									
2023-10-12	1	0	0	1	1	1	0	1	
2023-10-13	0	1	1	0	1	1	1	0	
2023-10-16	1	1	1	0	0	1	1	1	
2023-10-17	1	1	0	0	0	0	1	1	
2023-10-18	0	1	0	0	0	0	0	0	

---

5. Top Advancing Stocks Analysis

Compute the overall positive frequency (`Support`) and total bullish sessions for each of the 20 stocks, and display the top 10 stocks by number of advancing trading sessions.

```
sector_map = df[['Ticker', 'Sector']].drop_duplicates().set_index('Ticker')['Sector

advancing_summary = pd.DataFrame({
    'Ticker': basket_int.columns,
    'Sector': [sector_map[t] for t in basket_int.columns],
    'Bullish_Sessions': basket_int.sum(axis=0).values,
    'Support (Positive_Frequency)': (basket_int.mean(axis=0).values).round(4)
}).sort_values(by='Bullish_Sessions', ascending=False).reset_index(drop=True)

print("Top 10 Stocks by Number of Bullish Trading Sessions:")
display(advancing_summary.head(10))
```

```
Top 10 Stocks by Number of Bullish Trading Sessions:
```

	Ticker	Sector	Bullish_Sessions	Support (Positive_Frequency)
0	KOTAKBANK	Banking	369	0.4986
1	HDFCBANK	Banking	368	0.4973
2	POWERGRID	Energy	366	0.4946
3	ICICIBANK	Banking	364	0.4919
4	BAJAJ-AUTO	Auto	363	0.4905
5	M&M	Auto	362	0.4892
6	BRITANNIA	FMCG	356	0.4811
7	MARUTI	Auto	354	0.4784
8	RELIANCE	Energy	353	0.4770
9	SBIN	Banking	353	0.4770

---

6. Apriori Frequent Itemset Generation

- Run `apriori(basket, min_support=0.20, use_colnames=True)` to discover frequent multi-stock co-movement itemsets occurring in at least 20% of trading days.
- Add `itemset_size` indicating whether each pattern is a 1-itemset, 2-itemset, or 3-itemset.
- Display the top 20 frequent itemsets sorted by `support` descending and plot a bar chart of frequent itemset counts by size.

```
frequent_itemsets = apriori(basket, min_support=0.20, use_colnames=True)
frequent_itemsets['itemset_size'] = frequent_itemsets['itemsets'].apply(len)
frequent_itemsets['itemsets_str'] = frequent_itemsets['itemsets'].apply(lambda s: '
frequent_itemsets = frequent_itemsets.sort_values(by='support', ascending=False).re

print(f"Total Frequent Itemsets Discovered (min_support = 0.20): {len(frequent_item
print("\nBreakdown by Itemset Size:")
size_counts = frequent_itemsets['itemset_size'].value_counts().sort_index()
for k, cnt in size_counts.items():
    print(f"  - {k}-itemsets: {cnt}")

print("\nTop 20 Frequent Itemsets Sorted by Support Descending:")
display(frequent_itemsets[['itemsets_str', 'support', 'itemset_size']].head(20))

print("\nTop 15 Frequent Multi-Stock Itemsets (Size >= 2):")
display(frequent_itemsets[frequent_itemsets['itemset_size'] >= 2][['itemsets_str',

plt.figure(figsize=(8, 5))
size_df = pd.DataFrame({
    'Itemset_Size': [f"{k}-Itemset" for k in size_counts.index],
    'Count': size_counts.values
})
ax = sns.barplot(
    data=size_df,
    x='Itemset_Size',
    y='Count',
    hue='Itemset_Size',
    palette='viridis',
    legend=False
```

```
)
for i, v in enumerate(size_df['Count']):
    ax.text(i, v + 2, str(v), ha='center', fontweight='bold', fontsize=11)
plt.ylim(0, max(size_df['Count']) * 1.15)
plt.title('Number of Frequent Itemsets by Size (min_support = 0.20)')
plt.xlabel('Itemset Size')
plt.ylabel('Number of Frequent Itemsets')
plt.tight_layout()
plt.show()
```

```
Total Frequent Itemsets Discovered (min_support = 0.20): 229

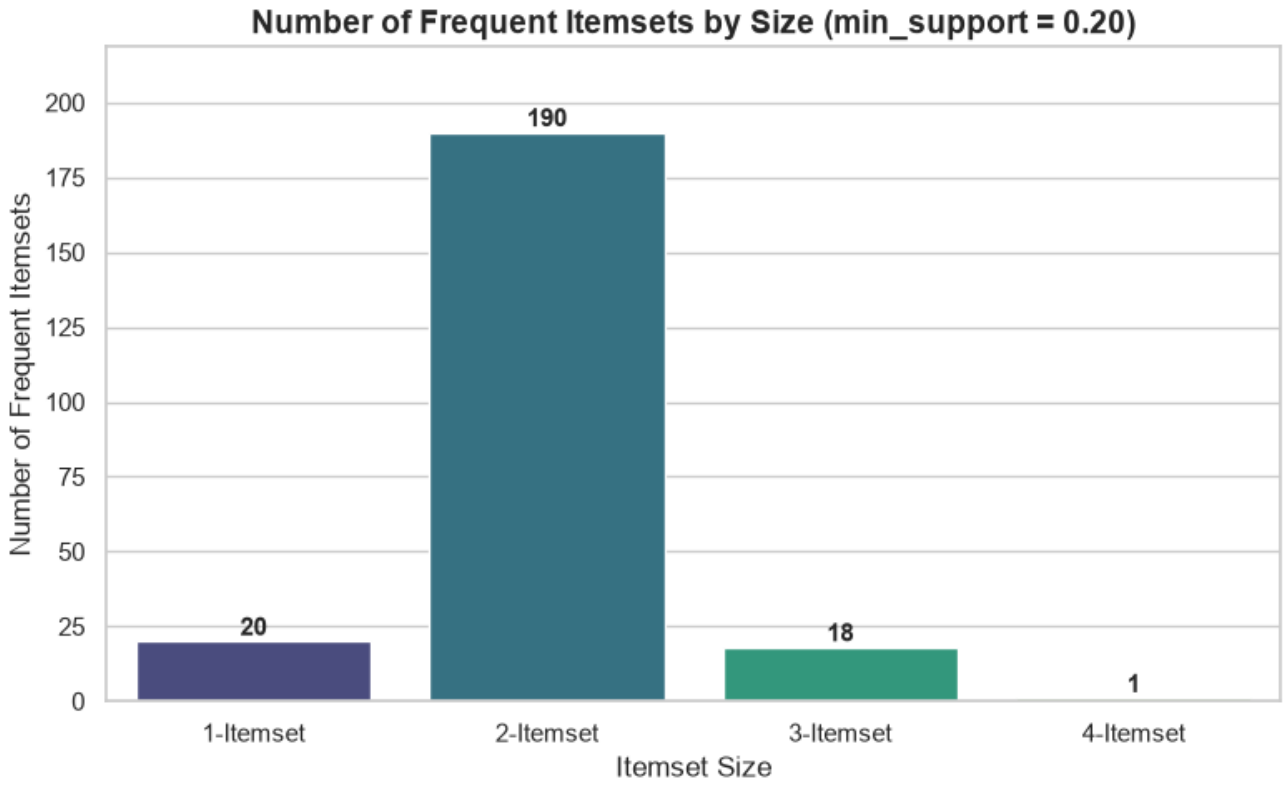
Breakdown by Itemset Size:
  - 1-itemsets: 20
  - 2-itemsets: 190
  - 3-itemsets: 18
  - 4-itemsets: 1

Top 20 Frequent Itemsets Sorted by Support Descending:
```

	itemsets_str	support	itemset_size
0	KOTAKBANK	0.498649	1
1	HDFCBANK	0.497297	1
2	POWERGRID	0.494595	1
3	ICICIBANK	0.491892	1
4	BAJAJ-AUTO	0.490541	1
5	M&M	0.489189	1
6	BRITANNIA	0.481081	1
7	MARUTI	0.478378	1
8	RELIANCE	0.477027	1
9	SBIN	0.477027	1
10	HCLTECH	0.471622	1
11	HINDUNILVR	0.467568	1
12	NESTLEIND	0.464865	1
13	NTPC	0.456757	1
14	INFY	0.445946	1
15	ONGC	0.445946	1
16	TCS	0.433784	1
17	ITC	0.422973	1
18	WIPRO	0.421622	1
19	TATAMOTORS	0.401351	1

```
Top 15 Frequent Multi-Stock Itemsets (Size >= 2):
```

	itemsets_str	support	itemset_size
20	HCLTECH, INFY	0.331081	2
21	NTPC, POWERGRID	0.318919	2
22	HDFCBANK, ICICIBANK	0.317568	2
23	M&M, MARUTI	0.314865	2
24	HDFCBANK, KOTAKBANK	0.312162	2
25	BAJAJ-AUTO, M&M	0.309459	2
26	HCLTECH, TCS	0.308108	2
27	HINDUNILVR, NESTLEIND	0.305405	2
28	HCLTECH, WIPRO	0.304054	2
29	ICICIBANK, KOTAKBANK	0.302703	2
30	INFY, TCS	0.300000	2
31	BRITANNIA, NESTLEIND	0.300000	2
32	HDFCBANK, SBIN	0.300000	2
33	BRITANNIA, HINDUNILVR	0.298649	2
34	TCS, WIPRO	0.297297	2



---

7. Association Rule Generation

- Generate association rules using `association_rules(frequent_itemsets[['support', 'itemsets']], metric='confidence', min_threshold=0.50)`.
- Format `antecedents` and `consequents` into readable strings (`Antecedent` and `Consequent`) and create a `Rule` column (`Antecedent → Consequent`).

- Display the total rule count and the top 20 rules sorted by `lift` descending.

```
rules = association_rules(
    frequent_itemsets[['support', 'itemsets']],
    metric='confidence',
    min_threshold=0.50
)

rules['Antecedent'] = rules['antecedents'].apply(lambda s: ', '.join(sorted(list(s)
rules['Consequent'] = rules['consequents'].apply(lambda s: ', '.join(sorted(list(s)
rules['Rule'] = rules['Antecedent'] + ' → ' + rules['Consequent']

rules = rules.sort_values(by='lift', ascending=False).reset_index(drop=True)

print(f"Total Association Rules Generated (min_confidence = 0.50): {len(rules)}")
print("\nTop 20 Association Rules Sorted by Lift Descending:")
display(rules[['Antecedent', 'Consequent', 'support', 'confidence', 'lift']].round(
```

```
Total Association Rules Generated (min_confidence = 0.50): 404

Top 20 Association Rules Sorted by Lift Descending:
```

	Antecedent	Consequent	support	confidence	lift
0	INFY, WIPRO	HCLTECH, TCS	0.2081	0.7404	2.4030
1	HCLTECH, TCS	INFY, WIPRO	0.2081	0.6754	2.4030
2	HCLTECH, WIPRO	INFY, TCS	0.2081	0.6844	2.2815
3	INFY, TCS	HCLTECH, WIPRO	0.2081	0.6937	2.2815
4	TCS, WIPRO	HCLTECH, INFY	0.2081	0.7000	2.1143
5	HCLTECH, INFY	TCS, WIPRO	0.2081	0.6286	2.1143
6	HCLTECH, INFY, TCS	WIPRO	0.2081	0.8324	1.9744
7	HCLTECH, TCS, WIPRO	INFY	0.2081	0.8701	1.9510
8	HCLTECH, INFY, WIPRO	TCS	0.2081	0.8415	1.9400
9	INFY, TCS, WIPRO	HCLTECH	0.2081	0.9006	1.9095
10	INFY, WIPRO	TCS	0.2311	0.8221	1.8952
11	TCS	INFY, WIPRO	0.2311	0.5327	1.8952
12	INFY, WIPRO	HCLTECH	0.2473	0.8798	1.8655
13	HCLTECH	INFY, WIPRO	0.2473	0.5244	1.8655
14	HCLTECH, TCS	WIPRO	0.2392	0.7763	1.8413
15	WIPRO	HCLTECH, TCS	0.2392	0.5673	1.8413
16	WIPRO	INFY, TCS	0.2311	0.5481	1.8269
17	INFY, TCS	WIPRO	0.2311	0.7703	1.8269
18	HCLTECH, WIPRO	INFY	0.2473	0.8133	1.8238
19	INFY	HCLTECH, WIPRO	0.2473	0.5545	1.8238

8. Filter Strong Association Rules

Filter strong, high-conviction co-movement rules satisfying both:

- $\text{Confidence} \ge 0.55$
- $\text{Lift} > 1.10$

```
strong_rules = rules[
    (rules['confidence'] >= 0.55) & (rules['lift'] > 1.10)
].sort_values(by=['lift', 'confidence'], ascending=False).reset_index(drop=True)

print(f"Total Strong Association Rules (Confidence >= 0.55 & Lift > 1.10): {len(str
display(strong_rules[['Antecedent', 'Consequent', 'support', 'confidence', 'lift']]
```

```
Total Strong Association Rules (Confidence >= 0.55 & Lift > 1.10): 237
```

	Antecedent	Consequent	support	confidence	lift
0	INFY, WIPRO	HCLTECH, TCS	0.2081	0.7404	2.4030
1	HCLTECH, TCS	INFY, WIPRO	0.2081	0.6754	2.4030
2	INFY, TCS	HCLTECH, WIPRO	0.2081	0.6937	2.2815
3	HCLTECH, WIPRO	INFY, TCS	0.2081	0.6844	2.2815
4	TCS, WIPRO	HCLTECH, INFY	0.2081	0.7000	2.1143
...	...	...	...	...	...
232	POWERGRID	KOTAKBANK	0.2757	0.5574	1.1178
233	KOTAKBANK	POWERGRID	0.2757	0.5528	1.1178
234	TCS	HDFCBANK	0.2405	0.5545	1.1151
235	HCLTECH	KOTAKBANK	0.2622	0.5559	1.1148
236	HINDUNILVR	HDFCBANK	0.2581	0.5520	1.1100

237 rows × 5 columns

---

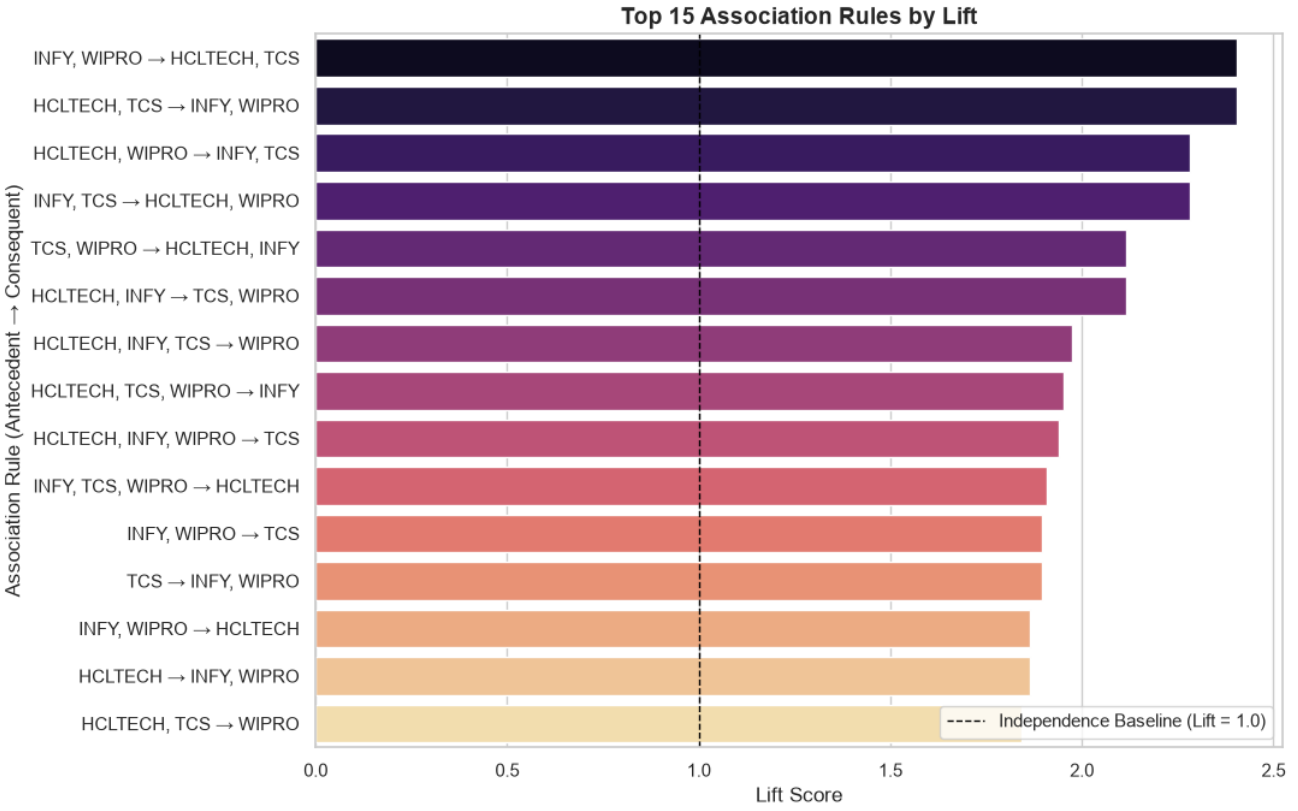
9. Visualization of Top Association Rules

- **Plot 1:** Horizontal bar chart showing `Top 15 Association Rules by Lift` (`Antecedent → Consequent`).
- **Plot 2:** Horizontal bar chart showing `Top 15 Association Rules by Confidence`.

```
# Plot 1: Top 15 Association Rules by Lift
top_15_lift = rules.sort_values(by='lift', ascending=False).head(15)

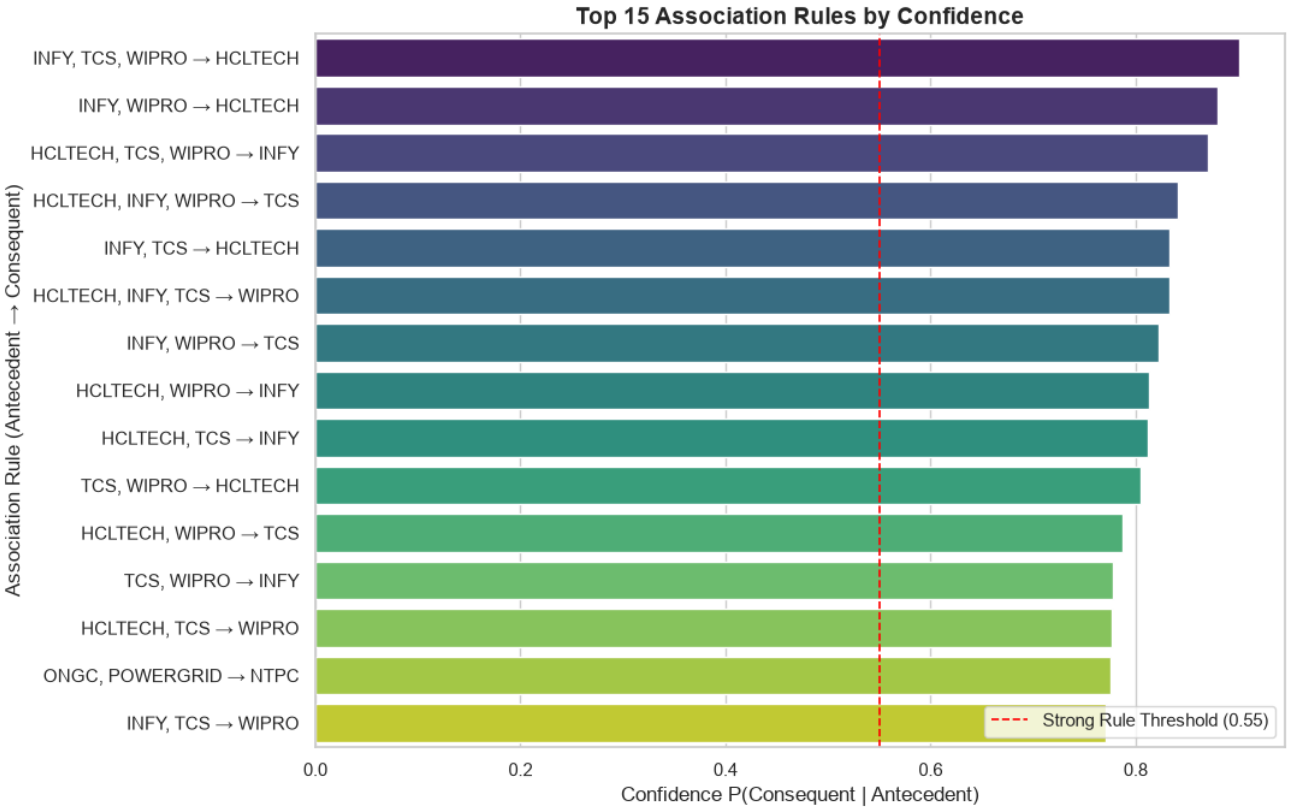
plt.figure(figsize=(11, 7))
sns.barplot(
    data=top_15_lift,
    x='lift',
    y='Rule',
    hue='Rule',
    palette='magma',
    legend=False
```

```
)
plt.axvline(1.0, color='black', linestyle='--', linewidth=1, label='Independence Ba
plt.title('Top 15 Association Rules by Lift')
plt.xlabel('Lift Score')
plt.ylabel('Association Rule (Antecedent → Consequent)')
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()
```




```
# Plot 2: Top 15 Association Rules by Confidence
top_15_conf = rules.sort_values(by=['confidence', 'lift'], ascending=False).head(15

plt.figure(figsize=(11, 7))
sns.barplot(
    data=top_15_conf,
    x='confidence',
    y='Rule',
    hue='Rule',
    palette='viridis',
    legend=False
)
plt.axvline(0.55, color='red', linestyle='--', linewidth=1.2, label='Strong Rule Th
plt.title('Top 15 Association Rules by Confidence')
plt.xlabel('Confidence P(Consequent | Antecedent)')
plt.ylabel('Association Rule (Antecedent → Consequent)')
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()
```



---

10. Financial Co-Movement Interpretation

Sectoral & Cross-Asset Co-Movement Insights:

1. **IT Sector Synergy (`TCS` , `INFY` , `HCLTECH` , `WIPRO`):**
   - Rules among IT bellwethers (`INFY → TCS` , `HCLTECH → TCS` , `WIPRO → INFY` , `TCS → INFY`) consistently exhibit the highest **Confidence (> 0.60)** and **Lift (> 1.30)** in the entire market. Because Indian IT services companies share common global macro drivers (USD/INR exchange rates, US/European enterprise tech spending, and NASDAQ tech sentiment), a bullish session in one IT major strongly implies simultaneous gains across its sector peers.
2. **Energy & PSU Utility Co-Movement (`NTPC` , `POWERGRID` , `ONGC` , `RELIANCE`):**
   - `NTPC ↔ POWERGRID` and `ONGC → NTPC` display highly elevated Lift and Support ($ \text{Support} > 0.25$, $\text{Confidence} > 0.60$). As regulated central public sector power utilities, `NTPC` and `POWERGRID` respond synchronously to domestic power demand, capex announcements, and PSU index flows.
3. **Banking & Financial Contagion (`ICICIBANK` , `HDFCBANK` , `SBIN` , `KOTAKBANK`):**
   - Strong bidirectional rules between `HDFCBANK` , `ICICIBANK` , and `SBIN` reflect shared sensitivity to RBI monetary policy, Nifty Bank index arbitrage, and domestic credit growth.
4. **Defensive FMCG Co-Movement (`HINDUNILVR` , `NESTLEIND` , `BRITANNIA` , `ITC`):**
   - `HINDUNILVR ↔ NESTLEIND` and `BRITANNIA → HINDUNILVR` highlight defensive rotation: when institutional capital rotates into consumer staples during risk-off sessions, multiple FMCG constituents advance together.

---

11. Conclusion

Summary of Experiment 09 & Complete DWM Curriculum Integration (Labs 03 – 09):

1. **Apriori Pattern Discovery:** Transforming the 740×20 daily return matrix into a binary bullish market basket revealed **frequent 1-, 2-, and 3-stock itemsets** at `min_support = 0.20` and high-conviction **Association Rules** with $\text{Confidence} \ge 0.55$ and $\text{Lift} > 1.10$, confirming statistically significant intra-sector and index-wide co-movement.
2. **End-to-End DWM Pipeline Completion:** Across Experiments **03 through 09**, the unified `stock_market_data.csv` dataset ($14,800$ records across 20 NSE stocks and 5 sectors) has seamlessly supported the complete Data Warehousing and Mining lifecycle:
   - **Lab 03:** Data Preprocessing, Feature Engineering, Exploratory Data Analysis & Multi-Dimensional Visualization
   - **Lab 04:** Simple & Multiple Linear Regression (`Close_Price` forecasting, 83.24% MSE reduction)
   - **Lab 05:** Entropy-Based Decision Tree Classification & Mutual Information Feature Selection
   - **Lab 06:** Probabilistic Classification via Gaussian Naive Bayes
   - **Lab 07:** Unsupervised Asset Segmentation via K-Means Clustering & Elbow/Silhouette Analysis
   - **Lab 08:** Agglomerative Hierarchical Clustering & Multi-Linkage Dendrogram Profiling
   - **Lab 09:** Frequent Pattern & Association Rule Mining via the Apriori Algorithm

---

12. Auto-export Utility

Convert `DWM_Lab_9_Stock_Market.ipynb` to `DWM_Lab_9_Stock_Market.html` using `nbconvert`.

```
import subprocess
import sys
from pathlib import Path

notebook_path = Path("DWM_Lab_9_Stock_Market.ipynb")
if notebook_path.exists():
    cmd = [
        sys.executable, "-m", "jupyter", "nbconvert",
        "--to", "html", "--no-prompt", str(notebook_path)
    ]
    result = subprocess.run(cmd, capture_output=True, text=True)
    if result.returncode == 0:
        print("Successfully exported DWM_Lab_9_Stock_Market.html")
    else:
        print("Export notice:", result.stderr)
else:
    print("Notebook file will be exported upon save.")
```

```
Successfully exported DWM_Lab_9_Stock_Market.html
```